

Bios 740- Chapter 4. Sequence Modeling: RNNs, LSTM, and GRU

Acknowledgement: Thanks to Mr. Shuai Huang for preparing some of the slides! I copied some pictures from the lecture presentations of StanfordCS224n. I also use some materials generated by chapgpt.

Content

0 Motivation to Sequence Modeling

1 Introduction to Language Modeling

2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 LSTM and GRU

5 Genomic Sequence Analysis

Content

0 Motivation to Sequence Modeling

1 Introduction to Language Modeling

2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 Extensions of RNNs

5 Genomic Sequence Analysis

Motivation

Recurrent Neural Networks (RNNs) are motivated by their ability to address challenges inherent in sequential data.



Weather
forecasting



Stock market
trends



Autocomplete
for texting



Genetic
sequencing

Many real-world datasets are inherently sequential, where **the order of data points is crucial**.

- ❖ **Time series:** Stock prices, weather forecasts, and sensor data require capturing patterns over time.
- ❖ **Text:** The meaning of a sentence depends on word order ("The cat chased the dog" vs. "The dog chased the cat").
- ❖ **Speech:** Phonemes and intonation must be processed sequentially to understand spoken language.
- ❖ **Video:** Frames in a video sequence have temporal relationships that determine the flow of events.



Speech
recognition



Video frame
prediction



Music
composition



...

Sequences

A **sequence** is an **ordered list of elements**, where the order of the elements matters. They are fundamentally different from unordered data because each element is **dependent** or **influenced** by the previous elements.

Examples:

- Letters (words)
- Words (sentences)
- Sentences (documents)
- Frames (video)
- Amino-acids (genetic code)
- fMRI/ECG signals

"Hello, how are you?" (chatbot input).

e.g., A security camera capturing a person walking.

e.g., "ACGTAGCTAGT" represents a biological sequence.

Why Are Sequences Important?

Unlike independent data points, sequences contain **temporal or contextual dependencies**:

- **Future values depend on past values** (e.g., predicting tomorrow's weather).
- **Words in a sentence rely on context** (e.g., in "bank deposit" vs. "river bank").
- **Biological sequences determine genetic functions.**

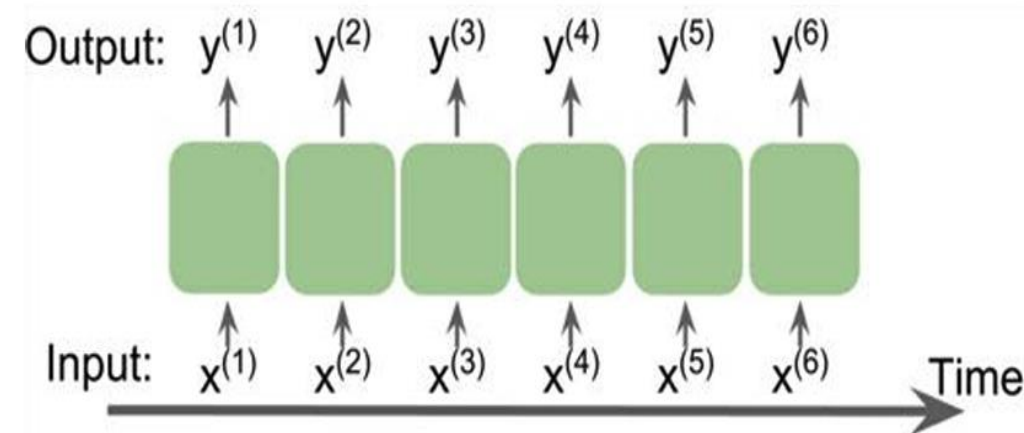
Sequences

Challenges in modeling sequential data

- **Infinite number of possible sequences:**
 - ❖ Sequences can vary in **length** (short vs. long sequences).
 - ❖ Sequences can have **variable patterns** (e.g., DNA sequences, language models).
 - ❖ Order **matters**, meaning different orders of the same elements can have different meanings.
- **Need for Probability Distributions Over Sequences:**
 - Since an infinite number of sequences exist, we **cannot store all possible sequences explicitly**.
 - Instead, we model a **probabilistic function** that assigns a likelihood to each possible sequence.
 - Example: Given a sequence $S=(x_1, x_2, \dots, x_T)$, we want to learn a **probability distribution** $P(S)$.

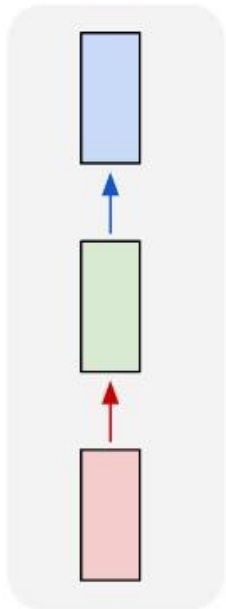
RNNs are designed for modeling sequences

- Sequences of any length in the input, in the output, or in both
- They can remember past information
- Apply the same weights on each step



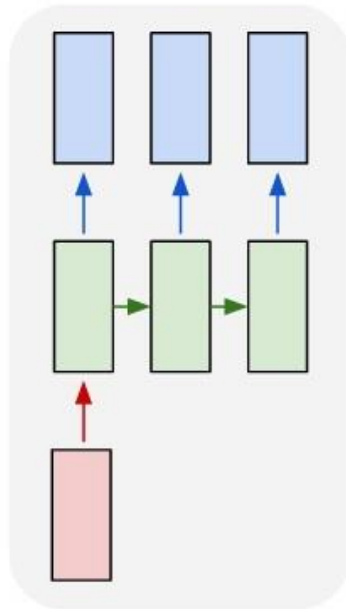
Different Categories of Sequence Modeling

one to one



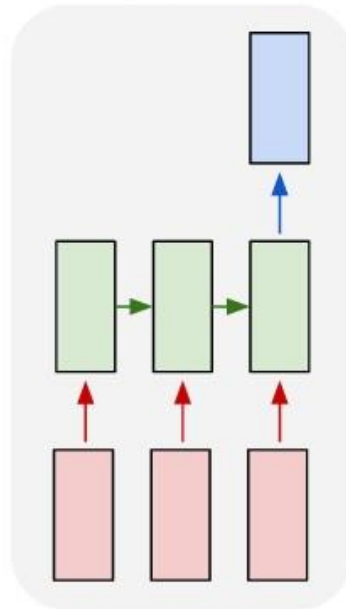
Vanilla mode without RNN
e.g. image classification

one to many



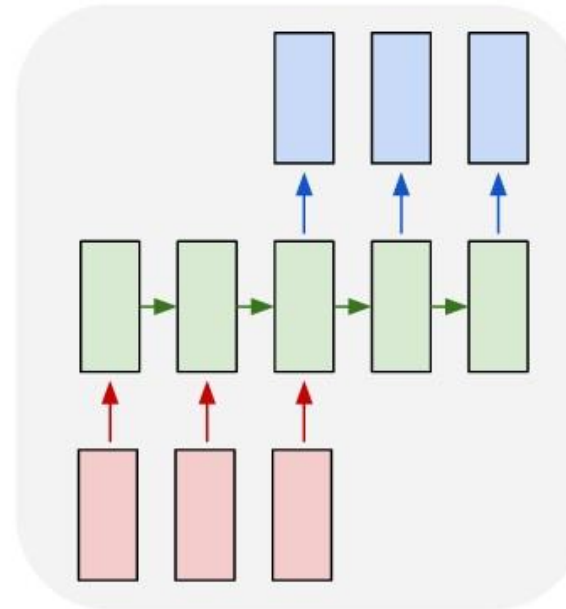
Sequence output
e.g., image captioning

many to one



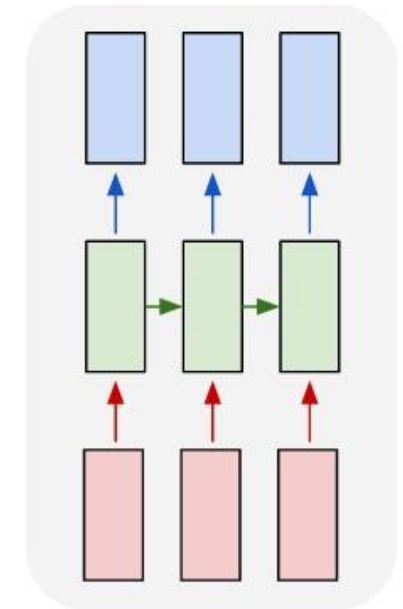
Sequence input
e.g., sentiment analysis

many to many



Sequence input and output
e.g., machine translation

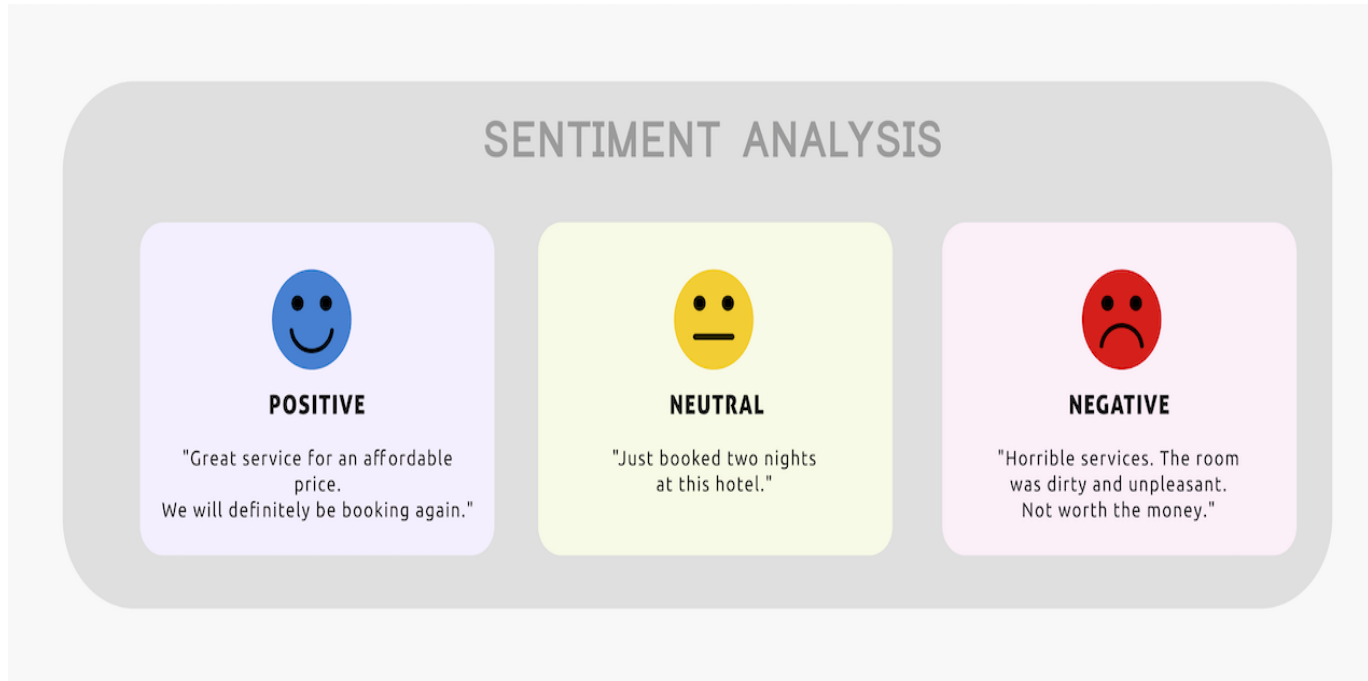
many to many



Synced sequence input and output
e.g., video classification

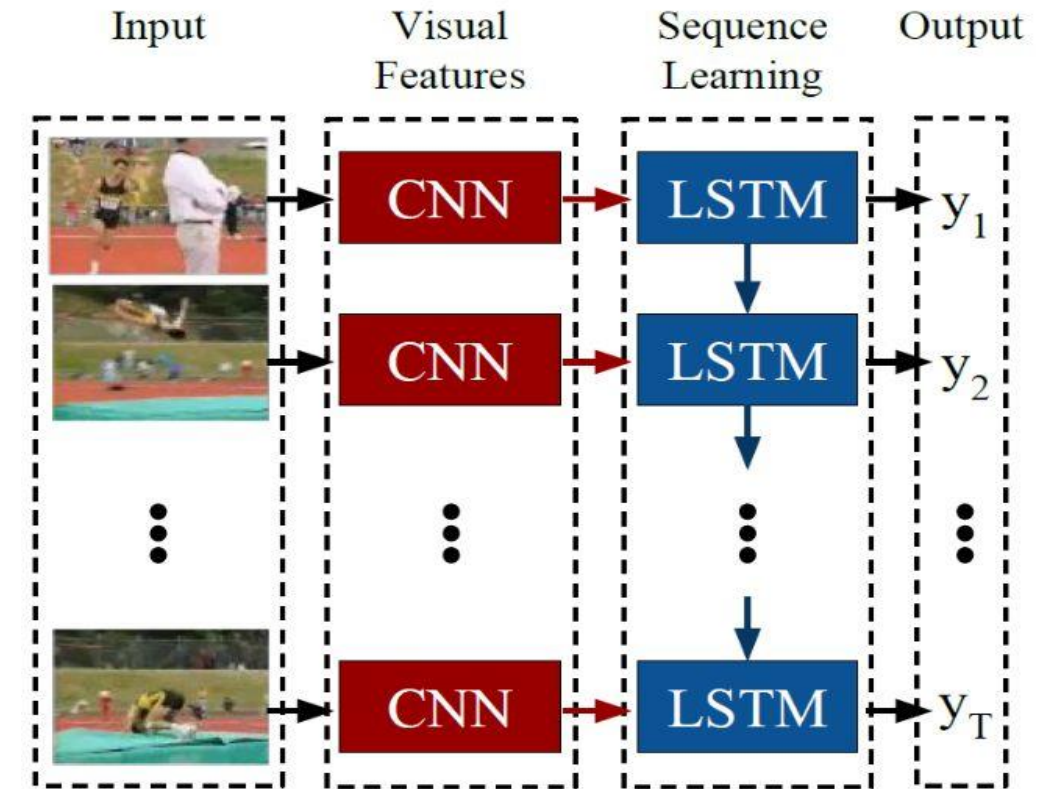
Source: <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Sentiment Analysis and Video Analytics



Sentiment Analysis is a **NLP** technique used to determine the emotional tone of a given text. It helps identify whether the sentiment of the text is **positive, negative, or neutral**.

<https://www.gosmar.eu/machinelearning/2020/08/23/recurrent-neural-networks-for-sentiment-analysis/>



Video analytics enable machines to **recognize actions, objects, and scenes** in videos.

<https://imerit.net/blog/using-neural-networks-for-video-classification-blog-all-pbm/>

Machine Translation

Machine Translation (MT) is the task of translating a sentence x from one language (the **source language**) to a sentence y in another language (the **target language**).

Goal: Produce translations that are both fluent and faithful to the meaning of the source text.

Applications: Global communication, localization cross-lingual information retrieval, etc.

x : *L'homme est né libre, et partout il est dans les fers*

English:

y : Man is born free, but everywhere he is in chains

Chinese:

“人生而自由，但无处不在被枷锁束缚。”

Japanese:

「人間は自由に生まれるが、どこにいても鎖に縛られている。」

The early history of MT: 1950s



Rule-Based MT: Early systems used manually-crafted rules and bilingual dictionaries.

StanfordCS224n

1990s-2010s: Statistical Machine Translation

Core idea: Utilized probabilistic models (e.g., IBM Models) and phrase-based translation. Relied on large parallel corpora to learn translation probabilities

- Suppose we're translating French $\rightarrow$ English.
- We want to find **best English sentence** y , given **French sentence** x

$$\operatorname{argmax}_y P(y|x)$$

- Use Bayes Rule to break this down into **two components** to be learned separately:

$$= \operatorname{argmax}_y \underbrace{P(x|y)}_{\text{Translation Model}} \underbrace{P(y)}_{\text{Language Model}}$$

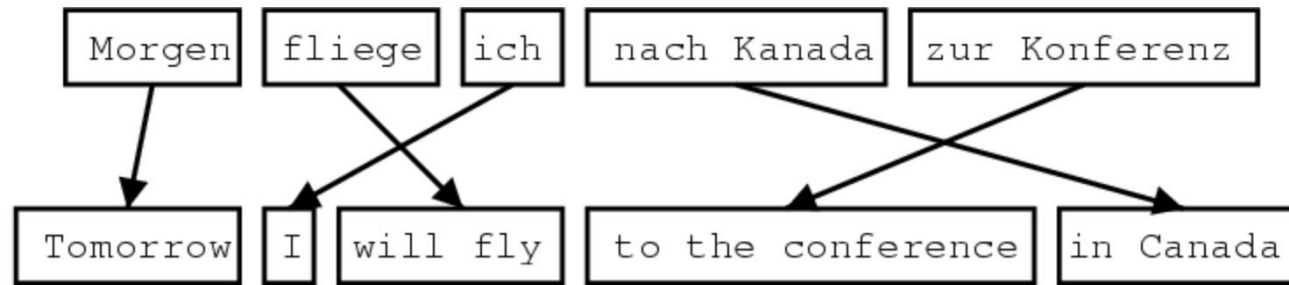
Translation Model

Models how words and phrases should be translated (*fidelity*).
Learned from parallel data.

Language Model

Models how to write good English (*fluency*).
Learned from monolingual data.

Not trivial to model!



1519年600名西班牙人在墨西哥登陆，去征服**几百万人口**
的阿兹特克帝国，初次交锋他们损兵三分之二。

In 1519, six hundred Spaniards landed in Mexico to conquer **the Aztec Empire** **with a population of a few million**. They lost two thirds of their soldiers in the first clash.

translate.google.com (2009): 1519 600 Spaniards landed in Mexico, **millions of people to conquer the Aztec empire**, the first two-thirds of soldiers against their loss.

translate.google.com (2013): 1519 600 Spaniards landed in Mexico **to conquer the Aztec empire, hundreds of millions of people**, the initial confrontation loss of soldiers two-thirds.

translate.google.com (2015): 1519 600 Spaniards landed in Mexico, **millions of people to conquer the Aztec empire**, the first two-thirds of the loss of soldiers they clash.

[StanfordCS224n](#)

1990s–2010s: Statistical Machine Translation

- ▶ SMT was once a huge research field aimed at automatically translating text between languages. It relied on probabilistic models and large parallel corpora to learn translation patterns.
- ▶ Despite its success in the past, SMT required extensive manual design and engineering. The best SMT systems were extremely complex, involving hundreds of important details. Systems were built from many separately-designed subcomponents, each addressing a specific aspect of translation. Every component was carefully engineered to optimize the overall translation quality.
- ▶ SMT required extensive feature engineering to capture specific language phenomena: Designing features to model syntax, semantics, and context. Crafting features to capture idiomatic expressions and local linguistic patterns. Each feature was manually designed, tested, and fine-tuned. This process was both time-consuming and highly dependent on expert knowledge.
- ▶ SMT systems often required compiling and maintaining extra resources: Tables of equivalent phrases, bilingual dictionaries, and syntactic rules. Language-specific resources had to be built and maintained. A large amount of human effort was needed to manage these resources. The process had to be repeated for each language pair, making it labor-intensive and costly.

2014

Neural Machine Translation

MT research

StanfordCS224n

(dramatic reenactment)

NMT: the first big success story

Neural Machine Translation (NMT): Uses deep learning and end-to-end training to model translation and offers improved fluency and the ability to capture complex dependencies.

Neural Machine Translation went from a **fringe research attempt** in **2014** to the **leading standard method** in **2016**

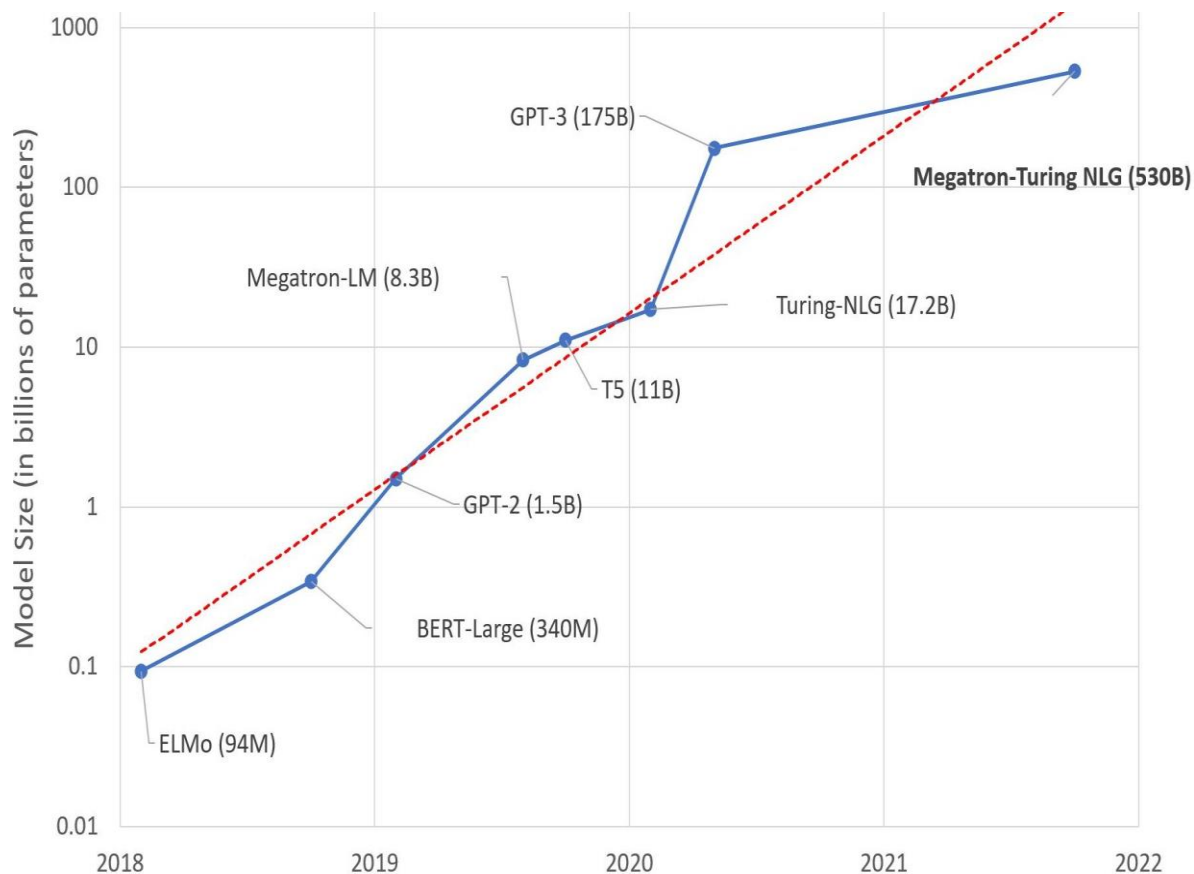
- **2014:** First seq2seq paper published [Sutskever et al. 2014]
- **2016:** Google Translate switches from SMT to NMT – and by 2018 everyone had
 - <https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html>



- **This was amazing!**
 - SMT systems, built by **hundreds** of engineers over many **years**, were outperformed by NMT systems trained by **small groups** of engineers in a few **months**

StanfordCS224n

Deepseek v.s. OpenAI



Modern NLP Systems

<https://ai.plainenglish.io/deepseek-r1-vs-chatgpt-01-my-experience-ddbe09e80aa9>

<https://huggingface.co/blog/large-language-models>

Challenges

- Standard NN models (MLPs, CNNs) are not able to handle sequences of data
 - ❖ They accept a **fixed-sized vector** as input and produce a **fixed-sized vector** as output.
 - ❖ The **weights are updated independently**, meaning there is **no memory** of past computations.
 - ❖ The models **do not have recurrence**, so they cannot learn patterns across time steps.
- Many real-world problems require capturing **context over time**:
 - ❖ **Speech Recognition** – Words depend on previous words.
 - ❖ **Time-series Prediction** – Future values depend on past observations.
 - ❖ **DNA Sequencing** – Genetic patterns unfold over long sequences.
 - ❖ **Natural Language Processing (NLP)** – Meaning depends on word order.
- **Example: Simple Context-Dependent Problem:** Output YES if the number of 1s in the sequence is even; otherwise, output NO.
 - Input: 1000010101 → **YES**; Input: 100011 → **NO**

Challenges

High Dimensionality and Complexity - Sequential data often involves high-dimensional inputs with complex interdependencies:

- ❖ **Text:** Words and phrases have semantic and syntactic relationships across sentences.
- ❖ **Time Series:** Multivariate time series data (e.g., temperature, humidity, and pressure) exhibit interdependencies between variables over time.
- ❖ **Biological Data:** DNA sequences and protein structures involve intricate, sequential patterns.

Solution: RNNs address this by learning hierarchical representations through their recurrent structure, encoding both local and global patterns.

Noise and Missing Data - Sequential data often contains noise or missing values:

- **Noise:** Sensor readings and time series data may have irregularities or anomalies.
- **Missing values:** Gaps in sequences arise from interruptions in data collection.

Solution: RNNs aggregate information over time, making them robust to noise and capable of interpolating missing values using contextual information.

Challenges

Temporal Dependencies

- ❖ **Short-term dependencies:** In text, the current word depends on immediately preceding words (e.g., ``I want to eat a...’’).
- ❖ **Long-term dependencies:** Distant elements in the sequence can influence the current state (e.g., in a paragraph, the topic sentence affects subsequent sentences).

Solution: RNNs maintain memory through hidden states, enabling them to model temporal dependencies. Variants like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) address challenges such as vanishing gradients, allowing effective modeling of long-term dependencies.

Variable-Length Inputs and Outputs - Many real-world tasks involve sequences of varying lengths, which traditional models struggle to handle. RNNs process inputs dynamically, making them ideal for tasks with variable-length data.

Examples:

- ❖ **Natural Language Processing (NLP):** Sentences have varying word counts, and RNNs can process each word without requiring fixed input dimensions.
- ❖ **Speech Recognition:** Audio recordings vary in duration depending on the speaker or content.
- ❖ **Time Series:** Data collected over irregular time intervals often results in sequences of differing lengths.

Content

0 Motivation to Sequence Modeling

1 Introduction to Language Modeling

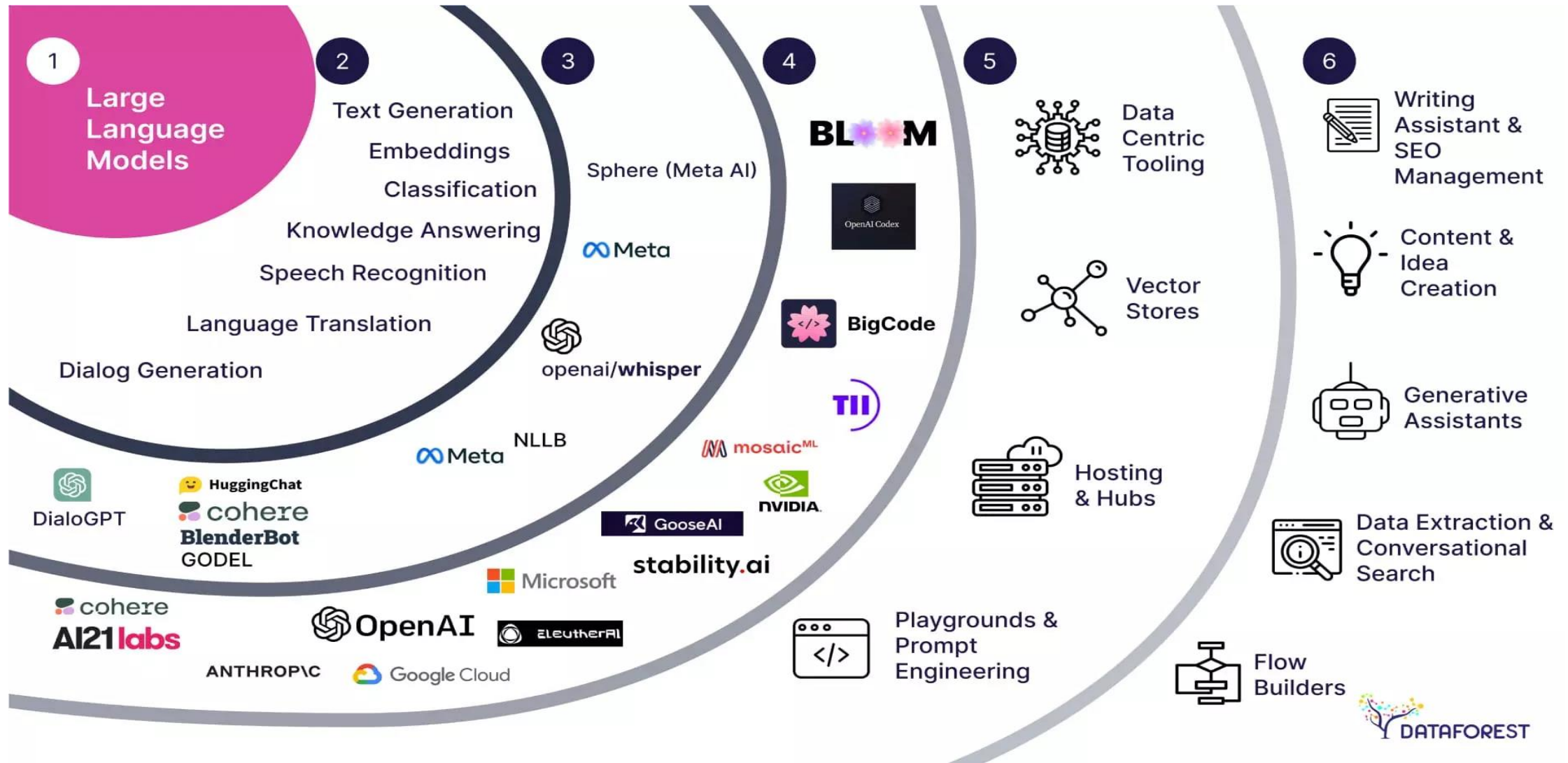
2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 Extensions of RNNs

5 Genomic Sequence Analysis

Large Language Models

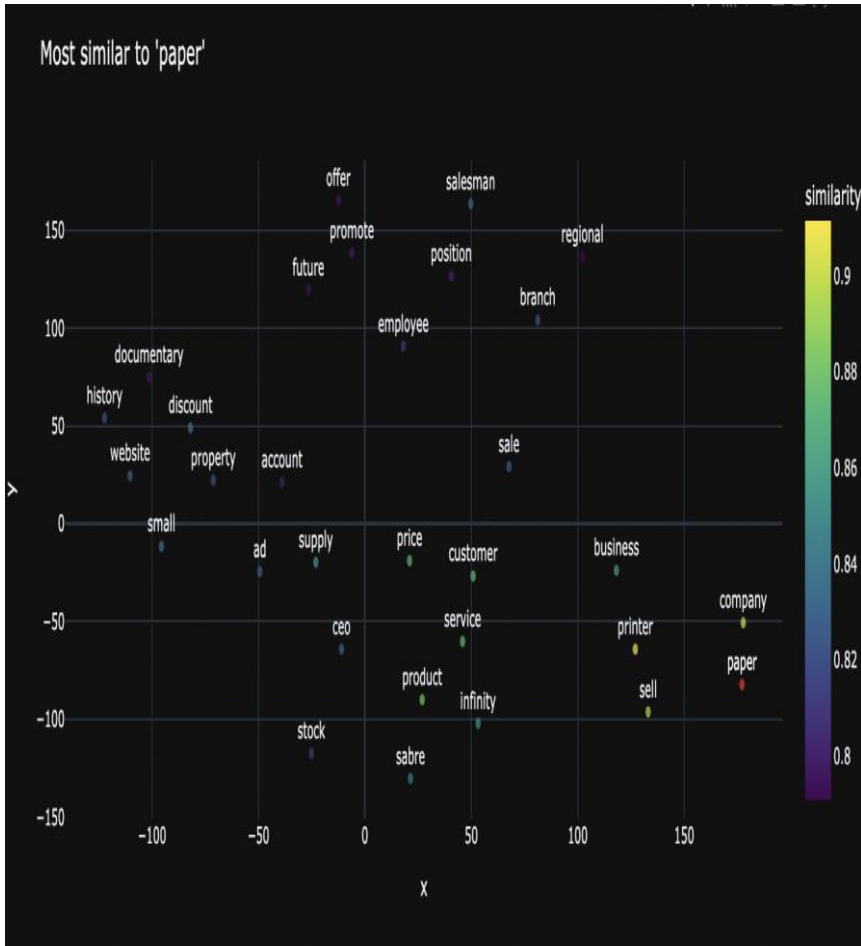


<https://dataforest.ai/blog/large-language-models-advanced-communication>

Text Processing for RNN

Step	Task	Description
1	Preprocessing Text	Clean text, tokenize, remove stopwords, and normalize case using NLTK.
2	Build Vocabulary	Assign a unique index to each word using a frequency-based vocabulary.
3	Convert Text to Sequences	Map tokenized words to their corresponding integer indices.
4	Padding Sequences	Standardize input sequence lengths by adding padding tokens where necessary.
5	Dataloader Preparation	Create PyTorch dataset and dataloader for mini-batch training.
6	Load Pretrained Word Embeddings	Use GloVe embeddings (100D) for better semantic representation.
7	Define RNN Model	Construct an RNN with an embedding layer, hidden layers, and output layer.
8	Loss and Optimization	Use Binary Cross-Entropy Loss ('BCEWithLogitsLoss') and Adam optimizer.
9	Train Model	Train the RNN model using mini-batches from the dataloader.
10	Make Predictions	Preprocess new text, convert it to sequences, and run inference using the trained model.

Word Embeddings



Word embeddings are a fundamental technique in NLP. They convert words into dense, continuous vector representations. Word embeddings place similar words closer in vector space. Unlike traditional one-hot encoding, embeddings preserve:

- ❖ **Semantic relationships** between words.
- ❖ **Contextual meaning** of words in sentences.
- ❖ **Word similarity and analogies.**

One-hot encoding.

For vocabulary size V , represent word w by $x(w) \in \{0, 1\}^V$.

Problem 1: No graded similarity.

Thus “cat” vs “dog” has the same similarity as “cat” vs “microwave”.

$$x(w_1)^T x(w_2) = \begin{cases} 1, & w_1 = w_2, \\ 0, & w_1 \neq w_2. \end{cases}$$

Problem 2: High dimensionality. V can be 10^5 – 10^7 .

Embedding matrix:

$$E \in \mathbb{R}^{V \times d}, \quad d \ll V.$$

For token id $i \in \{0, \dots, V - 1\}$, the embedding is the i -th row:

$$e_i = E[i] \in \mathbb{R}^d.$$

Word Embeddings

We will learn a dense vector for each word such that words appearing in similar contexts have similar vectors, where similarity is measured by the dot (inner) product. These word vectors are also called **word embeddings** or **neural word representations**, and they provide a **distributed representation** of meaning.

$$\cos(e(w_1), e(w_2)) = \frac{e(w_1)^\top e(w_2)}{\|e(w_1)\| \|e(w_2)\|}.$$

$$\text{NN}(w) = \arg \max_{u \in \mathcal{V}} \cos(e(w), e(u)).$$

cos(banking, monetary) ≈ 0.8570.

banking =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

monetary =

$$\begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

Close (high similarity)

Near-synonyms:

doctor–physician; car–automobile

Same category (co-hyponyms):

cat–dog; apple–orange

Strong association:

hospital–nurse; coffee–cup

Distant (low similarity)

Unrelated concepts (low similarity):

- ❖ banana–democracy;
- ❖ microscope–basketball;
- ❖ piano–chlorine;
- ❖ volcano–spreadsheet;
- ❖ neuron–suitcase

Teaching moments (quirks)

Antonyms can be close:

hot–cold; good–bad

Polysemy mixes senses: “bank” may blend finance + river neighbors

Tip: contextual models (BERT/GPT) resolve meaning using sentence context

Types of Word Embeddings

A. Count / Frequency-based (Distributional, “classical”)

- ❖ **TF-IDF** (sparse weighting; mainly for retrieval/features)
 - ❖ **LSA / LSI** (SVD on term–document or term–context matrix → dense vectors)
 - ❖ **PPMI + SVD** (common “explicit-to-dense” pipeline)
- GloVe** (factorizes a global co-occurrence objective → count-based/matrix factorization)

B. Predictive Neural (Static, context-free)

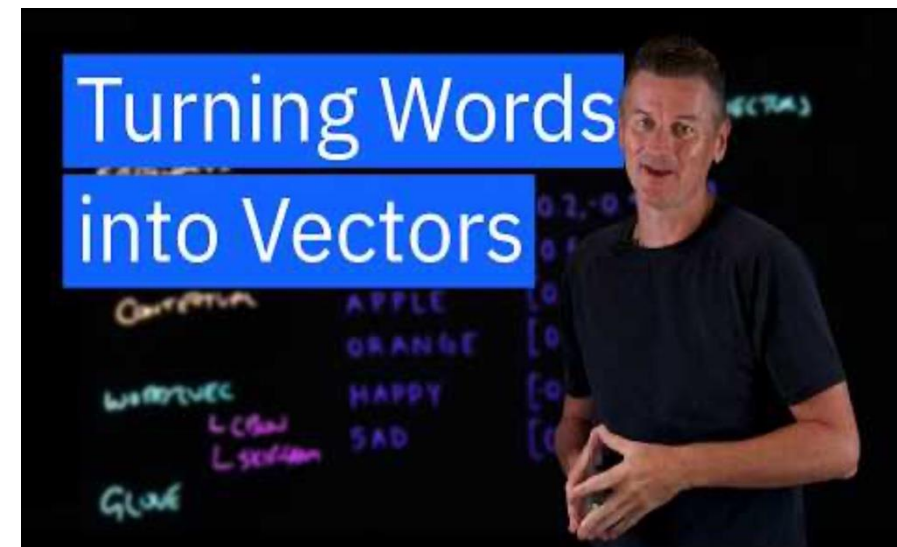
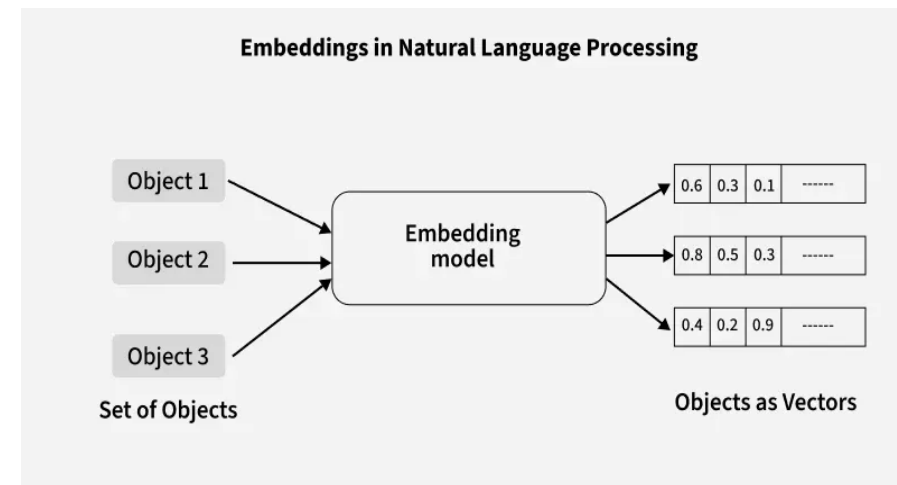
- **Word2Vec**
- **CBOW** (predict center word from context)
- **Skip-gram** (predict context from center word)

C. Subword / Character-aware (Static, better for Out-Of-Vocabulary (OOV))

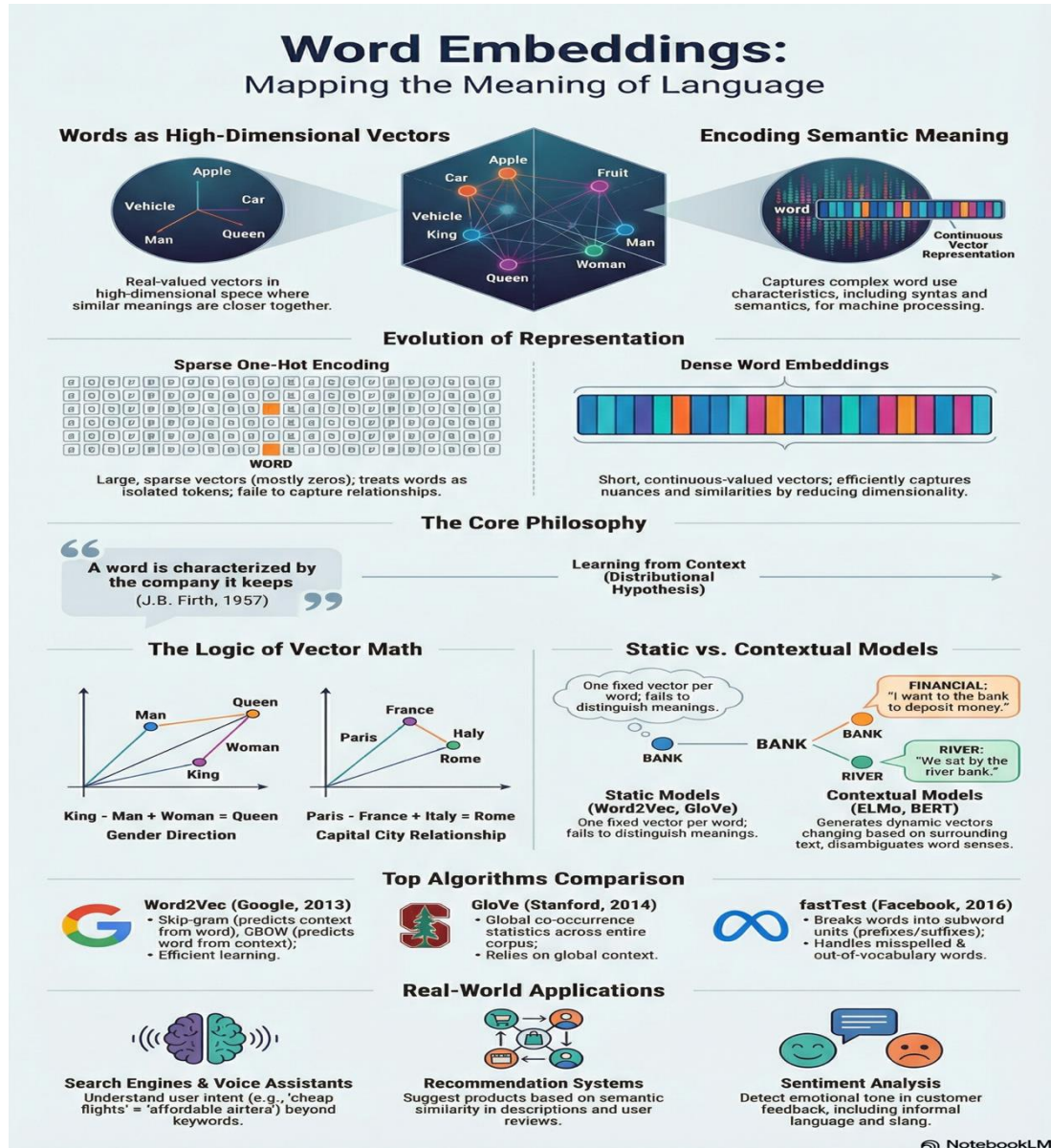
- ❖ **fastText** (word vectors built from character n-grams)

D. Contextual / Transformer-based (Dynamic, context-dependent)

- **BERT / RoBERTa** (masked language modeling; bidirectional context)
- **GPT** (causal language modeling; left-to-right context)



Word Embeddings Dimensions



Word Embeddings	Dimension	Key Features
Word2Vec	50-300	Trained on large corpora like Google News
GloVe	50-300	Uses word co-occurrence statistics
FastText	50-300	Handles subword information
ELMo	1024	Contextual embeddings from bidirectional LSTMs
BERT (base)	768	Transformer-based, context-aware
BERT (large)	1024	More parameters than BERT base
GPT-2 (small)	768	Transformer-based generative model
GPT-2 (medium)	1024	More layers and parameters
GPT-3	12288	High-dimensional transformer model

- ◆ For small models or mobile applications → Use **50-300 dimensions** (Word2Vec, GloVe).
- ◆ For NLP applications with context-awareness → Use **512-1024 dimensions** (BERT, ELMo).
- ◆ For large-scale generative AI → Use **1024+ dimensions** (GPT-3, Transformers).

Train an RNN Language Model

Task 1: *How to import this paragraph for training the language model?*

Obama *“I stand here today humbled by the task before us, grateful for the trust you have bestowed, mindful of the sacrifices borne by our ancestors. I thank President Bush for his service to our nation, as well as the generosity and cooperation he has shown throughout this transition. Forty-four Americans have now taken the presidential oath. The words have been spoken during rising tides of prosperity and the still waters of peace. Yet, every so often the oath is taken amidst gathering clouds and raging storms. At these moments, America has carried on not simply because of the skill or vision of those in high office, but because We the People have remained faithful to the ideals of our forbearers, and true to our founding documents. “*



Word2Vector

Word2Vec is a framework for learning dense word vectors (embeddings) from large text corpora.

Core Idea:

- ▶ Start with a large corpus: a long sequence of tokens $w_1, \dots, w_T$.
- ▶ Assign each vocabulary word a trainable vector representation.
- ▶ For each position t , choose a *center* word $c = w_t$ and its surrounding *context* (*outside*) words

$$o \in \{w_{t-m}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+m}\},$$

where m is the context-window radius.

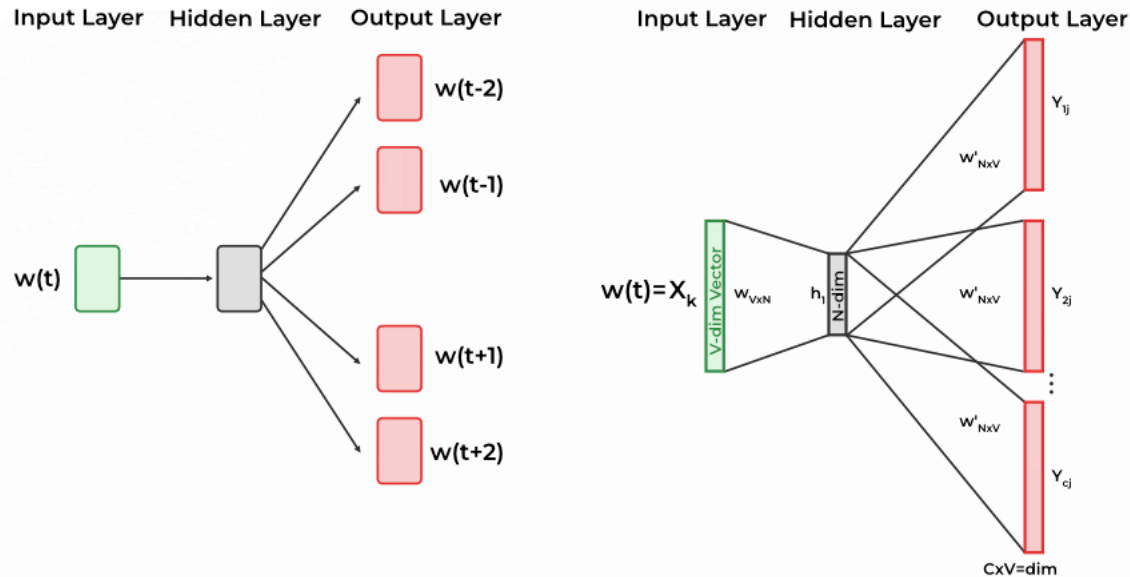
- ▶ Use a similarity score (typically a dot product) between the vectors of c and o to define a conditional probability, e.g., $p(o \mid c)$ (Skip-gram) or $p(c \mid \text{context})$ (CBOW).
- ▶ Update the vectors to *maximize the likelihood* of observed center–context pairs across the corpus (often implemented efficiently via negative sampling or hierarchical softmax).

Goldberg, Y., and Levy, O. (2014). "word2vec Explained: deriving Mikolov et al.'s negative-sampling word-embedding method." *arXiv preprint arXiv:1402.3722*.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013). "Distributed Representations of Words and Phrases and their Compositionality." *Advances in Neural Information Processing Systems (NeurIPS)*, 26.

Word2Vector: Skip-Gram Architecture

SKIP-GRAM ARCHITECTURE



For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t . Data likelihood:

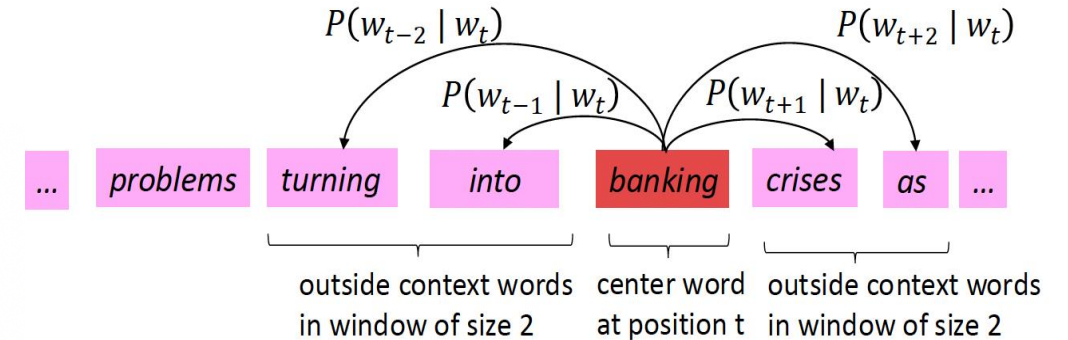
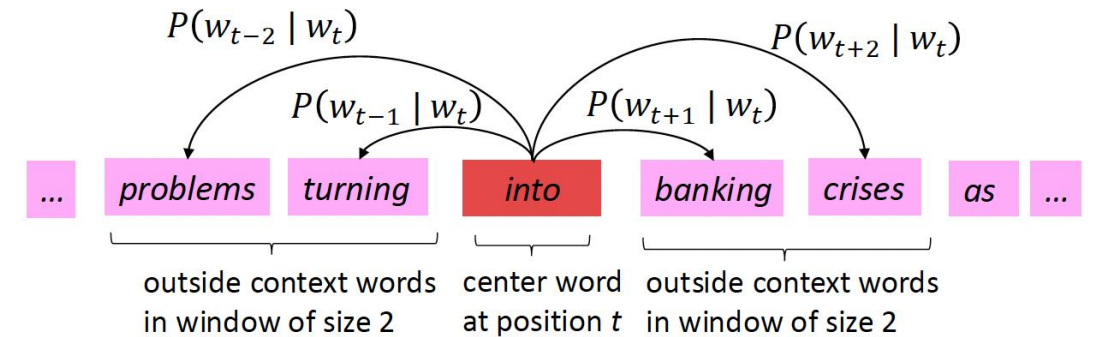
Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$

θ is all variables to be optimized

We want to minimize the objective function:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Example windows and process for computing



Word2Vector:

Question: How to calculate $P(w_{t+j} | w_t; \theta)$?

Answer: We will use two vectors per word w :

- v_w when w is a center word
- u_w when w is a context word

Softmax Function

We typically take v_w as the embedding; u_w is the context embedding; optionally use $(v_w + u_w)/2$.

② Exponentiation makes anything positive

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

① Dot product compares similarity of o and c .
 $u^T v = u \cdot v = \sum_{i=1}^n u_i v_i$
Larger dot product = larger probability

③ Normalize over entire vocabulary to give probability distribution

$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$

Why Two Vectors?

- **Modeling Flexibility:** The distribution of contexts given a word ("word as center") does not necessarily match the distribution of words appearing around it ("word as context").
- **Performance:** Having separate v_w and u_w allows the model to capture more nuance, typically leading to higher-quality embeddings.

StanfordCS224n

Negative Sampling Loss

The normalization term is computationally expensive for large V .

$$P(o|c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

Main idea: train binary logistic regressions to differentiate a true pair (center word and a word in its context window) versus several “noise” pairs (the center word paired with a random word)

- ▶ Treat an observed pair (c, o) as **positive** (label 1).
- ▶ Sample K noise words $n_1, \dots, n_K$ and treat (c, n_k) as **negatives** (label 0).
- ▶ Push $u_o^\top v_c$ up; push $u_{n_k}^\top v_c$ down.

Let $\sigma(z) = \frac{1}{1+e^{-z}}$. The per-pair loss is

$$\ell(c, o) = -\log \sigma(u_o^\top v_c) - \sum_{k=1}^K \log \sigma(-u_{n_k}^\top v_c).$$

Over a corpus $(w_1, \dots, w_T)$ with window radius m :

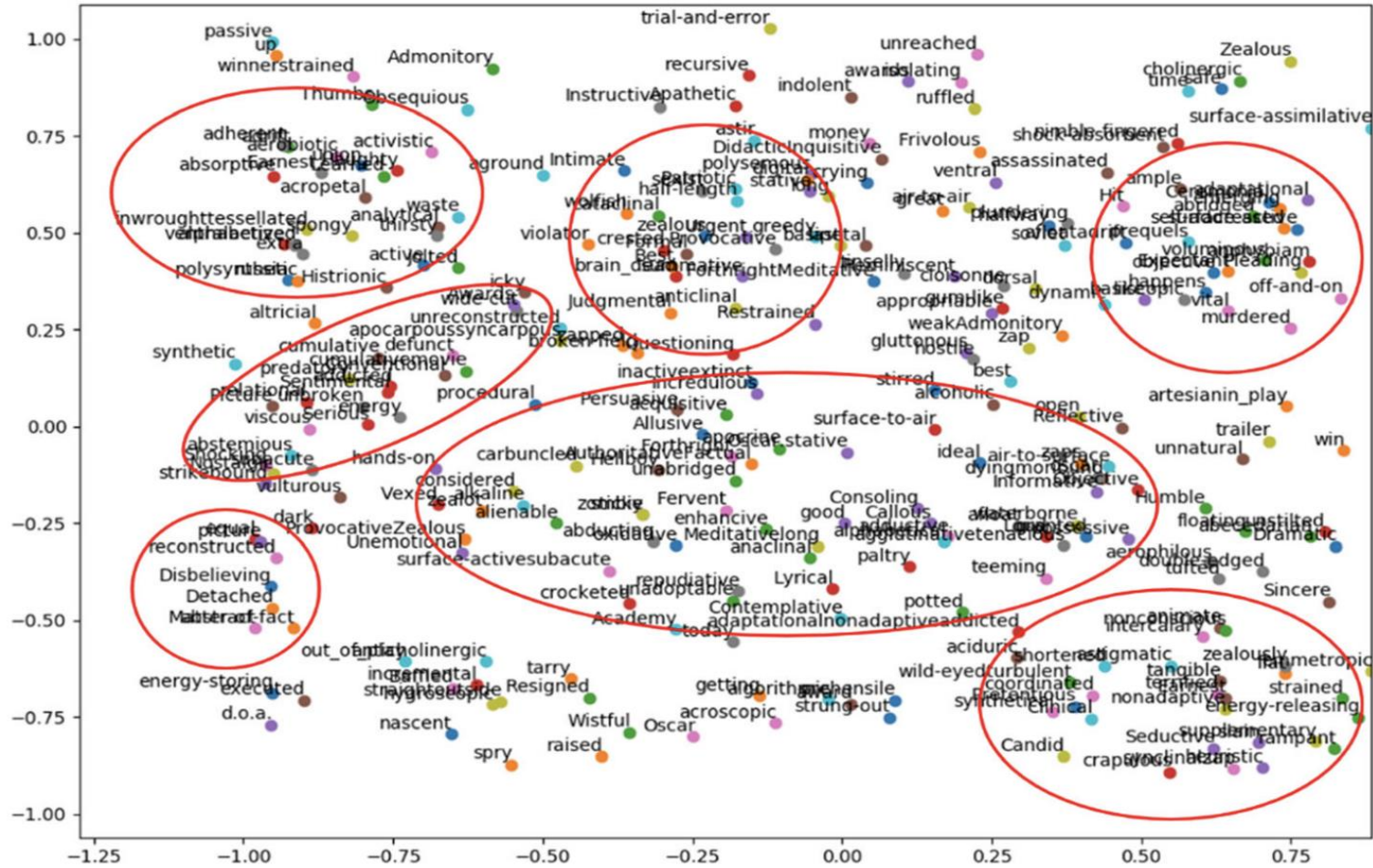
$$\mathcal{L}(\theta) = \sum_{t=1}^T \sum_{\substack{j=-m \\ j \neq 0}}^m \ell(w_t, w_{t+j}).$$

$$n_1, \dots, n_K \stackrel{iid}{\sim} q(\cdot).$$

$$q(w) = \frac{f(w)^\alpha}{\sum_{j=1}^V f(j)^\alpha}, \quad \alpha \in (0, 1) \text{ (often 0.75)}.$$

Let $f(w)$ be the corpus count of word w .

Word2Vector results



Baek, J. W., & Chung, K. Y. (2021). Multimedia recommendation using Word2Vec-based social relationship mining. *Multimedia Tools and Applications*, 80(1), 1-18.

```

import re
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from gensim.models import Word2Vec
import numpy as np
import random

# For reproducibility
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

# =====
# 1. Define and Preprocess the Text
# =====
text = (
    "I stand here today humbled by the task before us, grateful  

    for the trust you have bestowed, "  

    "mindful of the sacrifices borne by our ancestors. I thank  

    President Bush for his service to our nation, "  

    "as well as the generosity and cooperation he has shown  

    throughout this transition.\n\n"  

    "Forty-four Americans have now taken the presidential oath.  

    The words have been spoken during rising tides "  

    "of prosperity and the still waters of peace. Yet, every so  

    often the oath is taken amidst gathering clouds "  

    "and raging storms. At these moments, America has carried on  

    not simply because of the skill or vision of "  

    "those in high office, but because We the People have  

    remained faithful to the ideals of our forbearers, and "  

    "true to our founding documents."

```

```

print("\n--- Original Text ---\n")
print(text)

# Split the text into sentences using a simple regex
sentences = re.split(r'(?<=[!?\])\s+', text.strip())
print("\n--- Split Sentences ---")
for i, s in enumerate(sentences):
    print(f"Sentence {i+1}: {s}")

def tokenize(text):
    # Convert to lowercase and split using regex for word
    # boundaries
    return re.findall(r'\b\w+\b', text.lower())

# Tokenize each sentence
tokenized_sentences = [tokenize(sentence) for sentence in
    sentences]
print("\n--- Tokenized Sentences ---")
for i, tokens in enumerate(tokenized_sentences):
    print(f"Sentence {i+1} Tokens: {tokens}")

# Build vocabulary (set of unique words)
vocab = sorted(set(word for sentence in tokenized_sentences
    for word in sentence))
word_to_idx = {word: idx for idx, word in enumerate(vocab)}
idx_to_word = {idx: word for word, idx in word_to_idx.items()}
vocab_size = len(vocab)

print("\n--- Vocabulary ---")
print(word_to_idx)
print(f"Vocabulary Size: {vocab_size}")

```

```
# =====
# 2. Train Word2Vec and Build Embedding Matrix
# =====
embedding_dim = 100

w2v_model = Word2Vec(sentences=tokenized_sentences,
vector_size=embedding_dim, window=5, min_count=1, workers=4)

# Build the embedding matrix (vocab_size x embedding_dim)
embedding_matrix = torch.zeros(vocab_size, embedding_dim)
for word, idx in word_to_idx.items():
embedding_vector = w2v_model.wv[word]
if embedding_vector is not None:
embedding_matrix[idx] = torch.tensor(embedding_vector)
# =====
# 3. Prepare Input and Target Sequences for Language Modeling
# =====
# For language modeling, we treat the text as one continuous
sequence.
indices = [word_to_idx[word] for sentence in
tokenized_sentences for word in sentence]

# Create input sequence and target sequence.
# The target is the next word (shifted by one position). The
last target is omitted.
input_indices = indices[:-1] # all except last
target_indices = indices[1:] # all except first

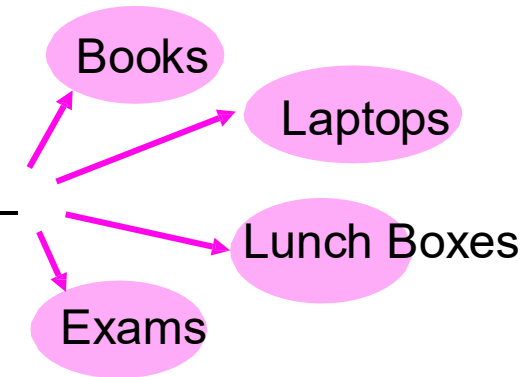
# Convert to PyTorch tensors. We use batch_size = 1 for
simplicity.
input_seq = torch.tensor([input_indices], dtype=torch.long) #
Shape: (1, seq_len)
target_seq = torch.tensor([target_indices], dtype=torch.long)
# Shape: (1, seq_len)
```

```
Sentence 1: I stand here today humbled by the task before us,
grateful for the trust you have bestowed, mindful of the
sacrifices borne by our ancestors.
--- Sentence 1 Tokens: ['i', 'stand', 'here', 'today',
'humbled', 'by', 'the', 'task', 'before', 'us', 'grateful',
'for', 'the', 'trust', 'you', 'have', 'bestowed', 'mindful',
'of', 'the', 'sacrifices', 'borne', 'by', 'our', 'ancestors']
--- Vocabulary --- {'america': 0, 'americans': 1, 'amidst': 2,
'ancestors': 3, 'and': 4, 'as': 5, 'at': 6, 'because': 7,
'been': 8, 'before': 9, 'bestowed': 10, 'borne': 11, 'bush':
12, 'but': 13, . . . 'we': 88, 'well': 89, 'words': 90,
'yet': 91, 'you': 92}
Vocabulary Size: 93
--- Training Word2Vec Model --- Word2Vec training completed. -
-- Embedding Matrix ---
Embedding Matrix Shape: torch.Size([93, 100]) Sample
Embeddings: america: tensor([-0.0081, -0.0009, 0.0064, 0.0087,
-0.0050])... americans: tensor([ 0.0010, 0.0086, -0.0040,
0.0030, 0.0032])... amidst: tensor([-0.0065, 0.0073, 0.0061, -
0.0049, -0.0017
--- Token Indices --- [37, 68, 33, 81, 36, 14, 74, 72, 9, 85,
29, 22, 74, 84, 92, 31, 10, 41, 47, 74, 61, 11, 14, 52, 3, 37,
73, 55, 12, 22, 35, 62, 80, 52, 43, 5, 89, 5, 74, 28, 4, 17,
32, 30, 63, 78, 76, 82, 24, 26, 1, 31, 45, 71, 74, 56, 46, 74,
90, 31, 8, 67, 19, 60, 79, 47, 57, 4, 74, 69, 87, 47, 53, 91,
20, 66, 49, 74, 46, 40, 71, 2, 27, 16, 4, 58, 70, 6, 75, 42,
0, 30, 15, 50, 44, 64, 7, 47, 74, 65, 51, 86, 47, 77, 39, 34,
48, 13, 7, 88, 74, 54, 31, 59, 21, 80, 74, 38, 47, 52, 23, 4,
83, 80, 52, 25, 18] --- Input and Target Indices --- Input
Indices (first 20): [37, 68, 33, 81, 36, 14, 74, 72, 9, 85,
29, 22, 74, 84, 92, 31, 10, 41, 47, 74] Target Indices (first
20): [68, 33, 81, 36, 14, 74, 72, 9, 85, 29, 22, 74, 84, 92,
31, 10, 41, 47, 74, 61] Input Sequence Shape: torch.Size([1,
126]) Target Sequence Shape: torch.Size([1, 126])
```

Formulation: Language Modeling (LM)

A **language model (LM)** is a statistical or machine learning model that **predicts the next word in a sequence** or assigns **probabilities** to sequences of words.

the students opened their



- ✓ Predicts the likelihood of a sequence of words
- ✓ Generates human-like text (e.g., GPT models)
- ✓ Understands context and meaning
- ✓ Enables AI systems to process and generate natural language

More formally: given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$: $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$ where $x^{(t+1)}$ can be any word in the vocabulary $V = \{w_1, \dots, w_{|V|}\}$

Example:

- **Input:** "I am going to the"
- **Model prediction:** "store" (80%), "beach" (15%), "moon" (5%)
- The model assigns probabilities and selects the most likely next word.

Language Modeling

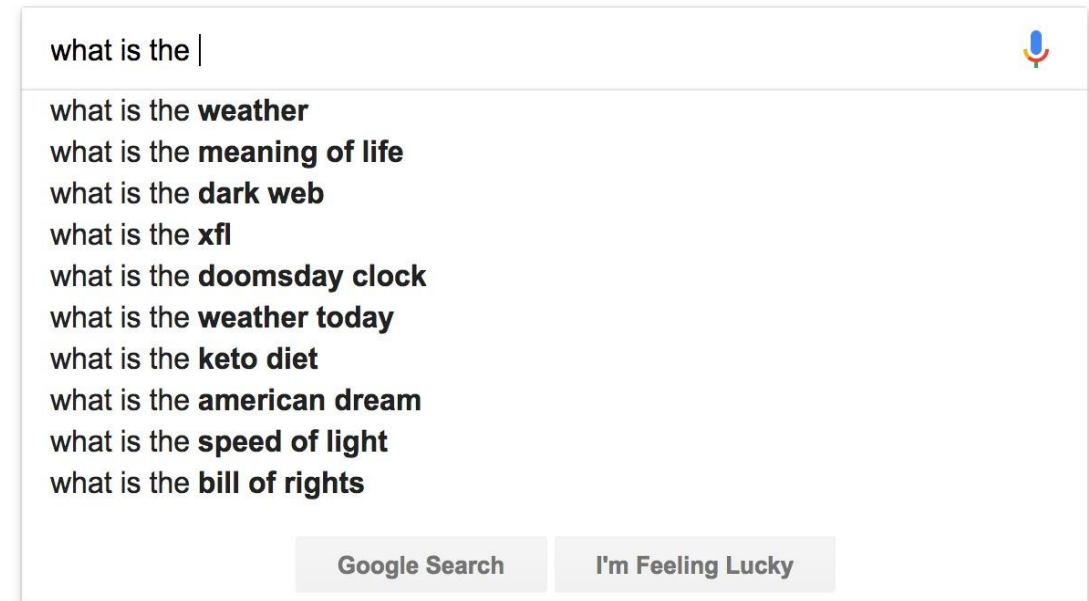
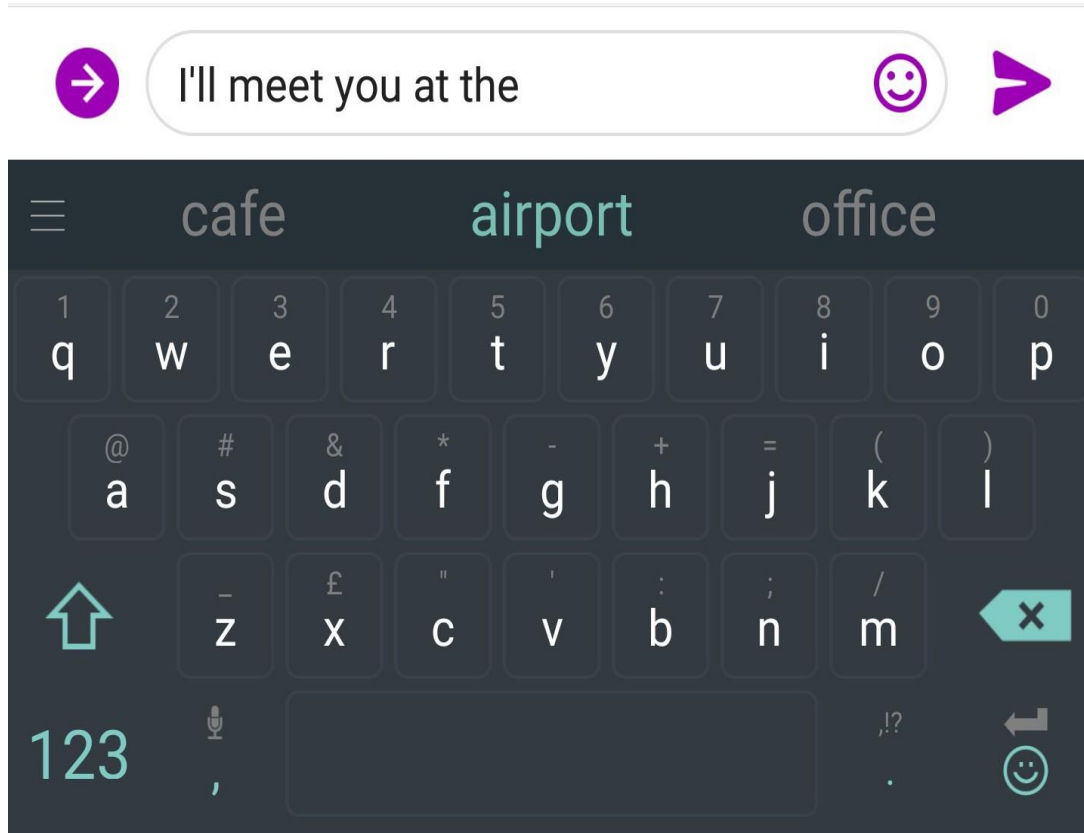
- You can also think of a Language Model as a system that assigns a probability to a piece of text
- For example, if we have some text $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$ then the probability of this text (according to the Language Model) is:

$$P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) = P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)})$$

$$= \prod_{t=1}^T P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)})$$

This is what our LM provides

You use Language Models every day!



n-gram Language Models

the students opened their _____

- ❑ **Question:** How to learn an n-gram Language Model during the pre-DL period?
- ❑ **Answer:** An **n-gram** is a sequence of **n consecutive words** from a text. The larger the **n**, the more context the model considers when making predictions.
- ❑ **Definition:** An *n*-gram is a chunk of *n* consecutive words.
 - **uni**grams: “the”, “students”, “opened”, “their”
 - **bi**grams: “the students”, “students opened”, “opened their”
 - **tri**grams: “the students opened”, “students opened their”
 - **four**-grams: “the students opened their”
- ❖ **Idea:**
 - **Collect statistics** on how frequently different **n-grams appear in a corpus**.
 - The **probability of the next word** is estimated using the **previous (n-1) words**.

n-gram Language Models

- Under the **Markov assumption**: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \overbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

prob of a n-gram $\rightarrow$

prob of a (n-1)-gram $\rightarrow$

$$= \frac{P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})} \quad (\text{definition of conditional prob})$$

- Question:** How do we get these n -gram and $(n-1)$ -gram probabilities?
- Answer:** By **counting** them in some large corpus of text!

$$\approx \frac{\text{count}(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{\text{count}(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})} \quad (\text{statistical approximation})$$

StanfordCS224n

n-gram Language Models: Example

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ students opened their _____

discard

condition on this

$$P(\mathbf{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- “students opened their” occurred 1000 times
- “students opened their books” occurred 400 times
 - $\Rightarrow P(\text{books} \mid \text{students opened their}) = 0.4$
- “students opened their exams” occurred 100 times
 - $\Rightarrow P(\text{exams} \mid \text{students opened their}) = 0.1$

Should we have discarded the “proctor” context?

Major Challenges

-  **Data Sparsity** – Rare n-grams may not appear frequently in training data.
-  **Fixed Context Window** – Cannot capture long-range dependencies beyond n words.
-  **Poor Generalization** – Cannot understand unseen word sequences.

MC1

Problem: What if “*students opened their w*” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

MC2

Problem: What if “*students opened their*” never occurred in data? Then we can’t calculate probability for *any* w !

(Partial) Solution: Just condition on “*opened their*” instead. This is called *backoff*.

MC3

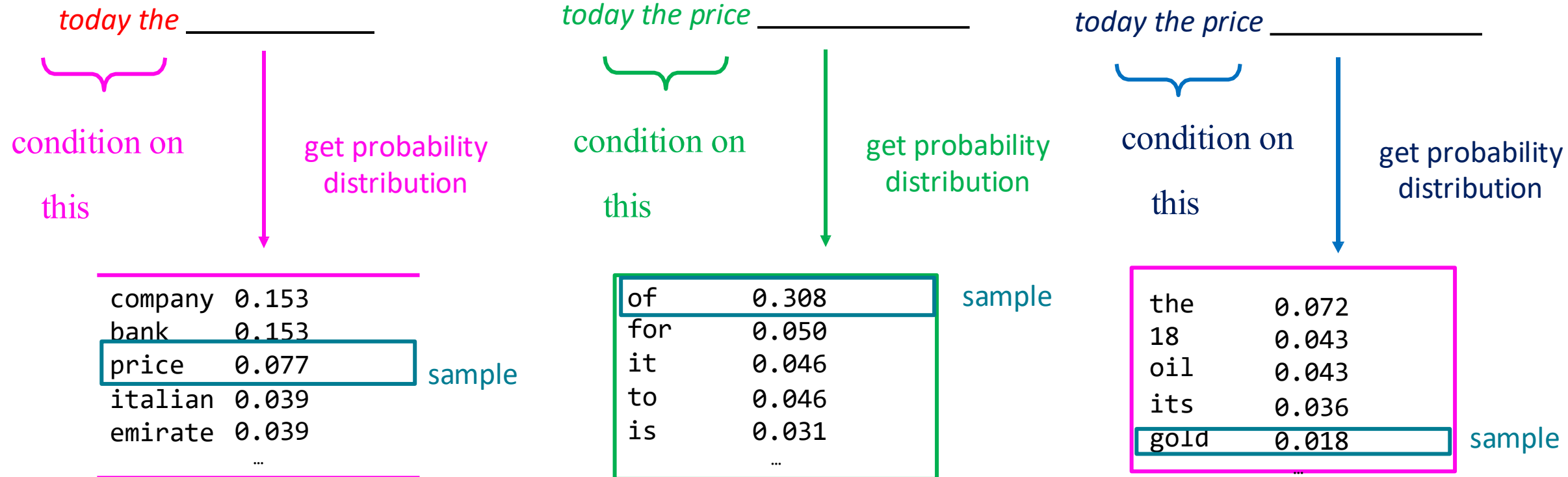
Storage: Need to store count for all n -grams you saw in the corpus. Increasing n or increasing corpus increases model size!

MC4

Increasing n makes sparsity problems *worse*. Typically, we can’t have n bigger than 5.

Generating text with a n-gram Language Model

You can also use a Language Model to **generate** text



StanfordCS224n

Generating text with a n-gram Language Model

You can also use a Language Model to **generate text**

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Surprisingly grammatical!

...but **incoherent**. We need to consider more than three words at a time if we want to model language well.

But increasing n worsens sparsity problem,
and increases model size...

StanfordCS224n

How to build a *neural* language model?

- Recall the Language Modeling task:

- Input: sequence of words
- Output: prob. dist. of the next word

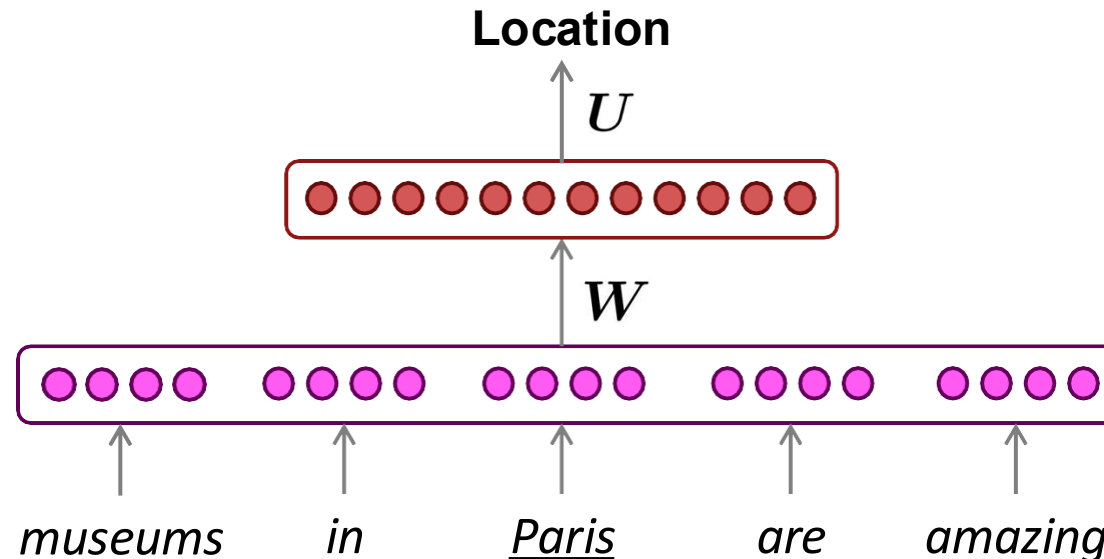
$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(t)}$$

$$P(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})$$

- How about a **window-based neural model**?

- We saw this applied to Named Entity Recognition (NER):

NER is a fundamental **NLP** task that involves identifying and classifying **specific entities** in a given text into predefined categories such as **names of people, organizations, locations, dates, monetary values, and more**.



StanfordCS224n

A Fixed-window Neural Language Model

Output Distribution

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|}$$

Hidden Layer

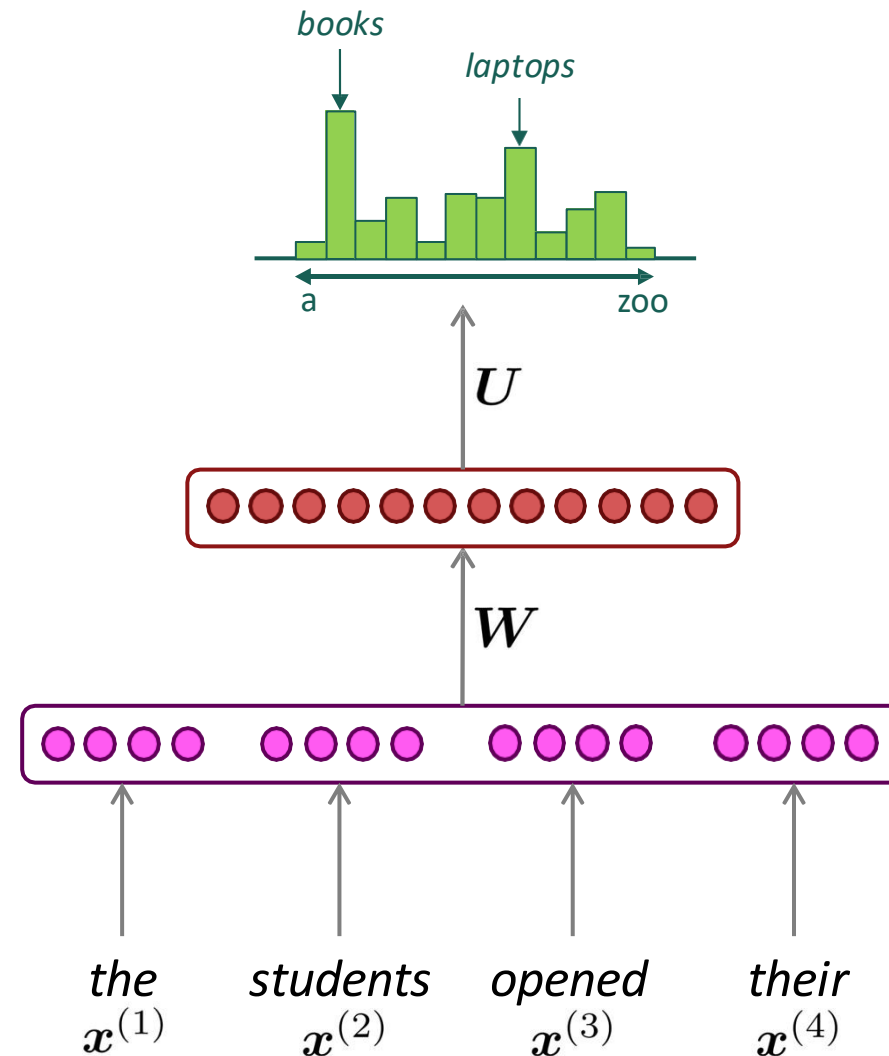
$$h = f(We + b_1)$$

Concatenated Word Embeddings

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

Words /One-hot vectors

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



A fixed-window neural Language Model

Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

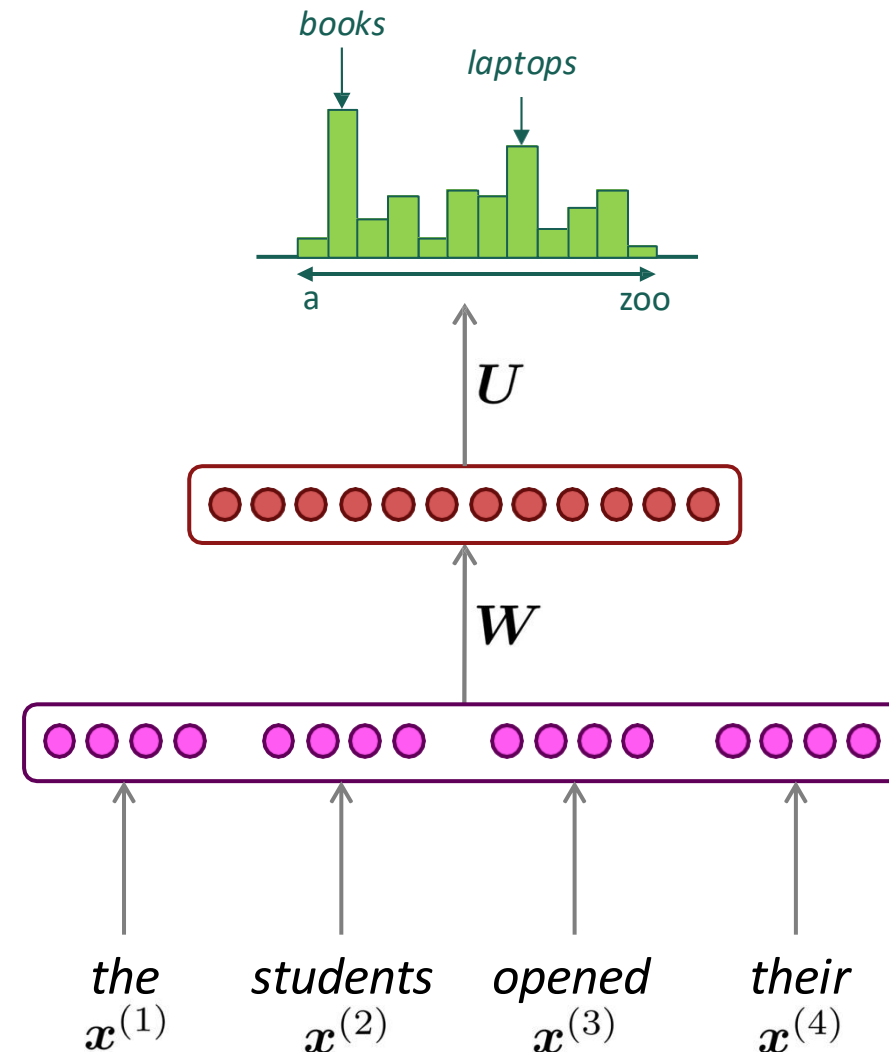
Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams

Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W . **No symmetry** in how the inputs are processed.

We need a neural architecture
that can process *any length input*



Content

0 Motivation to Sequence Modeling

1 Introduction to Language Modeling

2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 Extensions of RNNs

History of Deep RNNs

The Rise of Deep RNNs (2010s - Present)

◆ RNNs in NLP and AI

- **2013** – Google used LSTM for **speech recognition**.
- **2014** – **Seq2Seq Models** (Sutskever et al.) used LSTMs for **machine translation**.
- **2015** – Google Translate adopted LSTMs for **neural machine translation (NMT)**.

◆ Attention and Transformers Change the Game

- **2015** – **Bahdanau et al.** introduced **Attention Mechanisms**, improving Seq2Seq models.
- **2017** – **Vaswani et al.** introduced **Transformers**, replacing RNNs with a more parallelizable model.

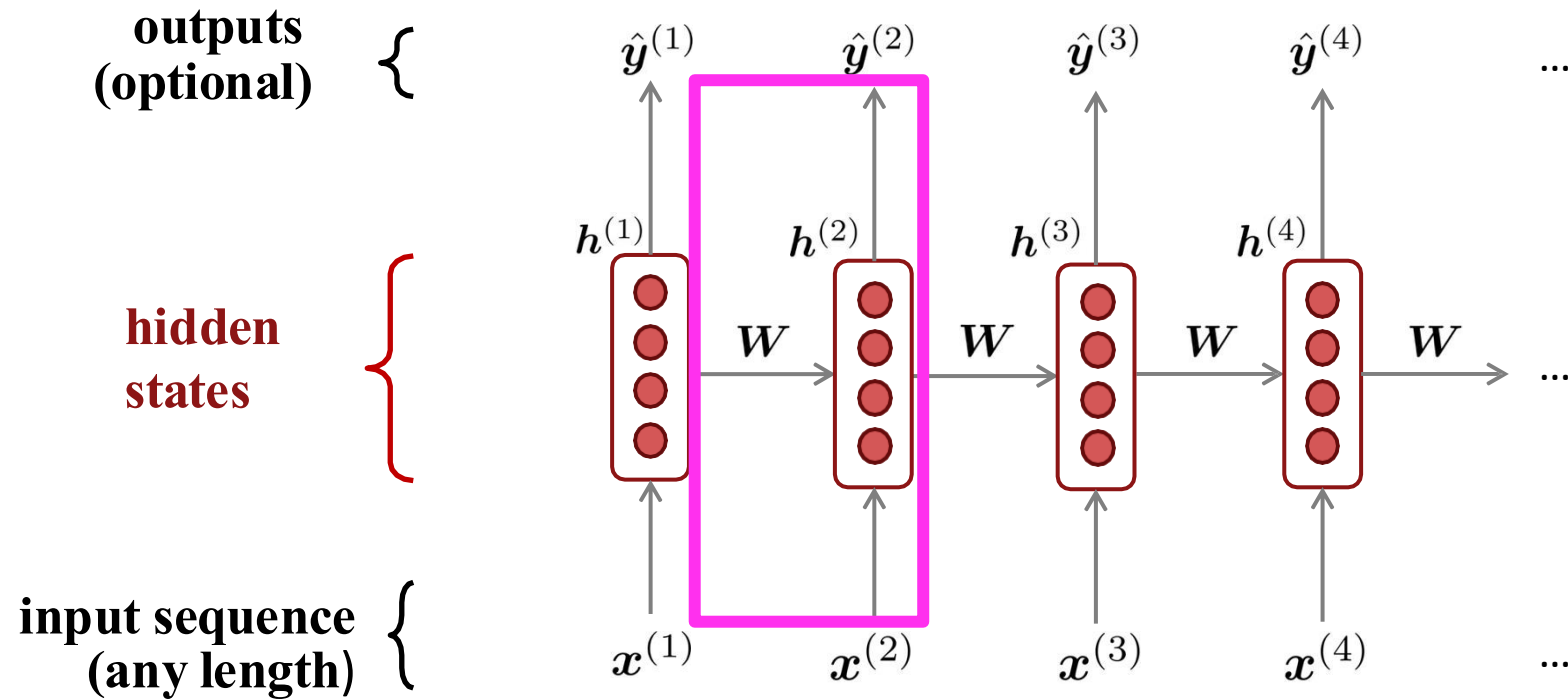
🔍 Key Concept:

Transformers like **BERT (2018)**, **GPT-3 (2020)**, and **ChatGPT (2022)** outperformed RNNs, leading to their decline in NLP.

Recurrent Neural Networks (RNN)

A family of neural architectures

Core idea: Apply the same weights W repeatedly



```
nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
```

StanfordCS224n

A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

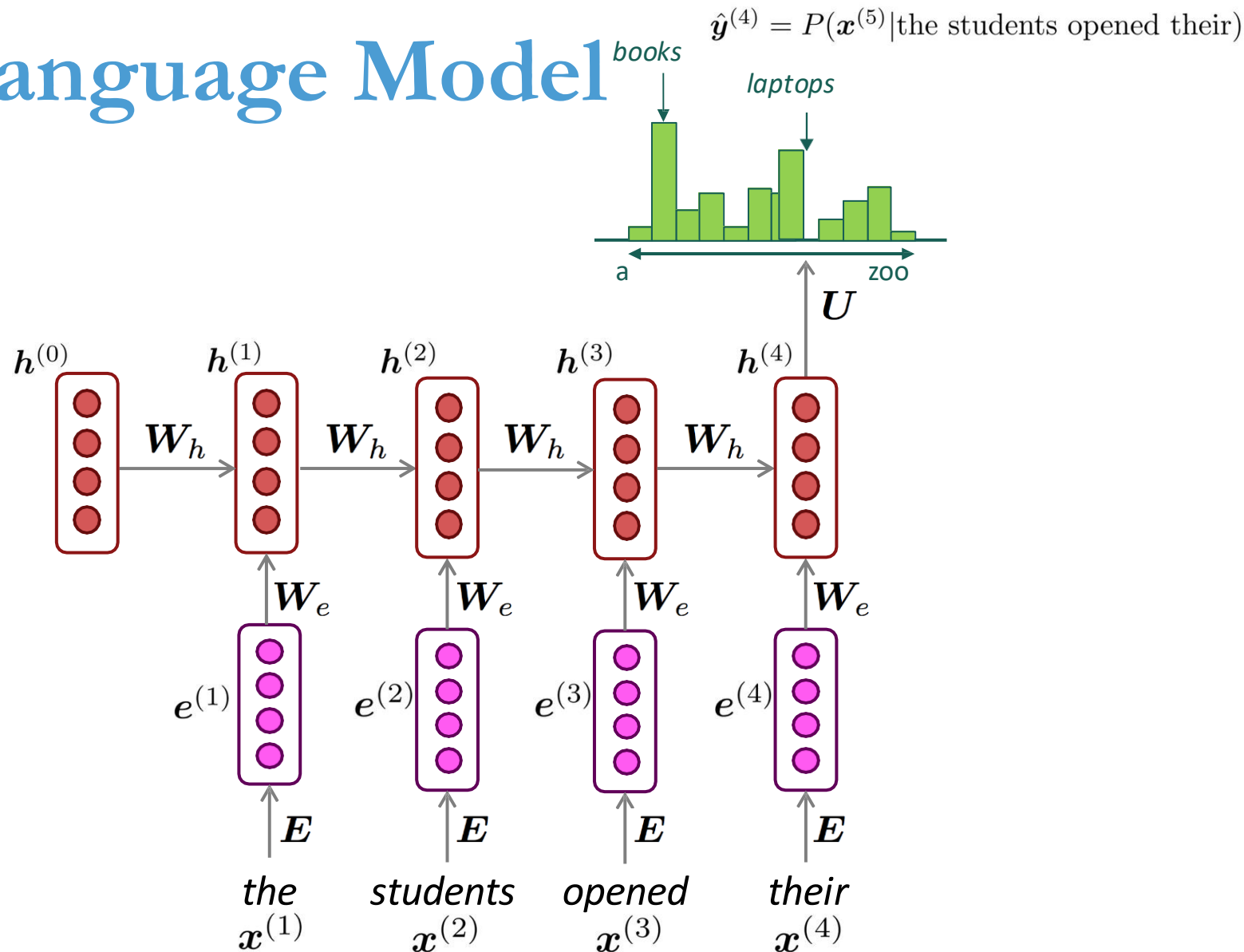
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Note: this input sequence could be much longer now!

RNN Language Models

RNN **Advantages**:

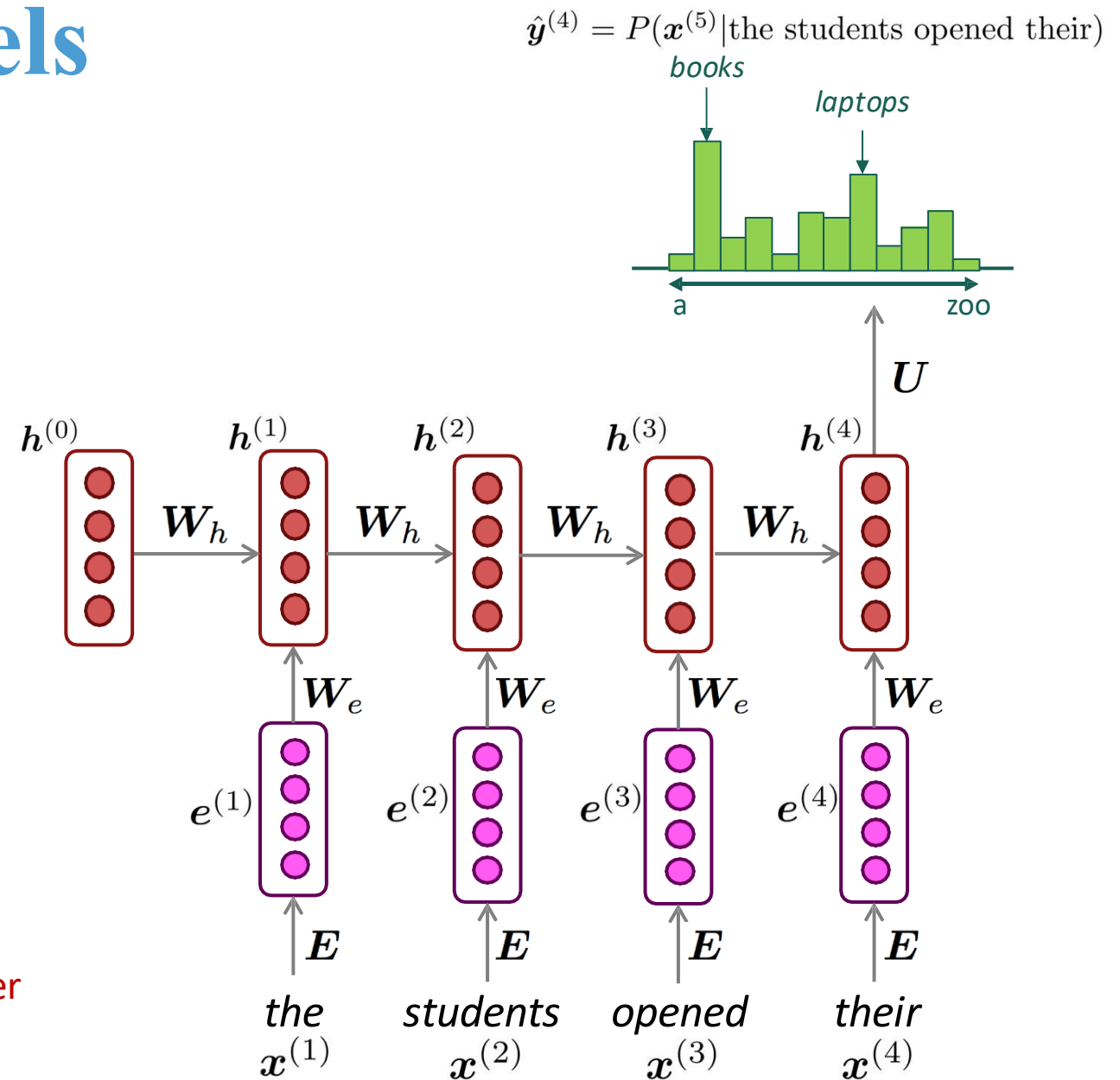
- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN **Disadvantages**:

- Recurrent computation is **slow**
- In practice, difficult to access information from **many steps back**

} More on these later

StanfordCS224n



Training an RNN Language Model

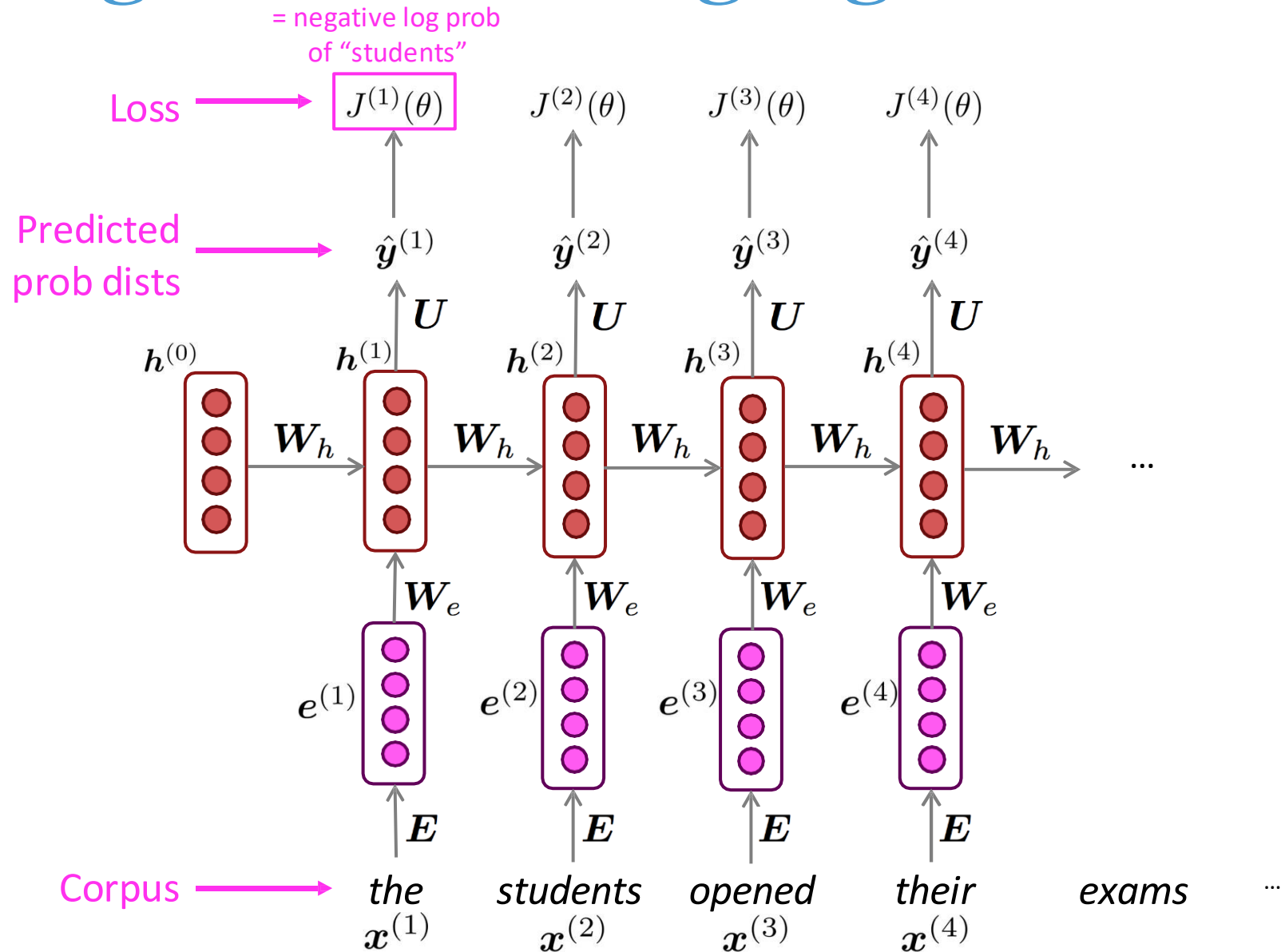
- ❖ Prepare the dataset (tokenize text into words) and convert words to numerical tensors (word embeddings).
- ❖ Build an RNN-LM and compute output distribution $\hat{\mathbf{y}}^{(t)}$ *for every step t* .
- ❖ *Loss function* on step t is *cross-entropy loss* between predicted probability distribution $\hat{\mathbf{y}}^{(t)}$ and the true next word $\mathbf{y}^{(t)}$ (one-hot for $\mathbf{x}^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

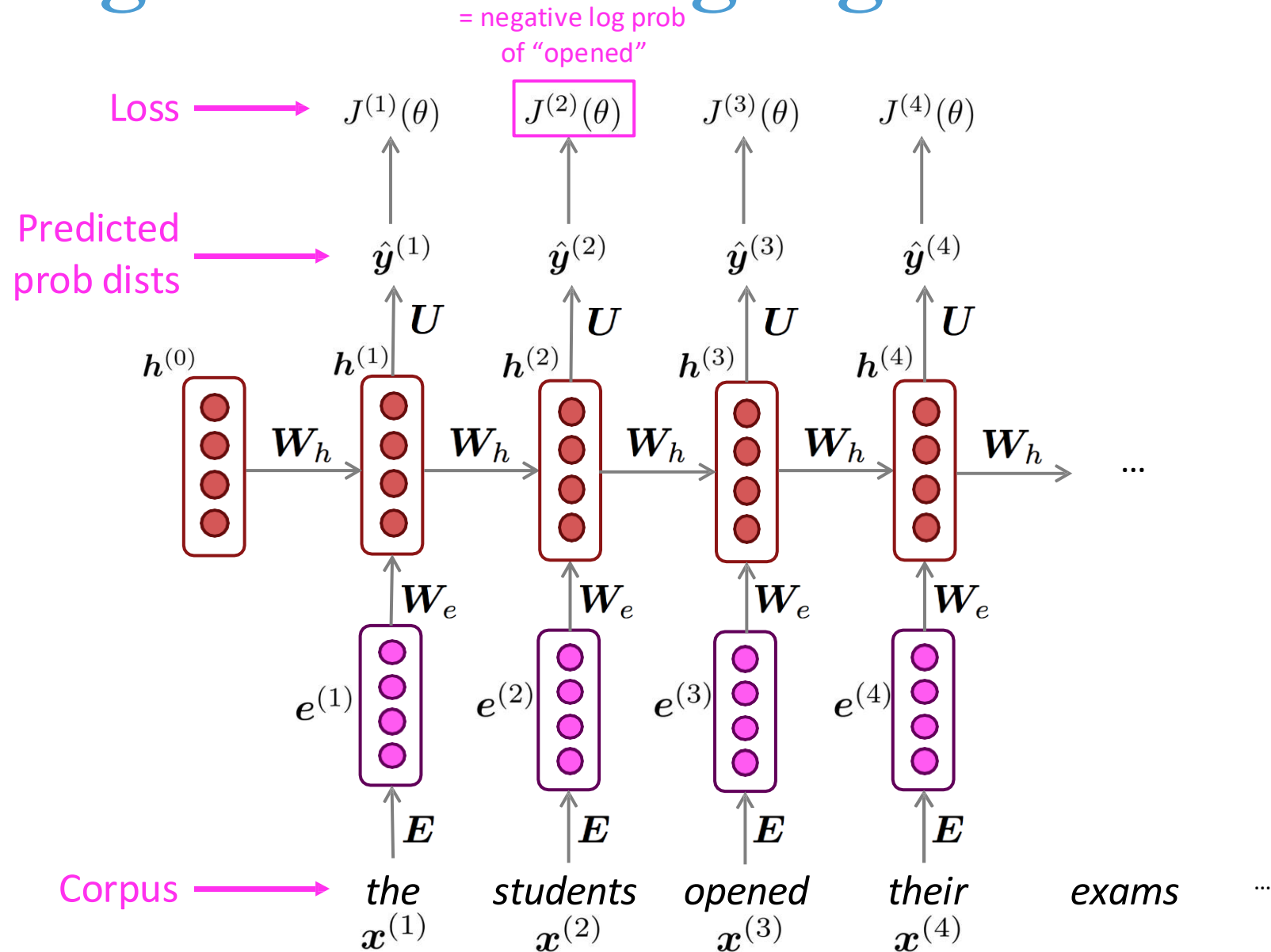
- ❖ Average this to get the *overall loss* for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

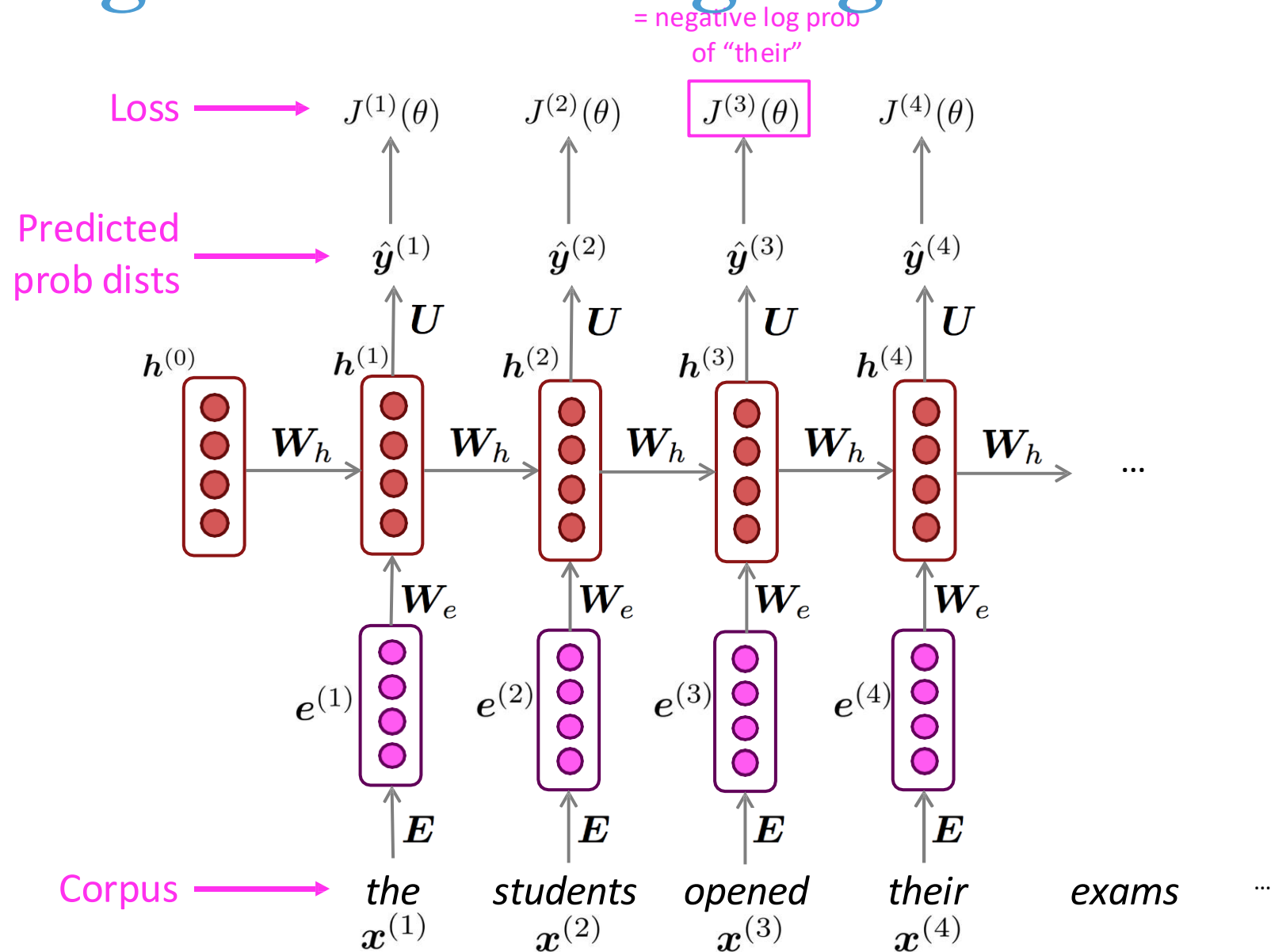
Training an RNN Language Model



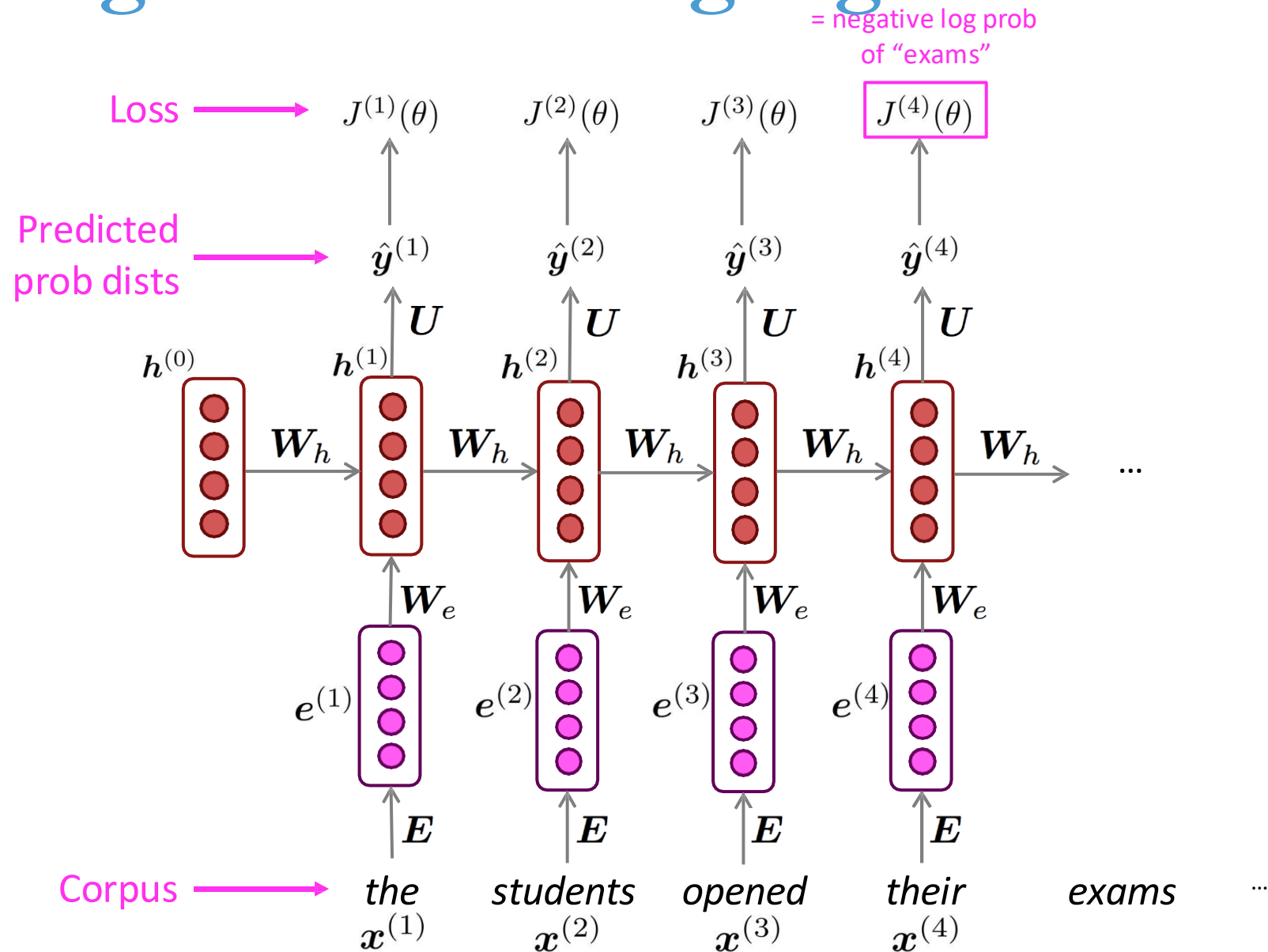
Training an RNN Language Model



Training an RNN Language Model

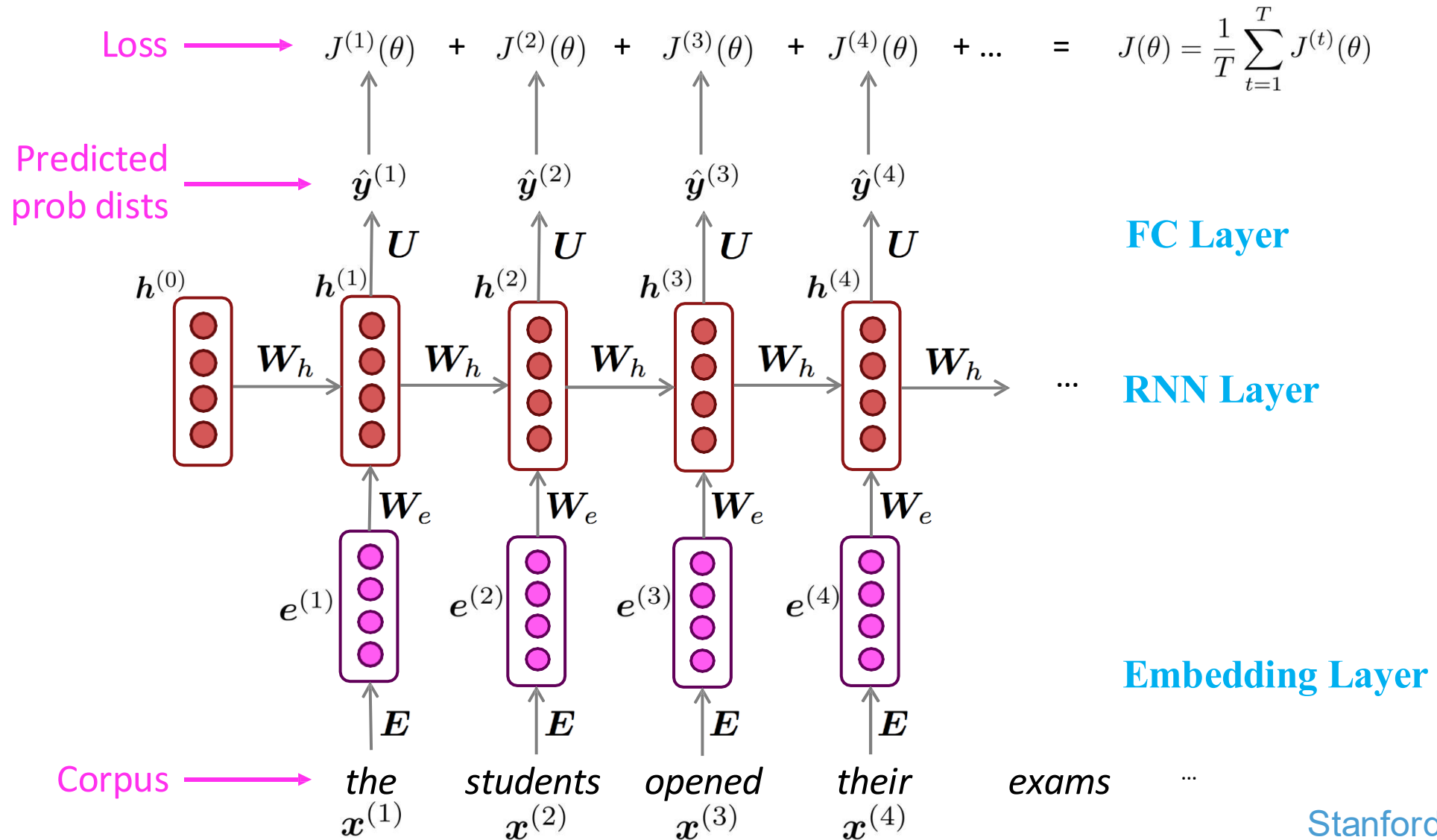


Training an RNN Language Model



Training an RNN Language Model

“Teacher forcing”



StanfordCS224n

Dimensions and Parameters

Input dimension (D): Size of each input vector (e.g., 100 for Word2Vec embeddings).

Hidden dimension (H): Size of the hidden state vector (e.g., 64).

Batch size (B): Number of sequences processed in parallel.

Sequence length (S): Number of time steps (tokens) per sequence.

Output dimension (V): For example, vocabulary size in language modeling or the number of classes in classification.

1. Embedding Layer

Maps input tokens to dense vectors (dimension: D).

2. RNN Layer

Processes the sequence of embeddings and produces hidden states $h^{(t)}$ at each time step.

Equation:

$$a_t = W_e \mathbf{e}_t + b_e + W_h h^{(t-1)} + b_h$$

$$h^{(t)} = \tanh(a_t)$$

3. Fully Connected (FC) Layer

Maps hidden state $h^{(t)}$ to output $y^{(t)}$ (dimension: V).

Equation:

$$y^{(t)} = W_{hy} h^{(t)} + b_y$$

Components:

- ▶ **Input-to-Hidden:** $W_e \in \mathbb{R}^{H \times D} \rightarrow \# \text{ Parameters: } H \times D$
- ▶ **Hidden-to-Hidden:** $W_h \in \mathbb{R}^{H \times H} \rightarrow \# \text{ Parameters: } H \times H$
- ▶ **Biases:** $b_e \in \mathbb{R}^H$ and $b_h \in \mathbb{R}^H \rightarrow \text{Total: } 2H$

Total RNN Cell Parameters:

$$(H \times D) + (H \times H) + 2H$$

Dimensions:

- ▶ $h^{(t)} \in \mathbb{R}^H$; $W_{hy} \in \mathbb{R}^{V \times H}$
- ▶ $b_y \in \mathbb{R}^V$; $y^{(t)} \in \mathbb{R}^V$ (e.g., logits over vocabulary)

Parameter Count:

$$\text{FC Parameters} = (V \times H) + V = V \times (H + 1)$$

Training Challenges

- **Memory Constraints:** Storing all word embeddings, gradients, and activations requires enormous memory.
- **Computational Cost:** Performing **backpropagation over the entire dataset** in a single step is impractical.
- **Batching is Required:** Instead of processing all data at once, models use **mini-batches** to update weights efficiently.

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

Challenges

Too much memory usage

Long sequences overflow memory

Exploding gradients

Slow training

Solution

✓ Mini-batch training

✓ Truncated BPTT (TBPTT)

✓ Gradient clipping

✓ Efficient batching and parallelization

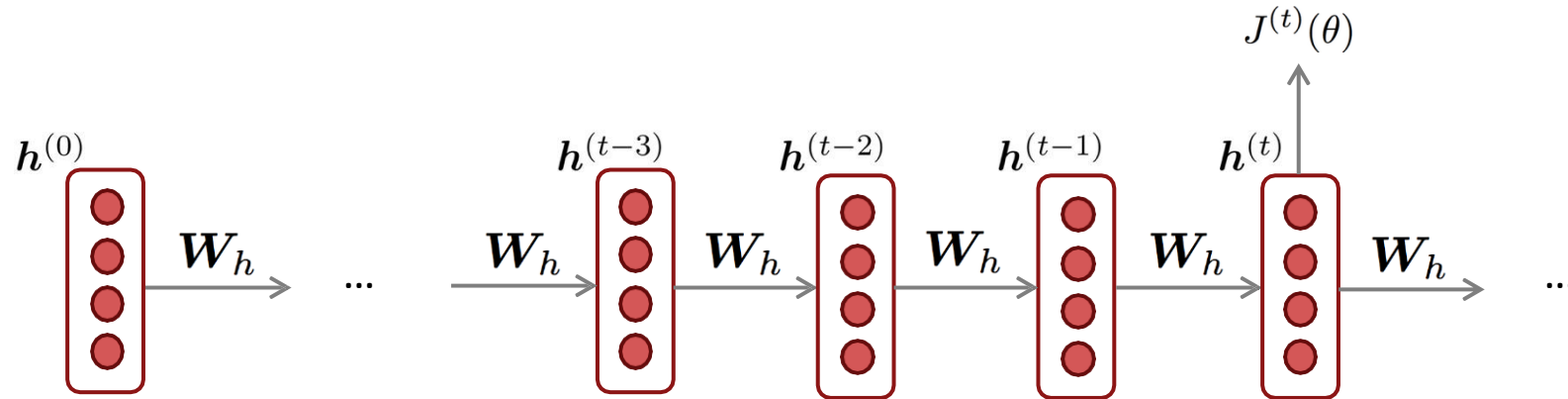
Divide the **entire dataset** into **mini-batches** (e.g., **batch size = 32**).

Truncate the sequence into smaller **sub-sequences** (e.g., **20 time steps** at a time).

Clip gradients to a maximum norm (e.g., **5**).

$$g = g \times \frac{C}{\max(\|g\|, C)}$$

Backpropagation for RNNs



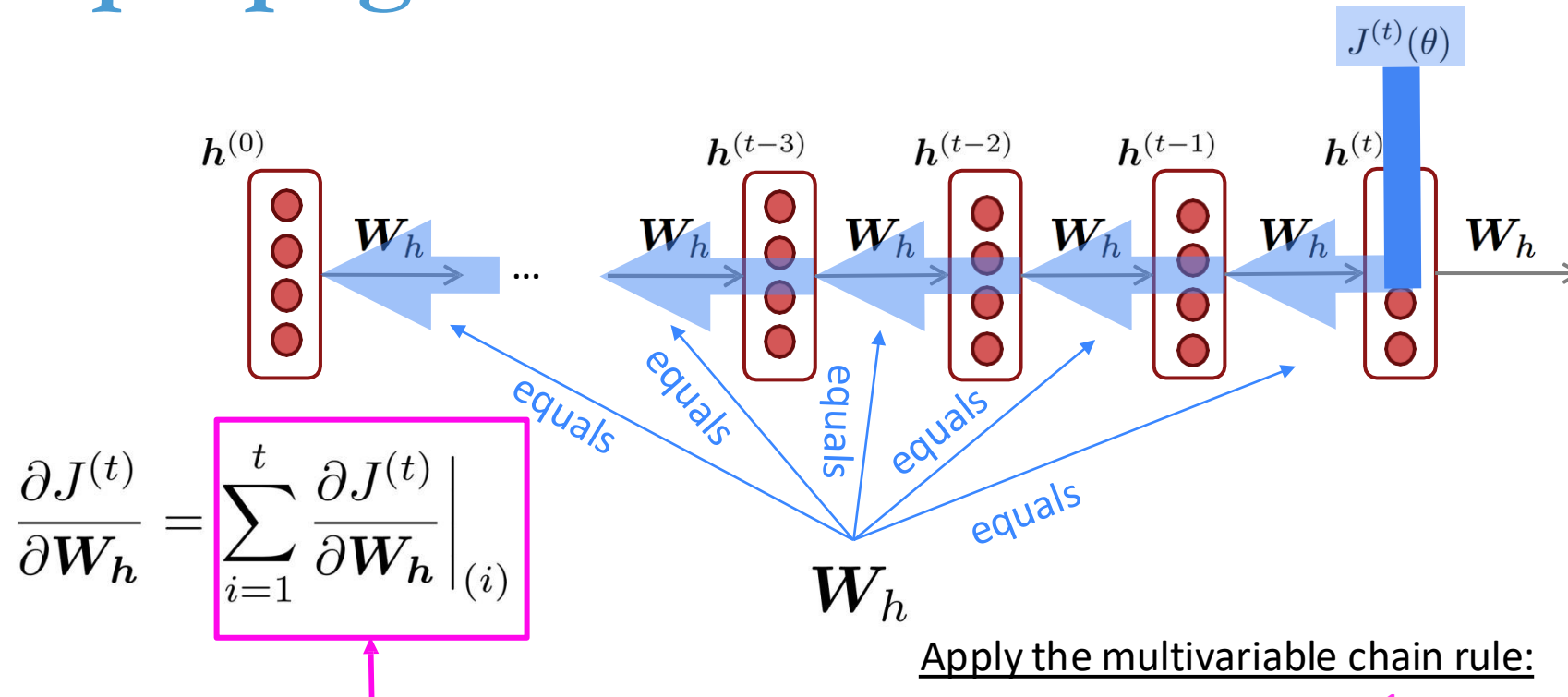
Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t. the **repeated** weight matrix W_h ?

Answer:
$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$$

“The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears”

Why?

Backpropagation for RNNs



In practice, often “truncated” after ~20 timesteps for training efficiency reasons

Question: How do we calculate this?

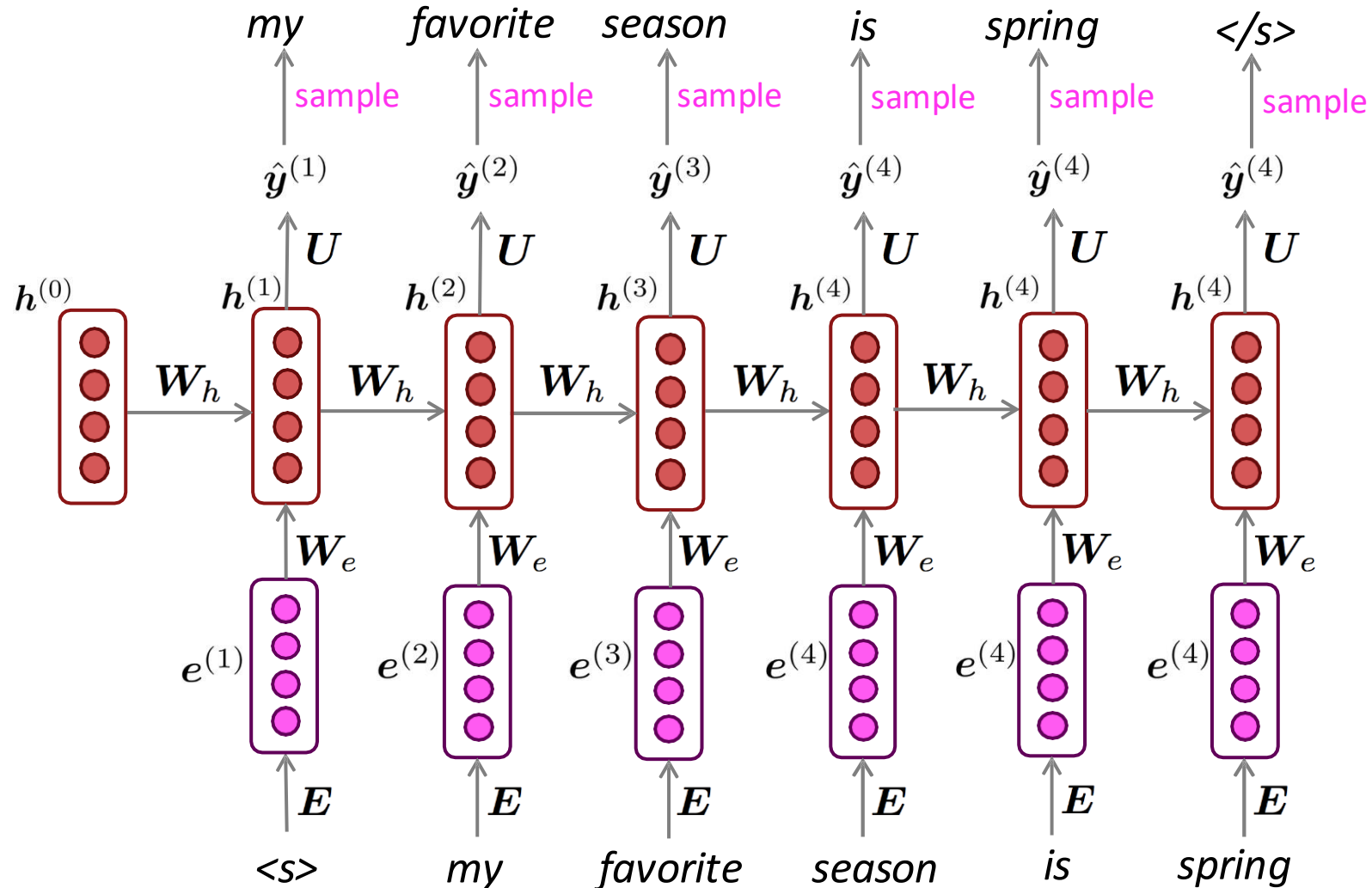
Answer: Backpropagate over timesteps $i = t, \dots, 0$, summing gradients as you go. This algorithm is called “**backpropagation through time**” [Werbos, P.G., 1988, *Neural Networks 1*, and others]

Apply the multivariable chain rule:

$$\begin{aligned} &= 1 \\ \frac{\partial J^{(t)}}{\partial W_h} &= \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)} \boxed{\frac{\partial W_h |_{(i)}}{\partial W_h}} \\ &= \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)} \end{aligned}$$

Generating roll outs

Just like an n-gram Language Model, you can use a RNN Language Model to **generate text** by **repeated sampling**. Sampled output becomes next step's input.



Train an RNN Language Model

Tasks 2 & 3: *How to train an RNN-LM on this paragraph and then generate text in that style?*

Obama *“I stand here today humbled by the task before us, grateful for the trust you have bestowed, mindful of the sacrifices borne by our ancestors. I thank President Bush for his service to our nation, as well as the generosity and cooperation he has shown throughout this transition. Forty-four Americans have now taken the presidential oath. The words have been spoken during rising tides of prosperity and the still waters of peace. Yet, every so often the oath is taken amidst gathering clouds and raging storms. At these moments, America has carried on not simply because of the skill or vision of those in high office, but because We the People have remained faithful to the ideals of our forbearers, and true to our founding documents.”*



```

# =====
# 4. Define the RNN Language Model in PyTorch
# =====
class RNNLanguageModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_size,
num_layers, embedding_matrix):
        super(RNNLanguageModel, self).__init__()
        self.embedding = nn.Embedding(num_embeddings=vocab_size,
embedding_dim=embedding_dim)
        self.embedding.weight.data.copy_(embedding_matrix)
        # Optionally freeze the embeddings if desired:
        # self.embedding.weight.requires_grad = False
        self.rnn = nn.RNN(input_size=embedding_dim,
hidden_size=hidden_size, num_layers=num_layers,
nonlinearity='tanh', batch_first=True)
        # Fully connected layer to map hidden state to vocabulary
logits
        self.fc = nn.Linear(hidden_size, vocab_size)
    def forward(self, x, h0):
        # x: (batch_size, seq_len)
        x_embed = self.embedding(x) # (batch_size, seq_len,
embedding_dim)
        out, hn = self.rnn(x_embed, h0) # out: (batch_size, seq_len,
hidden_size)
        logits = self.fc(out) # (batch_size, seq_len, vocab_size)
        return logits, hn

hidden_size = 64
num_layers = 1

model = RNNLanguageModel(vocab_size, embedding_dim,
hidden_size, num_layers, embedding_matrix)

```

```

# =====
# 5. Define Loss Function, Optimizer, and a Learning Rate
Scheduler
# =====
criterion = nn.CrossEntropyLoss() # for next-word prediction
optimizer = optim.Adam(model.parameters(), lr=0.01)
scheduler = optim.lr_scheduler.StepLR(optimizer,
step_size=50, gamma=0.5)

# =====
# 6. Training (Fitting) Process
# =====
num_epochs = 200
model.train()
loss_history = []

batch_size, seq_len = input_seq.shape
for epoch in range(1, num_epochs + 1):
    h0 = torch.zeros(num_layers, batch_size, hidden_size)
    optimizer.zero_grad()
    # Forward pass: get logits over vocabulary
    logits, hn = model(input_seq, h0)
    # logits shape: (1, seq_len, vocab_size)
    # Reshape logits and target for loss computation
    logits = logits.view(-1, vocab_size) # shape:
(batch_size*seq_len, vocab_size)
    targets = target_seq.view(-1) # shape: (batch_size*seq_len)
    loss = criterion(logits, targets)
    loss.backward()
    optimizer.step()
    scheduler.step() # update learning rate
    loss_history.append(loss.item())

print("\n--- Training Completed ---")

```

```

# -----
# 7. Text Generation Based on the Trained Model
# -----
def generate_text(model, seed_text, length, word_to_idx,
idx_to_word, hidden_size, num_layers):

model.eval()
seed_tokens = tokenize(seed_text)
seed_indices = [word_to_idx.get(word, 0) for word in
seed_tokens] # default to 0 if not found
input_seq = torch.tensor([seed_indices], dtype=torch.long)
h = torch.zeros(num_layers, 1, hidden_size)
generated_indices = seed_indices.copy()

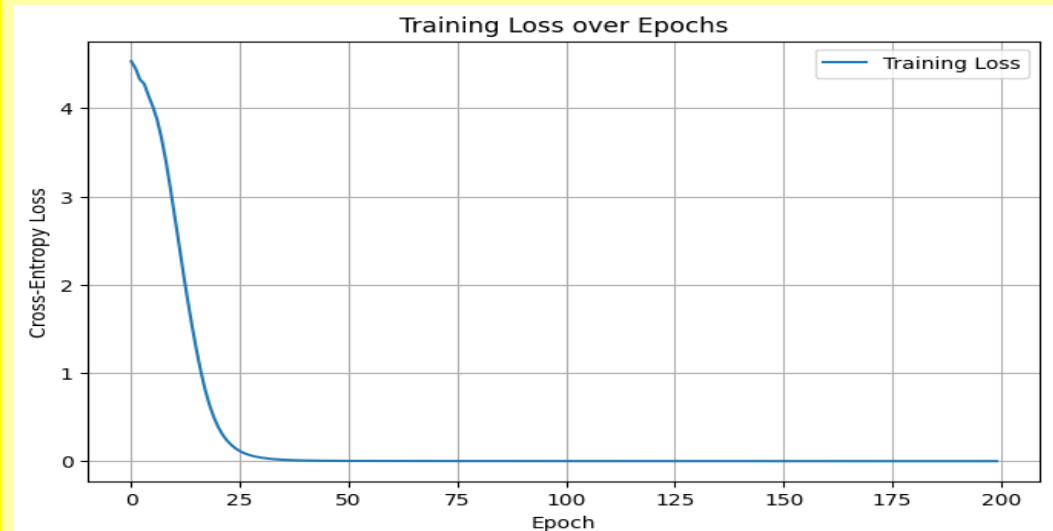
# Generate tokens one by one
for _ in range(length):
logits, h = model(input_seq, h)
last_logits = logits[:, -1, :]
next_token = last_logits.argmax(dim=1).item()
generated_indices.append(next_token)
# Prepare input for next iteration: the newly generated token
becomes the next input.
input_seq = torch.tensor([[next_token]], dtype=torch.long)
generated_words = [idx_to_word[idx] for idx in
generated_indices]
return " ".join(generated_words)
# Generate text from a seed
seed_text = "Obama"
generated_text = generate_text(model, seed_text, length=50,
word_to_idx=word_to_idx,
idx_to_word=idx_to_word, hidden_size=hidden_size,
num_layers=num_layers)
print("\n--- Generated Text ---")
print(generated_text)

```

```

--- RNN Language Model Architecture ---
RNNLanguageModel( (embedding): Embedding(93, 100) (rnn):
RNN(100, 64, batch_first=True) (fc): Linear(in_features=64,
out_features=93, bias=True) )
--- Loss Function and Optimizer ---
Loss Function: CrossEntropyLoss()
Optimizer: Adam ( Parameter Group 0 amsgrad: False betas:
(0.9, 0.999) capturable: False differentiable: False eps: 1e-
08 foreach: None fused: None initial_lr: 0.01 lr: 0.01
maximize: False weight_decay: 0 ) Scheduler:
<torch.optim.lr_scheduler.StepLR object at 0x7fc1615ae210>
--- Training Started ---
Epoch 1/200 | Loss: 4.5318 Epoch 20/200 | Loss: 0.5003 Epoch
40/200 | Loss: 0.0104 Epoch 60/200 | Loss: 0.0037 Epoch
80/200 | Loss: 0.0029 Epoch 100/200 | Loss: 0.0026 Epoch
120/200 | Loss: 0.0024 Epoch 140/200 | Loss: 0.0023 Epoch
160/200 | Loss: 0.0022 Epoch 180/200 | Loss: 0.0021 Epoch
200/200 | Loss: 0.0020
--- Training Completed ---

```



Outputs

- --- Generated Text ---

bush for his service to our nation as well as the generosity and cooperation he has shown throughout this transition forty four americans have now taken the presidential oath the words have been spoken during rising tides of prosperity and the still waters of peace yet every so often the oath

--- Generated Text ---

america has carried on not simply because of the skill or vision of those in high office but because we the people have remained faithful to the ideals of our forbearers and true to our founding documents for the trust you have bestowed mindful of the sacrifices borne by our ancestors

Evaluating Language Models

- ▶ **Perplexity:** Measures how well a model predicts a sample.
- ▶ **BLEU, ROUGE, METEOR:** Compare generated text to reference texts.
- ▶ **Accuracy and F1 Score:** Used in tasks with classification elements.
- ▶ **Human Evaluation:** Judges quality, fluency, and relevance.
- ▶ **Task-Specific Metrics:** Tailored metrics for particular applications.
 - ▶ **Fluency:** Is the generated text grammatically correct and natural?
 - ▶ **Coherence:** Does the text flow logically from one sentence to the next?
 - ▶ **Relevance:** How well does the generated text answer a prompt or capture key details?
 - ▶ **Engagement and Creativity:** Particularly important in creative writing or dialogue systems.

Normalized by.
number of words

$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T} = \prod_{t=1}^T \left(\frac{1}{\hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}} \right)^{1/T} = \exp \left(\frac{1}{T} \sum_{t=1}^T -\log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)} \right) = \exp(J(\theta))$$

Inverse probability of corpus, according to Language Model

Lower perplexity is better!

RNNs greatly improved perplexity

n-gram model →

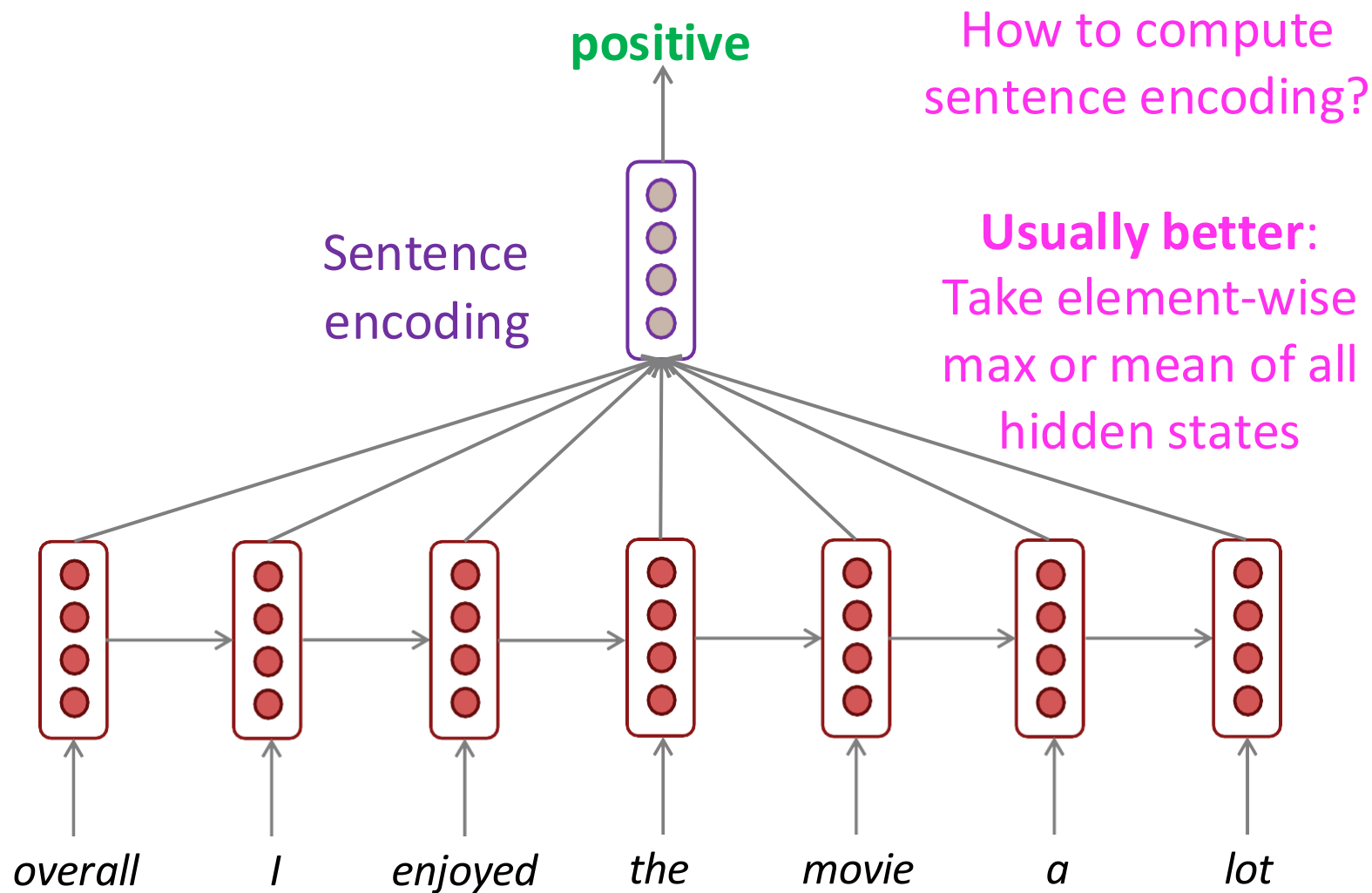
Increasingly
complex RNNs ↓

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

Perplexity improves
(lower is better)

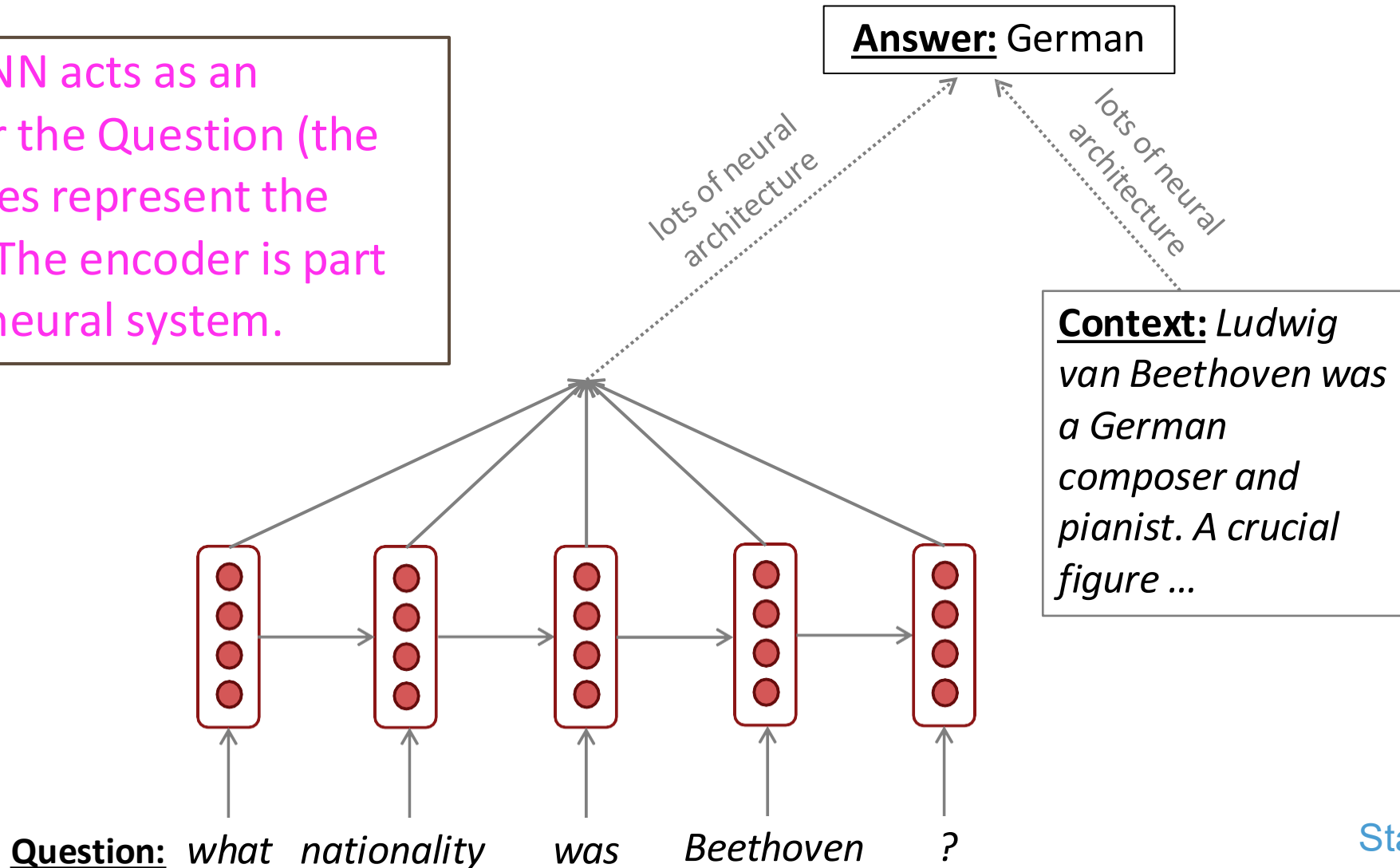
Source: <https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/>

Sentiment Classification

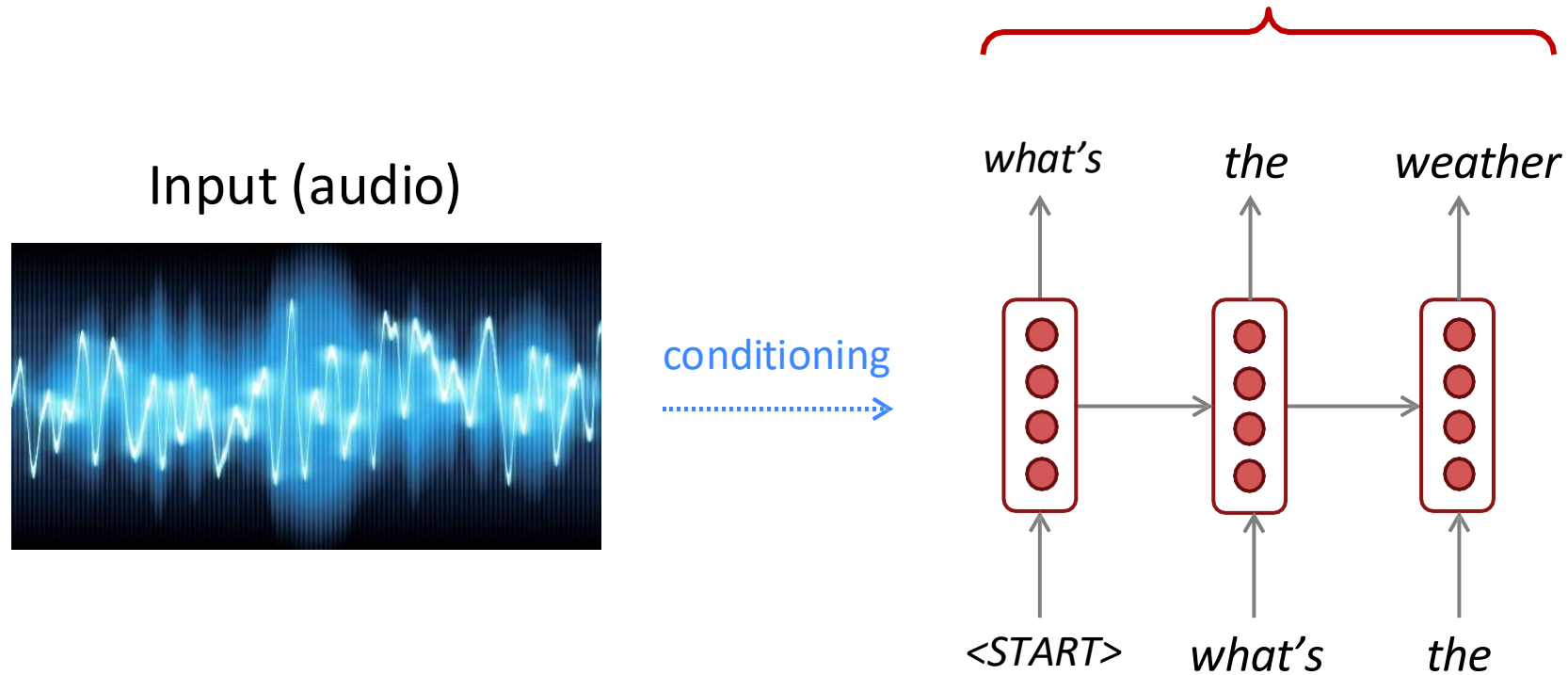


Question Answering

Here the RNN acts as an **encoder** for the Question (the hidden states represent the Question). The encoder is part of a larger neural system.



Speech Recognition



This is an example of a *conditional language model*.

Example: Classifying Names

```
import torch.nn as nn
import torch.nn.functional as F

class CharRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(CharRNN, self).__init__()

        self.rnn = nn.RNN(input_size, hidden_size)
        self.h2o = nn.Linear(hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, line_tensor):
        rnn_out, hidden = self.rnn(line_tensor)
        output = self.h2o(hidden[0])
        output = self.softmax(output)

        return output
```

```
n_hidden = 128
rnn = CharRNN(n_letters, n_hidden, len(alldata.labels_uniq))
print(rnn)
```

This tutorial serves as an introduction to **sequence modeling with RNNs** and shows how character-level information can be used for **text classification**.

- ❖ **Input:** A name (e.g., "Schmidt") is provided as a sequence of characters.
- ❖ **Processing:** Each character is converted into a numerical tensor (one-hot encoding or embeddings). The RNN processes the character sequence, updating its hidden state at each step.
- ❖ **Output:** The model predicts the **nationality/language** of the name (e.g., "German").

This is from the official PyTorch tutorial: https://pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html

Example: Classifying Names

```
import random
import numpy as np

def train(rnn, training_data, n_epoch = 10, n_batch_size = 64,
report_every = 50, learning_rate = 0.2, criterion = nn.NLLLoss()):
    """
    Learn on a batch of training_data for a specified number of
    iterations and reporting thresholds
    """
    # Keep track of losses for plotting
    current_loss = 0
    all_losses = []
    rnn.train()
    optimizer = torch.optim.SGD(rnn.parameters(), lr=learning_rate)

    start = time.time()
    print(f"training on data set with n = {len(training_data)}")

    for iter in range(1, n_epoch + 1):
        rnn.zero_grad() # clear the gradients

        batches = list(range(len(training_data)))
        random.shuffle(batches)
        batches = np.array_split(batches, len(batches) // n_batch_size )
```

```
        for idx, batch in enumerate(batches):
            batch_loss = 0
            for i in batch: #for each example in this batch
                (label_tensor, text_tensor, label, text) = training_data[i]
                output = rnn.forward(text_tensor)
                loss = criterion(output, label_tensor)
                batch_loss += loss

            # optimize parameters
            batch_loss.backward()
            nn.utils.clip_grad_norm_(rnn.parameters(), 3)
            optimizer.step()
            optimizer.zero_grad()

            current_loss += batch_loss.item() / len(batch)
            all_losses.append(current_loss / len(batches) )
        if iter % report_every == 0:
            print(f"{iter} ({iter / n_epoch:.0%}): = {all_losses[-1]}")
            current_loss = 0

    return all_losses

all_losses = train(rnn, train_set, n_epoch=27, learning_rate=0.15)
```

File: char_rnn_classification_tutorial.ipynb

This is from the official PyTorch tutorial: https://pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html

Content

1 Introduction to Sequence Modeling

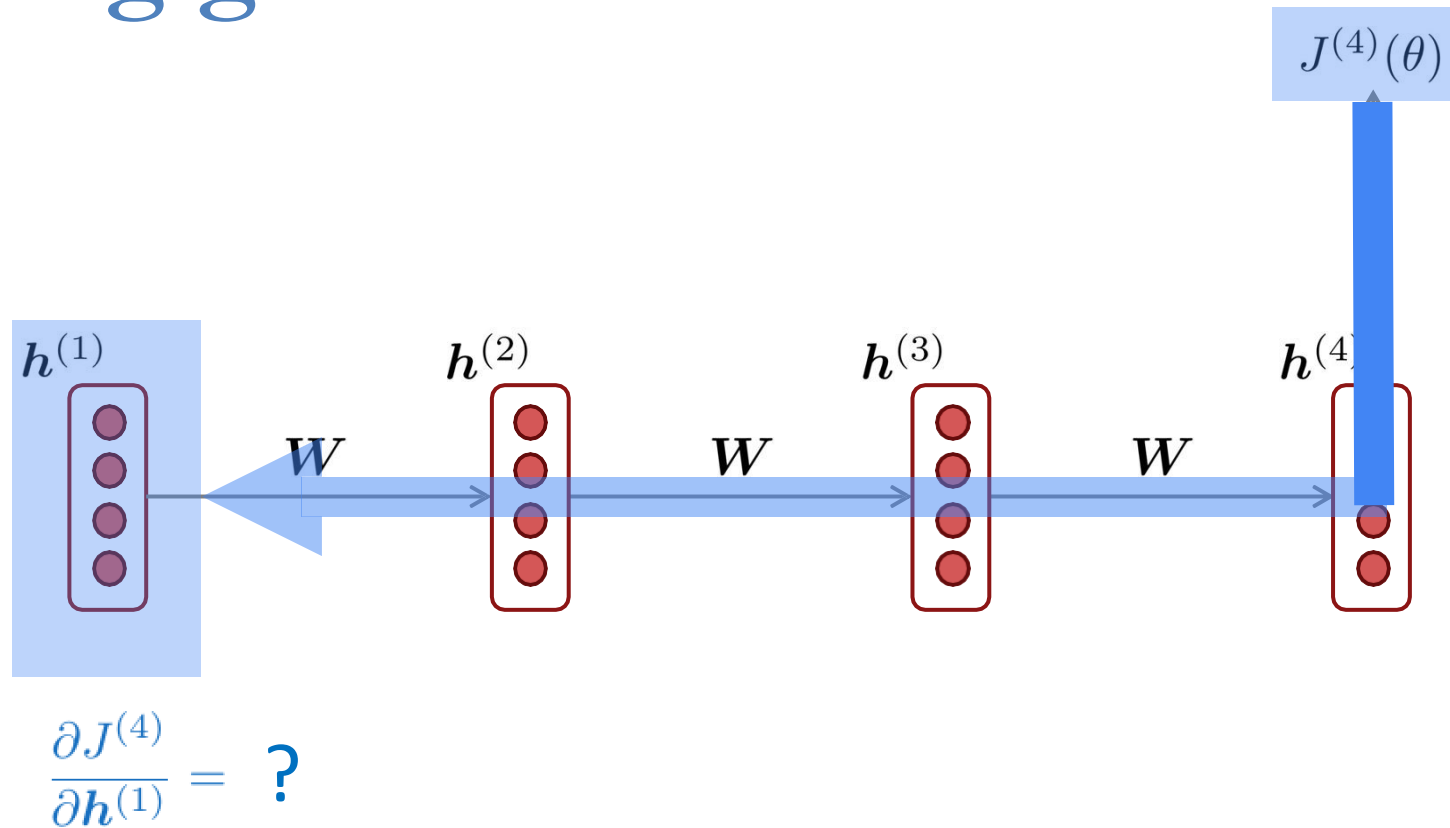
2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 Extensions of RNNs

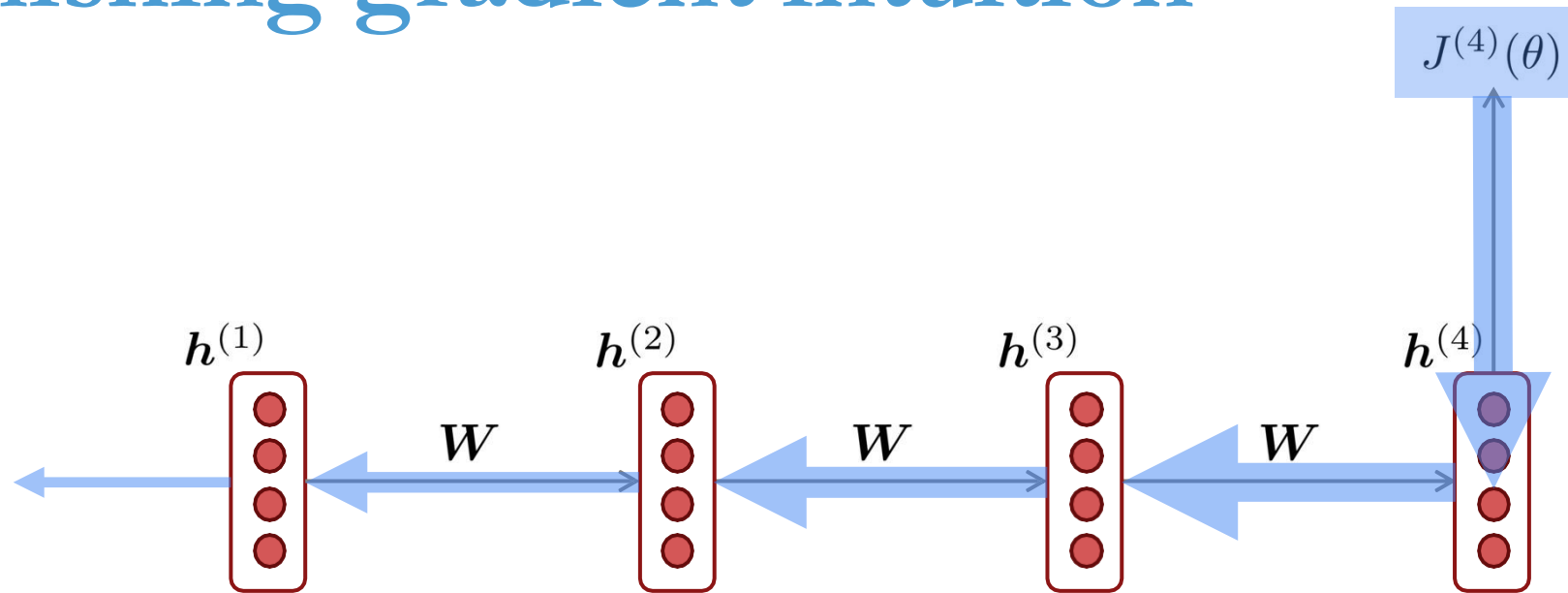
5 Genomic Sequence Analysis

Vanishing gradient intuition



- ▶ This decay makes it difficult for RNNs to learn long-term dependencies.
- ▶ Empirical evidence shows rapid gradient norm decay.

Vanishing gradient intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Vanishing gradient intuition

- Recall:

$$\mathbf{h}^{(t)} = \sigma \left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b}_1 \right)$$

- What if σ were the identity function, $\sigma(x) = x$?

$$\begin{aligned} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} &= \text{diag} \left(\sigma' \left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b}_1 \right) \right) \mathbf{W}_h && \text{(chain rule)} \\ &= \mathbf{I} \mathbf{W}_h = \mathbf{W}_h \end{aligned}$$

- Consider the gradient of the loss $J^{(i)}(\theta)$ on step i , with respect to the hidden state $\mathbf{h}^{(j)}$ on some previous step j . Let $\ell = i - j$

$$\begin{aligned} \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(j)}} &= \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} \prod_{j < t \leq i} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} && \text{(chain rule)} \\ &= \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} \prod_{j < t \leq i} \mathbf{W}_h = \frac{\partial J^{(i)}(\theta)}{\partial \mathbf{h}^{(i)}} \boxed{\mathbf{W}_h^\ell} && \text{(value of } \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \text{)} \end{aligned}$$

If $\mathbf{W}_h$ is “small”, then this term gets exponentially problematic as ℓ becomes large

Vanishing gradient intuition

- What's wrong with W_h^ℓ ?
- Consider if the eigenvalues of W_h are all less than 1:
 $\lambda_1, \lambda_2, \dots, \lambda_n < 1$
 $q_1, q_2, \dots, q_n$ (eigenvectors)
- We can write $\frac{\partial J^{(i)}(\theta)}{\partial h^{(i)}} W_h^\ell$ using the eigenvectors of W_h as a basis:

$$\frac{\partial J^{(i)}(\theta)}{\partial h^{(i)}} W_h^\ell = \sum_{i=1}^n c_i \lambda_i^\ell q_i \approx \mathbf{0} \text{ (for large } \ell \text{)}$$

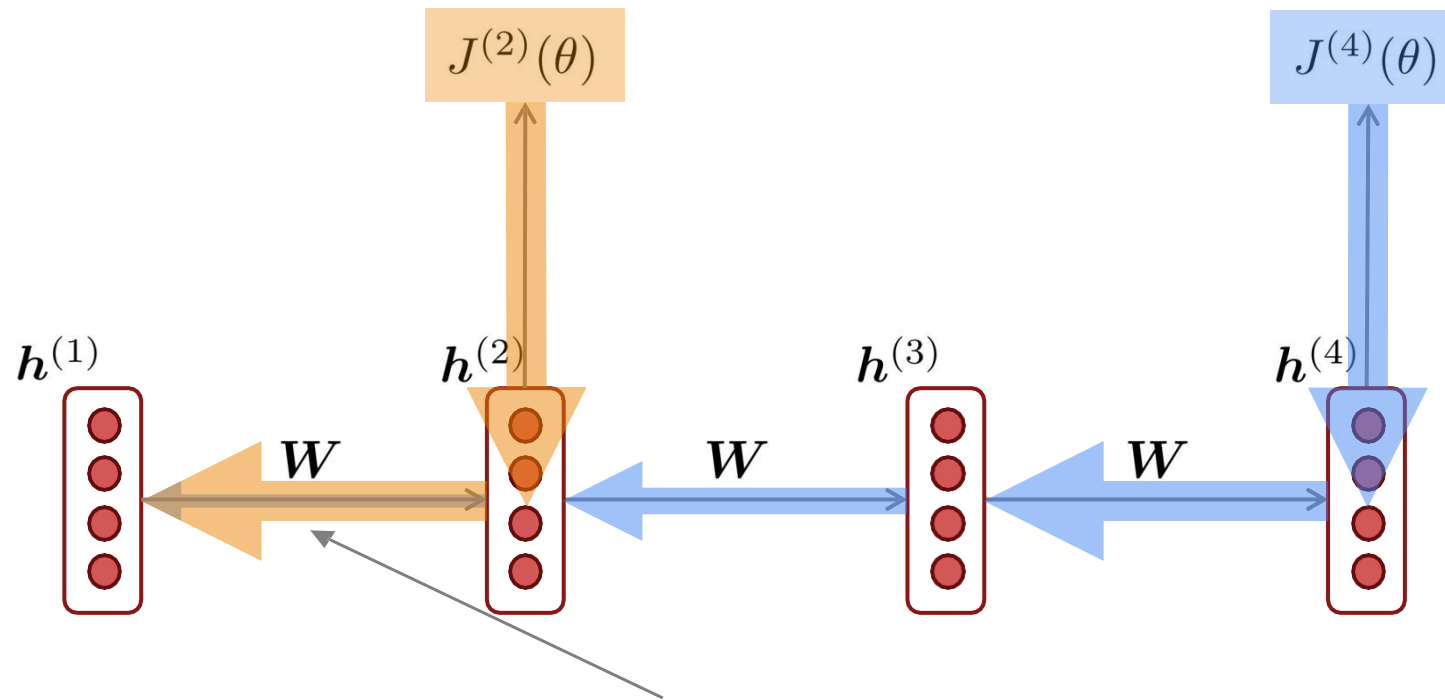
Approaches 0 as ℓ grows, so gradient vanishes

- What about nonlinear activations σ (i.e., what we use?)
 - Pretty much the same thing, except the proof requires $\lambda_i < \gamma$
for some γ dependent on dimensionality and σ

StanfordCS224n

Source: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013. <http://proceedings.mlr.press/v28/pascanu13.pdf>
(and supplemental materials), at <http://proceedings.mlr.press/v28/pascanu13-sup.pdf>

Why is vanishing gradient a problem?



- **Short-Range vs. Long-Range:** Imagine a long chain of dominos where each domino represents a layer or time step. If the force transferred from one domino to the next diminishes (say by a constant factor each time), then after a long chain, the force reaching the first domino becomes nearly zero. This means the first few dominos (or layers) barely "feel" the impact of the initial force (or error), and thus they do not adjust effectively based on long-term dependencies.
- **Resulting Behavior:** The model ends up "paying attention" only to the parts of the sequence that are immediately relevant (the nearby gradient signals) while ignoring distant context. This leads to challenges such as:
 - ❖ Inability to learn relationships or dependencies that span many time steps.
 - ❖ Poor performance on tasks that require integrating information over long sequences.

Effect of vanishing gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*

Problem:

The training example requires the model to understand that the first occurrence of "**tickets**" should influence the final output. However, due to the vanishing gradient, the error signal indicating that "**tickets**" was the correct prediction does not effectively travel back to where it is needed.

Outcome:

The model primarily updates its weights based on more immediate context (such as the recent words about installing toner), and as a result, it struggles to predict long-range dependencies like the repeated "**tickets**" at the end.

Key Takeaway:

When gradients vanish over long sequences, the model learns only the short-term (local) dependencies and fails to capture the long-term (global) context required to accurately model relationships spanning many time steps.

Why is exploding gradient a problem?

➤ Big Gradients = Big Updates:

If the gradient becomes too large, then multiplying it by the learning rate yields an update step that is far too big. This can drastically change the model's parameters in single update, causing the model to jump to a region in parameter space where the loss is extremely high.

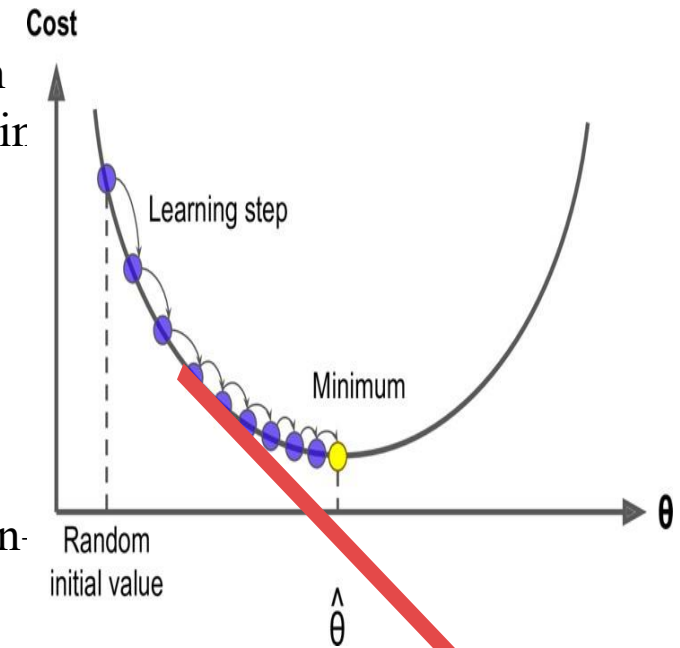
$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

➤ Overshooting and Divergence:

The model may overshoot any potential minima and end up in a poor configuration. figuratively, you might think you're following a path upward (finding a local minimum), but instead, you are suddenly in an entirely different and suboptimal region (like ending up in Iowa, far from your intended destination).

➤ Numerical Problems:

If updates are too extreme, you might encounter numerical issues like Inf or NaN values, which can halt training and require you to restart from a safe checkpoint.



Gradient clipping

- **Gradient clipping**: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

Stability:

With gradient clipping, you are less likely to encounter the situation where your network's parameters become so large that they result in numerical overflow (Inf or NaN values), which would require you to restart training.

Convergence:

Although exploding gradients can disrupt convergence, clipping helps maintain a controlled learning process where the update steps are kept within a safe range, thereby supporting steady convergence.

Focus on Other Challenges:

Since exploding gradients can be managed relatively easily through clipping, you can often shift your focus to more challenging issues like vanishing gradients or designing architectures that capture long-term dependencies.

Source: “On the difficulty of training recurrent neural networks”, Pascanu et al, 2013. <http://proceedings.mlr.press/v28/pascanu13.pdf>

How to fix the vanishing gradient problem?

➤ **Gated Architectures:**

LSTM and GRU models include gates that help regulate the flow of information and gradients over long time sequences, mitigating the vanishing gradient problem.

➤ **Activation Functions:**

ReLU and similar activations help reduce the saturation effect (common in sigmoid or $\tanh$), which can cause gradients to vanish.

➤ **Normalization Techniques:**

Batch or layer normalization helps to standardize activations and gradients, ensuring that they remain within a reasonable range.

➤ **Residual Connections:**

Adding skip connections enables gradients to flow directly from later layers to earlier layers, bypassing some of the multiplicative effects that cause vanishing.

➤ **Weight Initialization:**

Using initialization strategies like Xavier or He initialization keeps the scale of the activations and gradients more controlled.

Content

1 Introduction to Sequence Modeling

2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 Extensions of RNNs

5 Genomic Sequence Analysis

LSTMs: Apple WWDC Keynote 2016



Apple WWDC 2016: Apple is tentatively dipping its toe into the AI waters with technologies that can analyze your photos for faces and context - all done locally - and by applying LSTM deep learning technologies to Messaging.

[StanfordCS224n](#)

Long Short-Term Memory RNNs (LSTMs)

- The original proposal by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradient problem.
- Although that paper is widely cited, the modern LSTM architecture also owes much to innovations by Gers et al. (2000).
- The work of Alex Graves around 2006, who not only helped demonstrate the potential of LSTM models but also invented CTC for speech recognition.
- LSTM gained widespread attention when Hinton introduced it at Google in 2013, with Graves contributing significantly as his postdoc.



Hochreiter and Schmidhuber, 1997. Long short-term memory. <https://www.bioinf.jku.at/publications/older/2604.pdf>

Gers, Schmidhuber, and Cummins, 2000. Learning to Forget: Continual Prediction with LSTM. <https://dl.acm.org/doi/10.1162/089976600300015015>

Graves, Fernandez, Gomez, and Schmidhuber, 2006. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural nets. https://www.cs.toronto.edu/~graves/icml_2006.pdf

LSTMs: real-world success

- In 2013–2015, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the dominant approach for most NLP tasks
- Now (2019–2024), Transformers have become dominant for all tasks
 - For example, in WMT (a Machine Translation conference + competition):
 - In WMT 2014, there were 0 neural machine translation systems (!)
 - In WMT 2016, the summary report contains “RNN” 44 times (and these systems won)
 - In WMT 2019: “RNN” 7 times, “Transformer” 105 times

Source: "Findings of the 2016 Conference on Machine Translation (WMT16)", Bojar et al. 2016, <http://www.statmt.org/wmt16/pdf/W16-2301.pdf>

Source: "Findings of the 2018 Conference on Machine Translation (WMT18)", Bojar et al. 2018, <http://www.statmt.org/wmt18/pdf/WMT028.pdf>

Source: "Findings of the 2019 Conference on Machine Translation (WMT19)", Barrault et al. 2019, <http://www.statmt.org/wmt18/pdf/WMT028.pdf>

Long Short-Term Memory RNNs (LSTMs)

- At each time step t , the LSTM maintains a **hidden state** $h(t)$ and a **cell state** $c(t)$, both vectors of length n .
- The cell state stores **long-term information**, while the hidden state represents **the immediate output**.
- Three gates—the **forget gate**, **input gate**, and **output gate**—dynamically control the **erasing, reading, and writing** of information in the cell state.
- Each gate is computed as a vector with values between 0 and 1, where **values indicate the proportion of information to keep or update**.
- This gating mechanism allows the LSTM to **selectively maintain important information over long sequences**, effectively addressing the vanishing gradient problem encountered in traditional RNNs.

This dynamic gating system is a key innovation that makes LSTMs powerful for tasks requiring long-term memory, such as language modeling, speech recognition, and time-series prediction.

Long Short-Term Memory (LSTM)

A sequence of inputs $\mathbf{x}^{(t)}$. Compute a sequence of hidden states $\mathbf{h}^{(t)}$ and cell states $\mathbf{c}^{(t)}$. On timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$\mathbf{f}^{(t)} = \sigma \left(\mathbf{W}_f \mathbf{h}^{(t-1)} + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{b}_f \right)$$

$$\mathbf{i}^{(t)} = \sigma \left(\mathbf{W}_i \mathbf{h}^{(t-1)} + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{b}_i \right)$$

$$\mathbf{o}^{(t)} = \sigma \left(\mathbf{W}_o \mathbf{h}^{(t-1)} + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{b}_o \right)$$

$$\tilde{\mathbf{c}}^{(t)} = \tanh \left(\mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c \right)$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)}$$

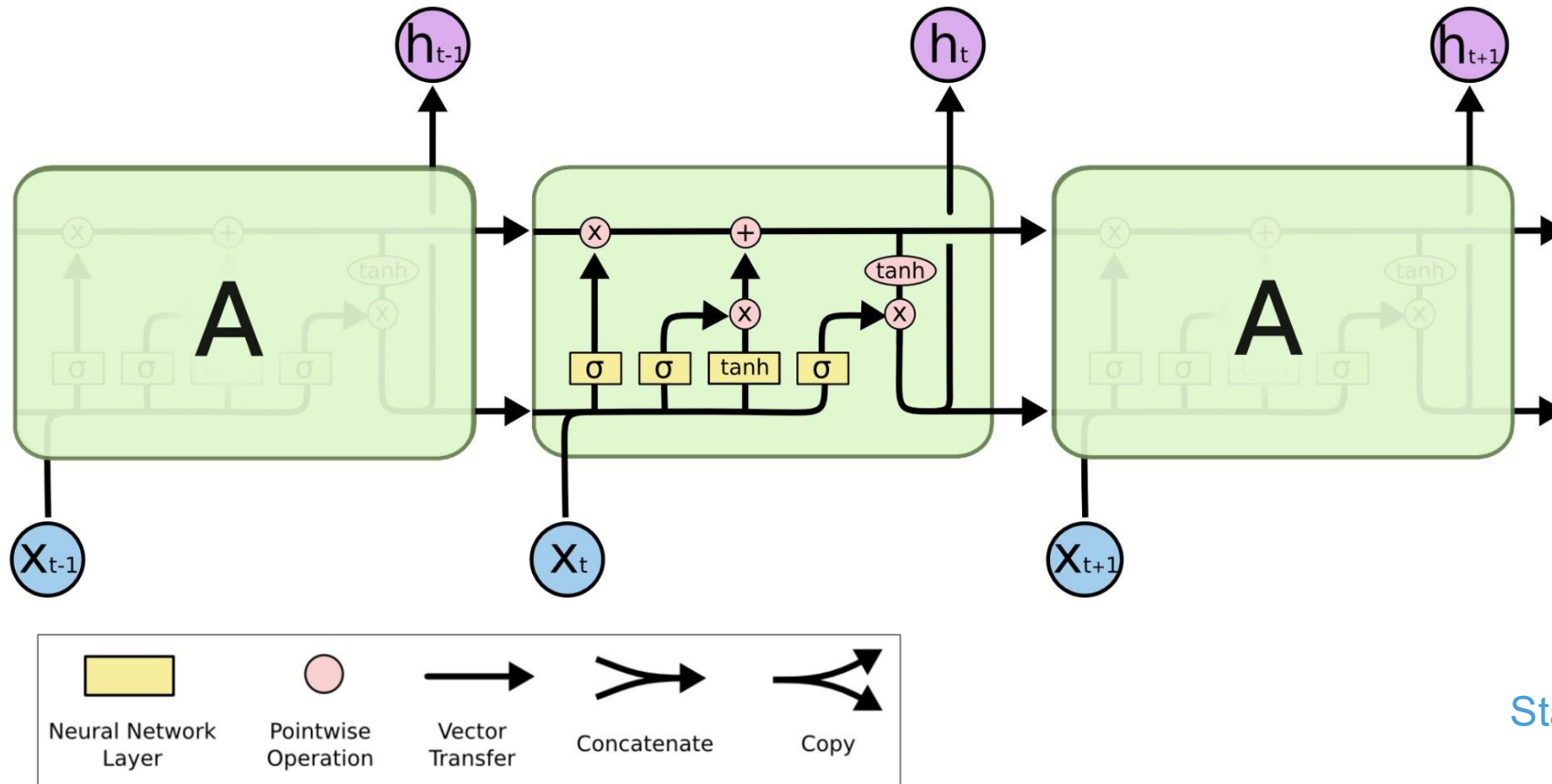
$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh \mathbf{c}^{(t)}$$

All these are vectors of same length n

Gates are applied using element-wise (or Hadamard) product: $\odot$

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:

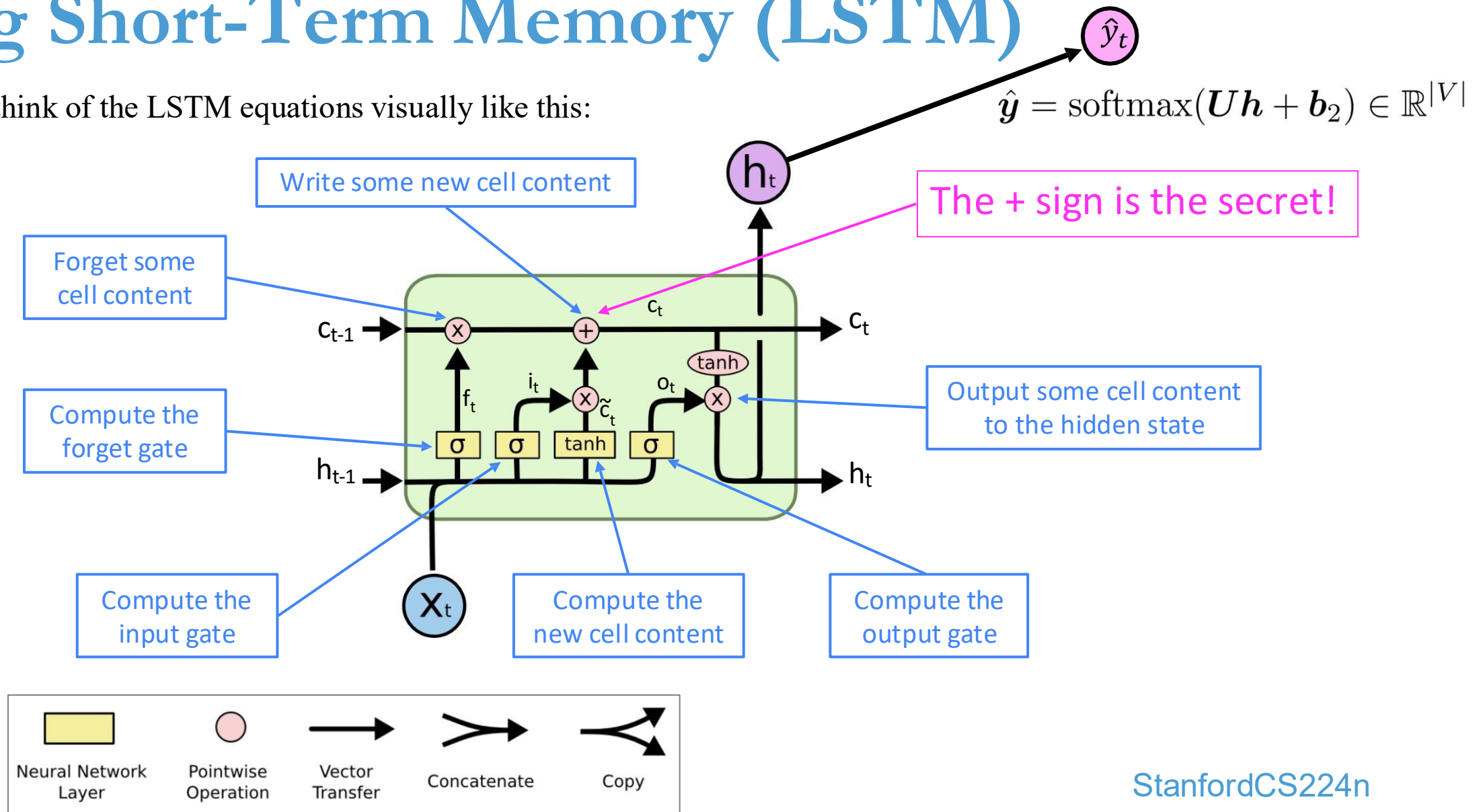


StanfordCS224n

Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



StanfordCS224n

Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Preserving Long-Term Dependencies

LSTMs vs. Vanilla RNNs

- **LSTM Advantage:**

The LSTM architecture makes it much easier for an RNN to preserve information over many timesteps. Example: If the forget gate is set to 1 and the input gate to 0 for a cell dimension, the corresponding cell state is preserved indefinitely.

- **Vanilla RNN Limitation:**

A vanilla RNN must learn a recurrent weight matrix that preserves information in the hidden state. In practice, vanilla RNNs typically manage to preserve information over only about 7 timesteps.

- **Extended Memory with LSTMs:**

LSTMs can effectively preserve information for around 100 timesteps, greatly enhancing the model's ability to capture long-term dependencies.

- **Alternative Approaches:**

There are other methods to create direct, linear pass-through connections that capture long-distance dependencies.

Common Problems

➤ Universality of the Problem:

Vanishing gradients are not unique to a single type of network—they affect all architectures, especially as depth increases.

➤ Root Cause:

It explains how the chain rule and activation function choices cause gradients to decay, which is the mathematical reason behind the problem.

➤ Impact on Learning:

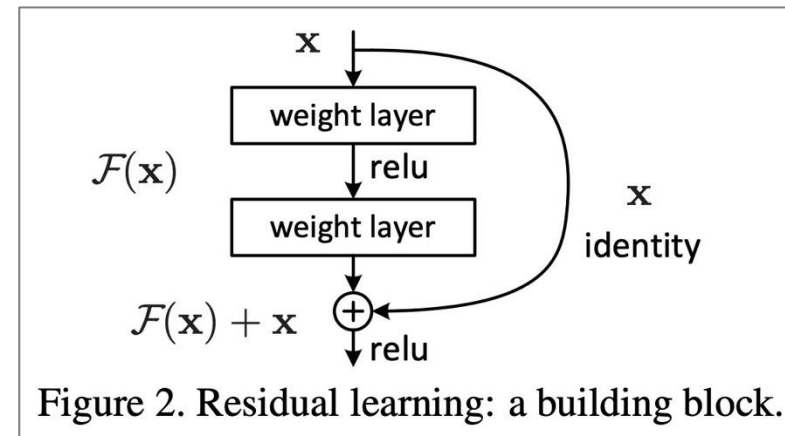
With vanishing gradients, lower layers learn very slowly, affecting the overall performance and convergence of the network.

➤ Architectural Remedies:

Modern networks counteract this problem by adding direct connections (skip/residual connections) that help maintain gradient flow, with examples like ResNets and DenseNets.

For example:

- Residual connections aka “ResNet”
- Also known as skip-connections
- The identity connection preserves information by default
- This makes deep networks much easier to train



"Deep Residual Learning for Image Recognition", He et al, 2015. <https://arxiv.org/pdf/1512.03385.pdf>

Code: Classifying Names with a LSTM

```
import torch.nn as nn
import torch.nn.functional as F

class CharLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(CharLSTM, self).__init__()

        # Replace RNN with LSTM
        self.lstm = nn.LSTM(input_size, hidden_size)
        self.h2o = nn.Linear(hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

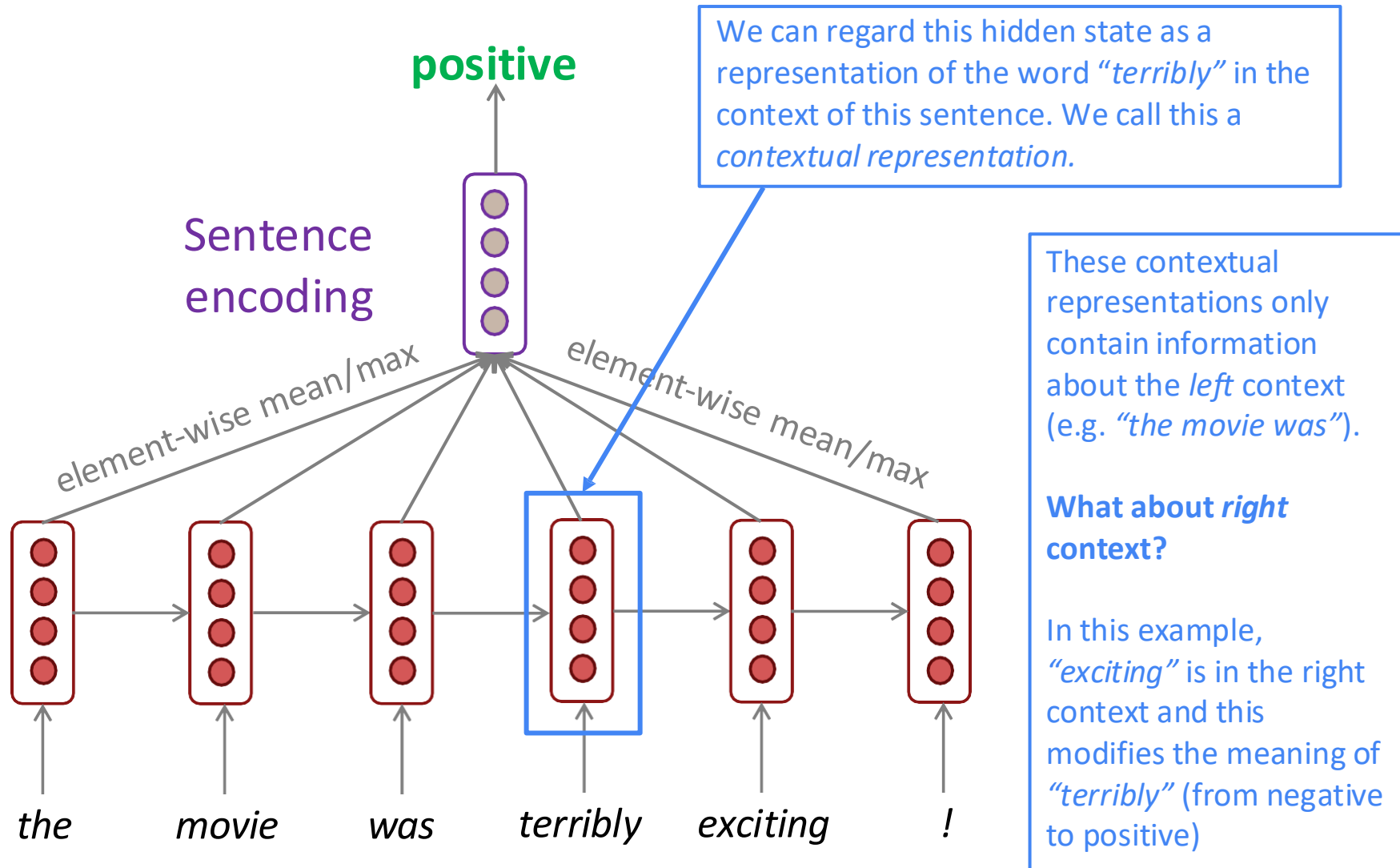
    def forward(self, line_tensor):
        # LSTM produces two outputs: output and (hidden_state, cell_state)
        lstm_out, (hidden_state, cell_state) = self.lstm(line_tensor)
        # Use the last hidden state to produce output
        output = self.h2o(hidden_state[-1])
        output = self.softmax(output)

        return output

lstm = CharLSTM(n_letters, n_hidden=128, len(alldata.labels_uniq))
```

Bidirectional and Multi-layer RNNs

Task: Sentiment Classification



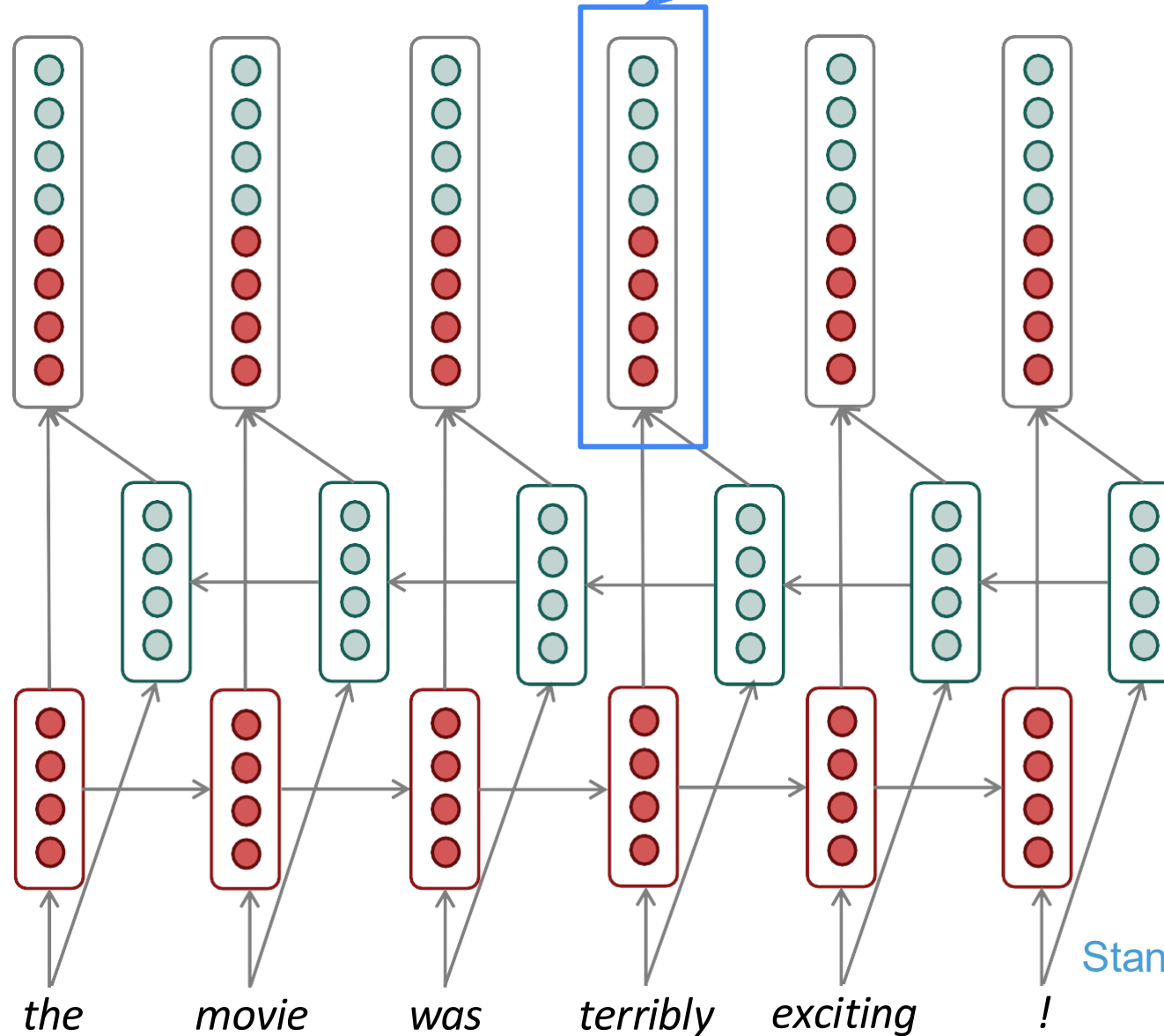
Bidirectional RNNs

This contextual representation of “terribly” has both left and right context!

Concatenated
hidden states

Backward RNN

Forward RNN



StanfordCS224n

Bidirectional RNNs

On timestep t :

This is a general notation to mean “compute one forward step of the RNN” – it could be a simple RNN or LSTM computation.

Forward RNN $\vec{h}^{(t)} = \text{RNN}_{\text{FW}}(\vec{h}^{(t-1)}, \mathbf{x}^{(t)})$

Backward RNN $\overleftarrow{h}^{(t)} = \text{RNN}_{\text{BW}}(\overleftarrow{h}^{(t+1)}, \mathbf{x}^{(t)})$

Generally, these two RNNs have separate weights

Concatenated hidden states $\mathbf{h}^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$

We regard this as “the hidden state” of a bidirectional RNN. This is what we pass on to the next parts of the network.

StanfordCS224n

Pros and Cons of Bidirectional RNNs

Pros Section:

- **Enhanced Context:** Emphasizes that BiRNNs incorporate both past and future context, leading to richer representations.
- **Improved Accuracy:** Highlights the improved performance in various tasks.
- **Better Long-Term Dependency Modeling:** Points out that the dual-direction approach can capture dependencies over longer sequences.

Cons Section:

- ❖ **Increased Computational Cost:** Notes the extra resources required for running both forward and backward RNNs.
- ❖ **Not Suitable for Real-Time Applications:** Explains that access to future context makes BiRNNs less practical in scenarios where such information isn't available.
- ❖ **Complexity:** Mentions that the increased complexity can make training and tuning more challenging.

Bidirectional RNNs

- ▶ Bidirectional RNNs require access to the entire input sequence to compute both forward and backward passes. They are best suited for tasks where the complete sequence is available in advance (e.g., sequence encoding, text classification, and machine translation).
- ▶ In Language Modeling (LM), the model generates text one token at a time. At each generation step, only the left context (previous tokens) is available. Bidirectional RNNs are not applicable to LM because future context is unknown during generation.
- ▶ When the entire input sequence is available (e.g., for encoding), bidirectionality is very powerful. It provides a richer representation by capturing both past and future context for each token.
- ▶ Each token's embedding in BERT is contextualized with both left and right information. Typically, BERT models produce embeddings of 768 or 1024 dimensions. BERT has set new standards for many NLP tasks by providing rich, pretrained representations.

Code: Bidirectional RNNs

```
class CharBiRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(CharBiRNN, self).__init__()

        # Bidirectional RNN
        self.rnn = nn.RNN(input_size, hidden_size, bidirectional=True)
        # Adjust output size to account for bidirectional hidden states (2 * hidden_size)
        self.h2o = nn.Linear(hidden_size * 2, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, line_tensor):
        # rnn_out contains outputs for all timesteps, hidden contains the last hidden state
        rnn_out, hidden = self.rnn(line_tensor)
        # Combine the last forward and backward hidden states
        hidden_cat = torch.cat((hidden[-2], hidden[-1]), dim=1)
        output = self.h2o(hidden_cat)
        output = self.softmax(output)
        return output

bidir_rnn = CharBiRNN(n_letters, n_hidden, len(alldata.labels_uniq))
```

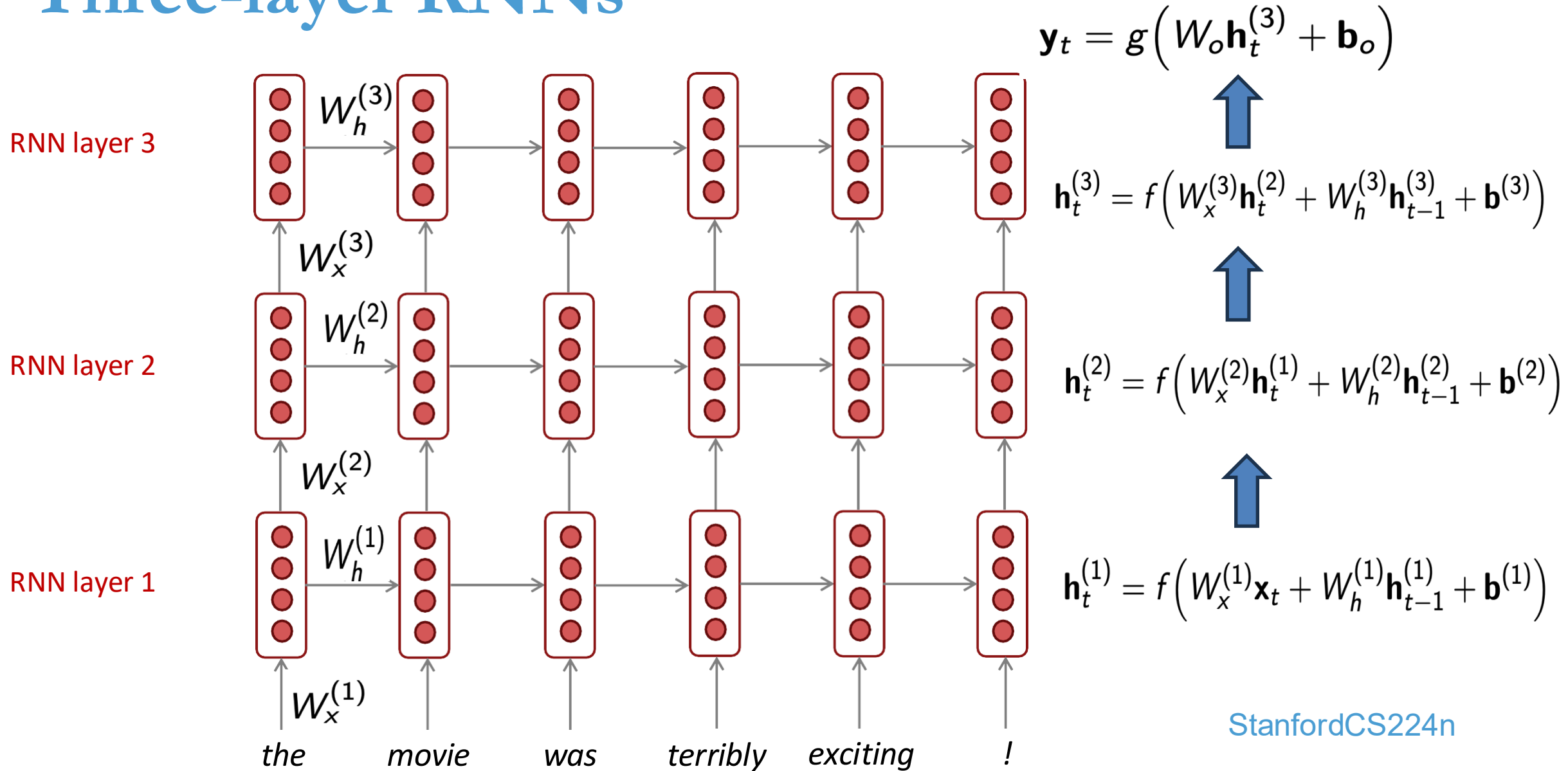
Stacked RNNs

Stacked RNNs (also known as deep RNNs): multiple recurrent layers are placed on top of each other.

- ▶ The output of each RNN layer serves as the input to the next, creating a hierarchical representation of the sequential data. The **lower RNNs** should **compute lower-level features** and the **higher RNNs** should compute **higher-level features**.
- ▶ This architecture is used to capture complex, abstract temporal patterns. Hierarchical layers can learn more abstract features. Better capture complex temporal dependencies.
- ▶ Improved Performance: Often achieve higher accuracy on tasks such as language modeling, speech recognition, and time series prediction.
- ▶ However, they also introduce challenges such as increased training complexity and computational cost.
- ▶ With careful design and tuning, stacked RNNs are a powerful tool for sequence modeling.



Three-layer RNNs



StanfordCS224n

Multi-layer RNNs in practice

- ❖ **Stacked RNNs allow for more complex, hierarchical representations:** Lower layers capture basic, low-level features, while higher layers integrate these into more abstract, high-level features.'
- ❖ **Performance improvements are observed with moderate stacking:** Empirical results (e.g., from Britz et al., 2017) indicate that 2–4 layers can be optimal for tasks such as machine translation.
- ❖ **Deep RNNs require architectural innovations:** For very deep RNNs (e.g., 8 layers or more), skip-connections or dense connections are necessary to maintain gradient flow and facilitate training.
- ❖ **Transformers push depth further with built-in residual connections:** Models like BERT, which can have 12–24 layers, offer a different approach to capturing long-range dependencies via self-attention and deep stacking.

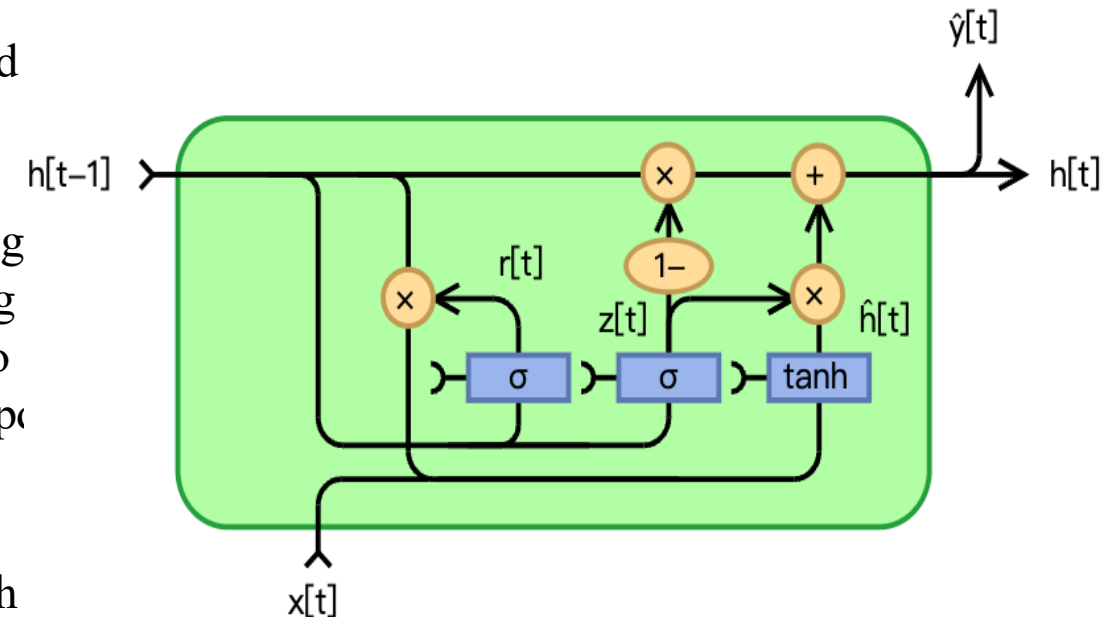
Code: Multi-layer Bidirectional RNNs

```
class MultiLayerBiRNNClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_layers):
        super(MultiLayerBiRNNClassifier, self).__init__()
        # Multi-layer bidirectional RNN
        self.rnn = nn.RNN(
            input_size,
            hidden_size,
            num_layers=num_layers,
            bidirectional=True,
            batch_first=True
        )
        # Adjust the linear layer to account for bidirectional hidden states
        self.h2o = nn.Linear(hidden_size * 2, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, input_tensor):
        # rnn_out: Outputs from all time steps, hidden: Hidden states for all layers
        rnn_out, hidden = self.rnn(input_tensor)
        # Combine the last hidden states from both directions in the last layer
        hidden_cat = torch.cat((hidden[-2], hidden[-1]), dim=1)
        output = self.h2o(hidden_cat)
        output = self.softmax(output)
        return output
```

GRU (Gate Recurrent Unit)

- **Gate Recurrent Unit** is one of the ideas that has enabled RNN to become much better at capturing very **long-range dependencies** and made RNN much more effective.
- The GRU is like a LSTM with a **gating mechanism** to input or forget certain features, but lacks a context vector or output gate, resulting in **fewer parameters** than LSTM. Proposed as a simpler alternative to LSTM, the GRU merges the forget and input gates into a single update gate.
- GRUs are known for having fewer parameters than LSTMs, which lead to faster training and similar performance in many tasks.



Each GRU cell has two main gates:

- ▶ Reset Gate ($r(t)$): Controls how much of the previous hidden state to forget.
- ▶ Update Gate ($z(t)$): Decides how much of the candidate activation to use.
- ▶ The GRU combines these gates to update its hidden state without a separate cell state.

1. **Update Gate (z_t)**: Determines how much of the past hidden state h_{t-1} should be retained and how much of the new candidate state $\hat{h}_{t-1}$ should be added to form the current hidden state.

$$z_t = \sigma(W_z \cdot x_t + U_z \cdot h_{t-1} + b_z)$$

2. **Reset Gate (r_t)**: Determines how much of the past hidden state h_{t-1} contributes to the computation of the new candidate state $\hat{h}_{t-1}$.

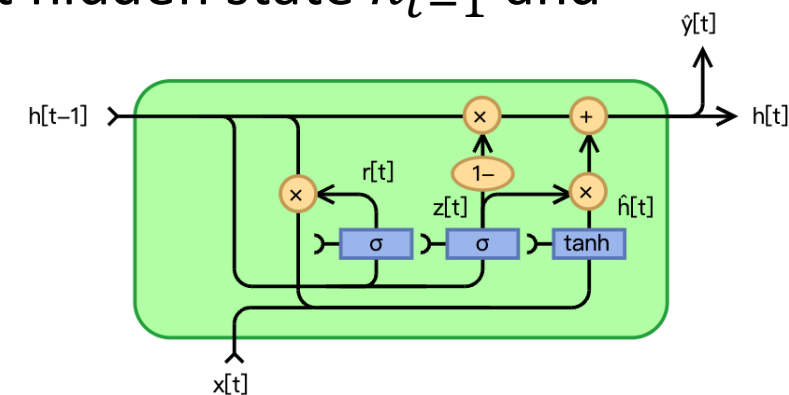
$$r_t = \sigma(W_r \cdot x_t + U_r \cdot h_{t-1} + b_r)$$

3. **Candidate State ($\hat{h}_t$)**: New information computed at the current time step.

$$\hat{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}) + b_h)$$

4. **Hidden State Update**: Combines contributions from the past hidden state h_{t-1} and the new candidate state $\hat{h}_{t-1}$ using the update gate z_t .

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \hat{h}_t$$



Classifying Names with a Character-Level GRU

```
import torch.nn as nn
import torch.nn.functional as F

class CharGRU(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(CharGRU, self).__init__()

        # Replace RNN with GRU
        self.gru = nn.GRU(input_size, hidden_size)
        self.h2o = nn.Linear(hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, line_tensor):
        # GRU produces outputs and hidden states
        gru_out, hidden = self.gru(line_tensor)
        # Use the last hidden state for classification
        output = self.h2o(hidden[-1])
        output = self.softmax(output)

        return output
```

Content

1 Introduction to Sequence Modeling

2 Introduction to Recurrent Neural Networks (RNNs)

3 Problems with RNNs

4 Extensions of RNNs

5 Applications: Genomic Sequence Analysis

Functional Effects of Noncoding Variants

- Most disease-associated variants identified by GWAS are located in noncoding regions of the genome.
- These variants can have significant regulatory effects. For example, they may modify transcription factor binding sites, alter chromatin accessibility, or impact epigenetic marks such as DNA methylation and histone modifications. These changes can, in turn, affect gene expression patterns and contribute to the development of complex diseases such as cancer, diabetes, and autoimmune disorders.
- Predicting the functional effects of these noncoding variants is therefore crucial for understanding the genetic basis of complex diseases and for identifying potential therapeutic targets. Traditional approaches relying on hand-crafted features or statistical methods often struggle to capture the intricate, nonlinear relationships within genomic data.
- Deep learning offers a powerful alternative by automatically learning hierarchical representations directly from raw genomic sequences. With the ability to model both local and long-range dependencies, deep learning models can identify subtle sequence patterns and interactions that are critical for gene regulation. This capability has led to significant advances in predicting regulatory functions and understanding the mechanisms by which noncoding variants contribute to disease pathology.

Deep Learning-based Sequence Analyzer

❖ DeepSEA Model Architecture

- Incorporates wide sequence context (1 kbp) and use one-hot to encode the final sequence.
- Uses hierarchical convolutional neural networks (CNNs) to learn sequence dependencies.
- Employs multitask learning for sharing predictive features across chromatin factors.
- Optimized for accuracy in functional variant prediction.

One-Hot Encoding

- ▶ A → [1, 0, 0, 0]
- ▶ C → [0, 1, 0, 0]
- ▶ G → [0, 0, 1, 0]
- ▶ T → [0, 0, 0, 1]

❖ Training and Data Sources

- Training data from ENCODE and Roadmap Epigenomics projects.
- 690 TF binding profiles, 125 DNase I hypersensitivity profiles, 104 histone-mark profiles.
- Covers 17% of the human genome (521.6 Mbp).
- Trained using CNN.

Output:
variant functionality
prediction

Output:
predicted chromatin
effect

Output:
predicted allele-
specific chromatin
profile

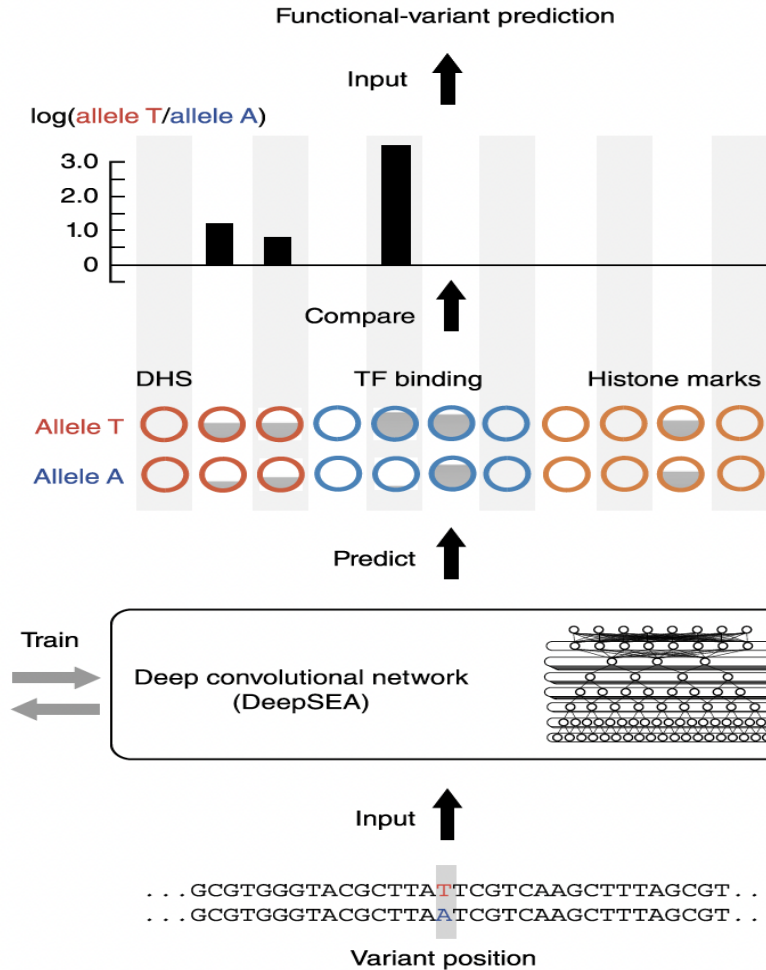
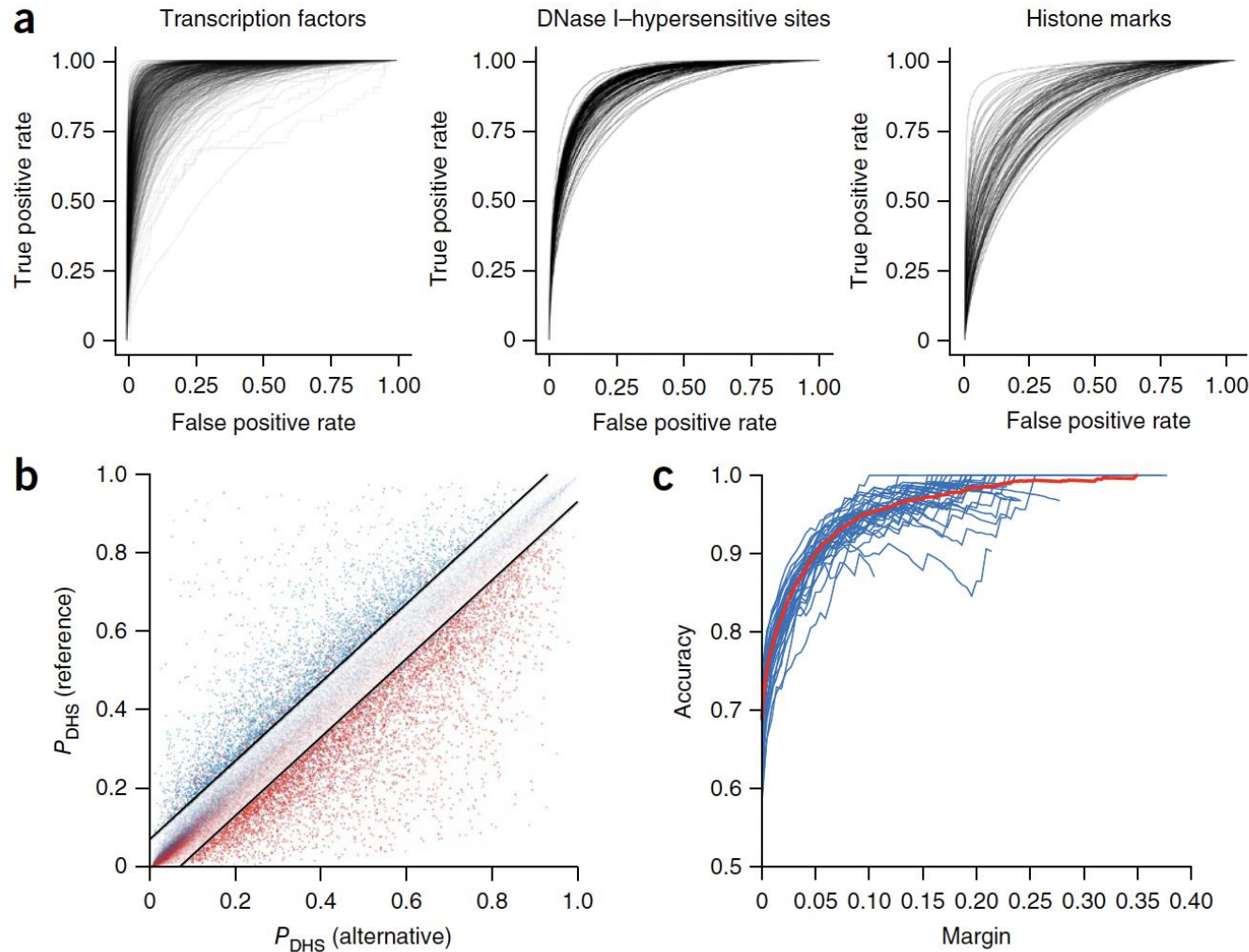


Figure 1 | Schematic overview of the DeepSEA pipeline, a strategy for predicting chromatin effects of noncoding variants.

Zhou, J., & Troyanskaya, O. G. (2015)

Performance and Sensitivity



- DeepSEA achieves high accuracy in predicting chromatin features:

- **TF binding: AUC = 0.958**
- **DNase I sensitivity: AUC = 0.923**
- **Histone marks: AUC = 0.856**

- Outperforms gkm-SVM on nearly all TFs.
- Enables accurate sequence-based functional predictions.

- Uses 'in silico saturated mutagenesis' to assess sequence feature importance.
- Evaluated on 57,407 allelically imbalanced SNPs from 35 cell types.
- Achieves >95% accuracy for high-confidence predictions.
- Consistently predicts known SNP effects on TF binding (e.g., FOXA1, GATA1, FOXA2).

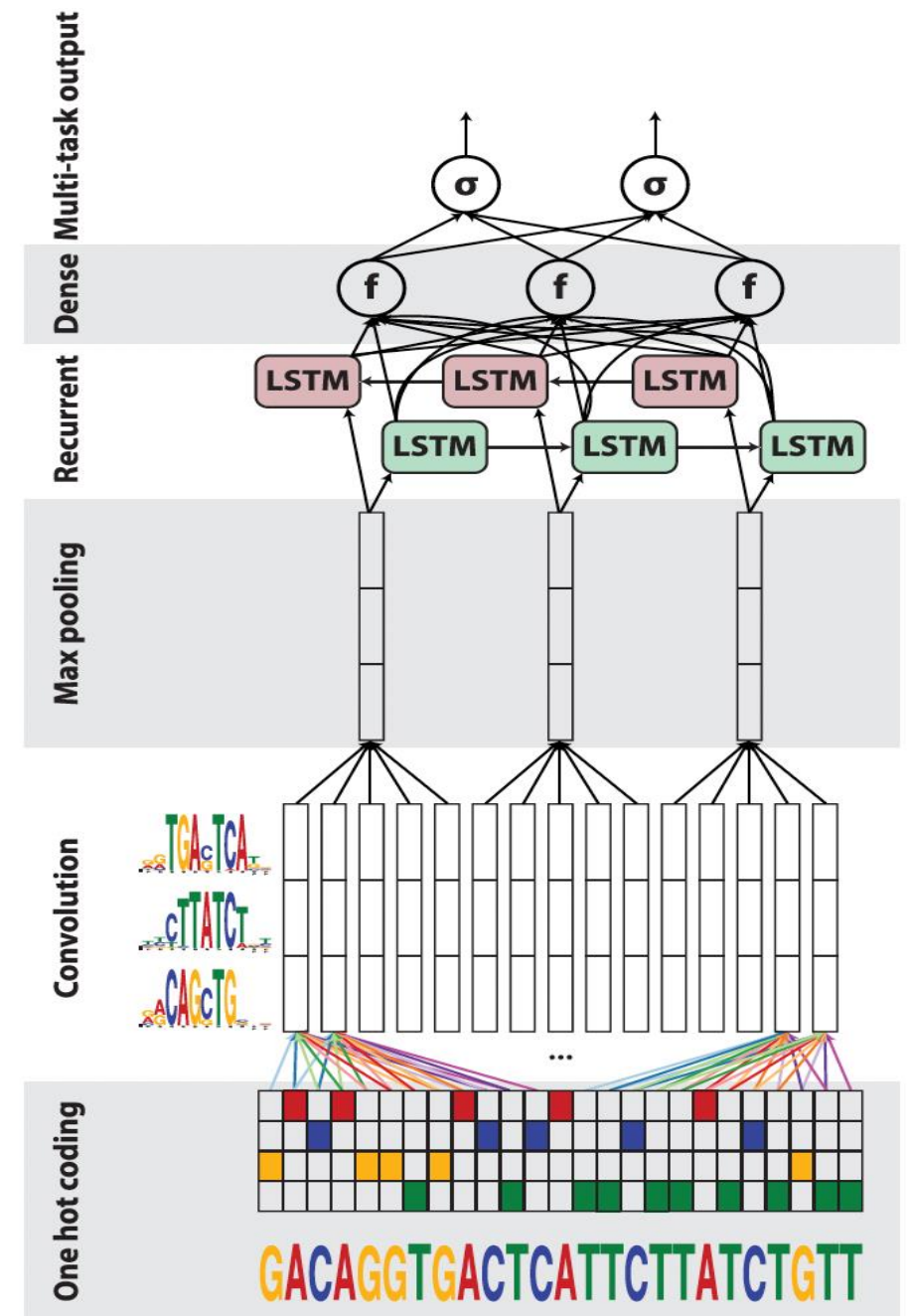
DanQ

❖ Motivations

- Previous models like DeepSEA and gkm-SVM predict regulatory function but lack recurrent components.
- Need for a model that integrates motif discovery and long-term dependencies in DNA sequences.

❖ DanQ Model Framework

- **Input Data:** One-hot encoded 1000-bp DNA sequences from GRCh37 genome.
- **Training Data:** 919 chromatin features from ENCODE & Roadmap Epigenomics datasets.
- **Architecture:**
 - CNN layer for motif scanning.
 - Max pooling layer to reduce spatial size.
 - BLSTM layer to capture motif relationships.
 - Fully connected layer with sigmoid outputs.



Quang, D., & Xie, X. (2016).

Performance and Sensitivity

❖ Comparisons:

- ROC AUC: DanQ outperforms DeepSEA for 94.1% of targets.
- PR AUC: DanQ achieves over 50% relative improvement on some markers.
- Enhanced motif discovery compared to previous CNN-based models.

❖ Results

- ROC and PR curves demonstrate significant performance gain over DeepSEA.
- • Motif analysis shows DanQ effectively learns biologically relevant patterns.
- • Functional SNP prioritization shows improved detection of regulatory variants

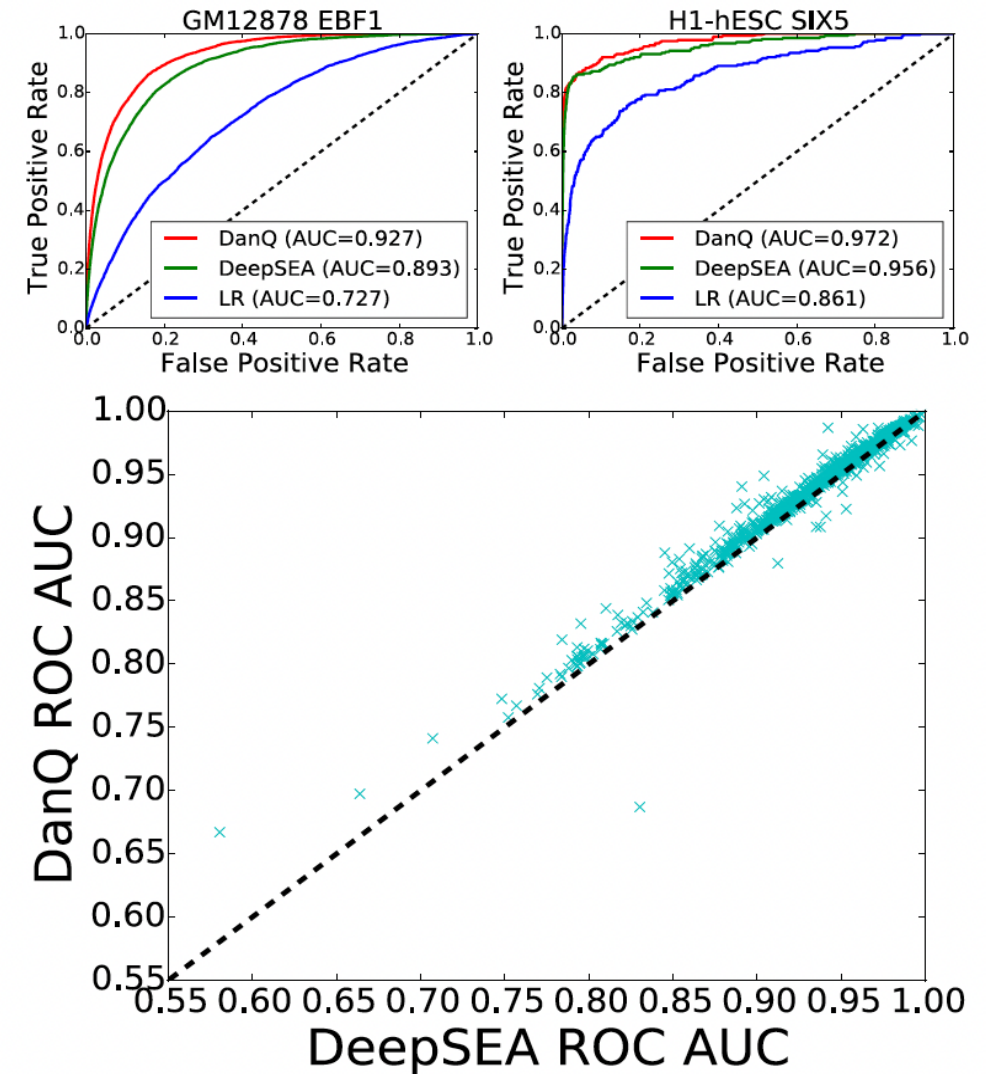


Figure 2. (Top) ROC curves for the GM12878 EBF1 and H1-hESC SIX5 targets comparing the performance of the three models. (Bottom) Scatterplot comparing DanQ and DeepSEA ROC AUC scores. DanQ outperforms DeepSEA for 94.1% of the targets in terms of ROC AUC.

References

- Baek, J. W., & Chung, K. Y. (2021). Multimedia recommendation using Word2Vec-based social relationship mining. *Multimedia Tools and Applications*, 80(1), 1-18.
- Levy, O., Goldberg, Y., & Dagan, I. (2015). Improving distributional similarity with lessons learned from word embeddings. *Transactions of the association for computational linguistics*, 3, 211-225.
- Lipton, Z. C. (2015). A Critical Review of Recurrent Neural Networks for Sequence Learning. *arXiv Preprint, CoRR*, abs/1506.00019.
- Mienye, I. D., Swart, T. G., & Obaido, G. (2024). Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, 15(9), 517.
- Novakovsky, G., Dexter, N., Libbrecht, M. W., Wasserman, W. W., & Mostafavi, S. (2023). Obtaining genetics insights from deep learning via explainable artificial intelligence. *Nature Reviews Genetics*, 24(2), 125-137.
- Pennington, J., Socher, R., & Manning, C. D. (2014, October). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532-1543).
- Quang, D., & Xie, X. (2016). DanQ: a hybrid convolutional and recurrent deep neural network for quantifying the function of DNA sequences. *Nucleic acids research*, 44(11), e107-e107.
- Sapoval, N., Aghazadeh, A., Nute, M. G., Antunes, D. A., Balaji, A., Baraniuk, R., ... & Treangen, T. J. (2022). Current progress and open challenges for applying deep learning across the biosciences. *Nature Communications*, 13(1), 1728.
- Sherstinsky, A. (2020). Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D: Nonlinear Phenomena*, 404, 132306.
- Staudemeyer, R. C., & Morris, E. R. (2019). Understanding LSTM--a tutorial into long short-term memory recurrent neural networks. *arXiv preprint arXiv:1909.09586*.
- Yu, Y., Si, X., Hu, C., & Zhang, J. (2019). A review of recurrent neural networks: LSTM cells and network architectures. *Neural computation*, 31(7), 1235-1270.
- Zhou, J., & Troyanskaya, O. G. (2015). Predicting effects of noncoding variants with deep learning-based sequence model. *Nature methods*, 12(10), 931-934.

How to succeed in this course?

```
from torch import nn

# the neural network
net = nn.Module()
def __init__(self):
    super(Net, self).__init__()
    self.conv1 = nn.Conv2d(3, 6, 5)
    self.pool = nn.MaxPool2d(2, 2)
    self.conv2 = nn.Conv2d(6, 16, 5)
    self.fc1 = nn.Linear(16 * 5 * 5, 120)
    self.fc2 = nn.Linear(120, 84)
    self.fc3 = nn.Linear(84, 10)

def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = x.view(-1, 16 * 5 * 5)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x
```

Practice



Explore

Discuss



Visualize

Ask

